



JobLinker

Project Documentation Book

Connecting Employers & Job Seekers

Student:	Majd Ahmed Majdoub
ID Number:	217229301
School:	Al-Bayan Comprehensive School
Supervisor:	Mr. Iyad Marieh
Track:	Computer Science – Systems Planning
Platform:	Android • Java • Firebase • Agora
Date:	April 16, 2026

Android

Java

Firebase

Agora RTC

Material Design

Table of Contents

Table of Contents	2
Introduction	7
What is JobLinker?	7
Project Background	7
Target Audience	7
Key Problems Solved	7
Quick Info	8
Project Statistics	8
Technology Stack	9
Project Structure	10
Navigation Flow	10
Screen Navigation Flow Diagram	12
UML Class Diagram	13
Part 1 – System Architecture Overview	13
Part 2 – Full Class Diagram	14
Splash Screen	16
Screen Overview	16
Login Screen	17
Screen Overview	17
Register – Step 1	18
Screen Overview	18
Register – Step 2	19
Screen Overview	19

Main – Jobs Screen	20
Screen Overview	20
Job Details	21
Screen Overview	21
Post Job	22
Screen Overview	22
Chats List	23
Screen Overview	23
Chat Screen	24
Screen Overview	24
Voice Call	25
Screen Overview	25
Profile Screen	26
Screen Overview	26
Edit Profile	27
Screen Overview	27
Settings	28
Screen Overview	28
Filter Jobs	29
Screen Overview	29
Saved Jobs	30
Screen Overview	30
User Guide	33
Job Seeker — Screen Navigation Flow	33

Employer — Screen Navigation Flow	36
Features Available by Role	38
User	41
Job	46
Application	52
Conversation	53
Message	55
Call	58
AuthManager	64
DatabaseManager	65
JobLinkerFirebaseManager	67
BaseActivity	92
SplashActivity	93
LoginActivity	95
RegisterActivity	98
MainActivity	101
PostJobActivity	103
JobDetailsActivity	107
ChatActivity	116
CallActivity	128

EditProfileActivity	132
FilterActivity	141
SettingActivity	144
SavedJobsActivity	153
EmailVerificationActivity	155
JobsFragment	159
ChatsFragment	165
ProfileFragment	168
LoginEmailFragment	174
LoginPhoneFragment	178
RegisterStep1Fragment	181
RegisterStep2Fragment	183
RegisterStep3Fragment	187
JobAdapter	193
ConversationAdapter	196
MessageAdapter	198
LoginPagerAdapter	201
RegisterPagerAdapter	202
JobLinkerApplication	205

CallService	207
JobLinkerFireBaseMessagingServices	210
NetworkChangeReceiver	212
BootReceiver	214
ValidationHelper	217
ImageUtils	219
DateTimeHelper	222
LocaleHelper	230
LogoutHelper	233
SharedPreferencesManager	235
ProgressDialogHelper	238
Self-Reflection & Conclusion	242
Personal Reflection	242
Challenges Faced	242
What I Learned	242
Future Improvements	242
References	243

Introduction

What is JobLinker?

JobLinker is a full-featured Android application built to bridge the gap between employers who have job openings and candidates who are actively searching for work. The platform serves as a unified marketplace where both parties can connect, communicate, and complete the hiring process within a single mobile application.

Upon registration, each user selects their account type — **Employer** or **Job Seeker** — and the app customizes its features accordingly. Employers can post vacancies, manage applications, and conduct interviews. Job seekers can browse listings, apply with a cover letter, and communicate directly with hiring managers through real-time chat and voice calls.

Project Background

This project was developed as part of the Computer Science graduation project requirement at Al-Bayan Comprehensive School, specializing in Systems Planning. The idea was motivated by the increasing demand for mobile-first job platforms and the desire to apply modern Android development skills in a real-world context.

Target Audience

- **Employers:** Business owners, HR managers, and recruiters who need to fill open positions quickly and communicate with candidates.
- **Job Seekers:** Students, fresh graduates, and experienced professionals looking for full-time, part-time, or remote job opportunities.
- **Recruiters:** Third-party recruitment agents who manage multiple employer accounts and track candidates across different companies.

Key Problems Solved

- Fragmented communication — JobLinker consolidates job search, application submission, and employer-candidate messaging in one app.
- Slow hiring — Real-time notifications and instant messaging accelerate the hiring timeline.
- Limited reach — Advanced search filters and category-based browsing help both parties find the right match faster.
- Lack of verification — Email and phone OTP verification reduce fake accounts.

Quick Info

Field	Details
Application Name	JobLinker
Platform	Android (API 24 – Android 7.0+)
Language	Java
Backend	Firebase (Auth, Firestore, Storage, FCM)
Real-time Calls	Agora RTC SDK
Architecture	Activity / Fragment (MVVM-inspired)
Screens	20+ Activities and Fragments
User Roles	Employer / Job Seeker
Auth Methods	Email/Password, Google Sign-In, Phone OTP
Message Types	Text, Image, Voice, Document, Location, Contact
Localization	English, Arabic, Hebrew
GitHub	github.com/Majd868/JobLinker
Developer	Majd Ahmed Majdoub • ID: 217229301
Supervisor	Mr. Iyad Marieh
Submission Date	April 16, 2026

Project Statistics

47 Java Source Files	30 XML Layout Files	20+ Screens / Activities / Fragments
7 Firestore Collections	6 Message Types in Chat	3 Registration Steps

Technology Stack

■ Android SDK (Java)	The primary development platform. Java was chosen for its maturity, broad documentation, and excellent support in Android Studio. The app targets API 24 (Android 7.0) and above, covering over 95% of active Android devices.
■ Firebase Authentication	Handles user sign-in and registration via Email/Password, Google Sign-In, and Phone Number (OTP via SMS). Manages session tokens and supports email verification and password reset out of the box.
■ Firebase Cloud Firestore	NoSQL document database for real-time data sync. Stores all app data: users, jobs, messages, conversations, applications, saved jobs, calls, and reports. Real-time listeners power the live chat and job update features.
■ Firebase Cloud Storage	Object storage for user-uploaded files including profile avatars, chat images, voice recordings, and document attachments.
■ Firebase Cloud Messaging (FCM)	Push notification service. Sends notifications for new messages, job applications, and job alerts. Handled by JobLinkerFireBaseMessagingServices.
■ Agora RTC SDK	Real-time communication SDK providing voice and video calling capabilities. Integrated in CallActivity with mute, speaker, and camera toggle controls.
■ Glide	Image loading and caching library by BumpTech. Used throughout the app for loading profile photos and chat images from URLs with placeholder support.
■ Material Design Components	Google's design system providing pre-built UI components: BottomNavigationView, TextInputLayout, ChipGroup, FAB, CardView, TabLayout, ViewPager2, and more.
■ RecyclerView + DiffUtil	Efficient list rendering for jobs, chats, and messages. DiffUtil calculates the minimum changes needed to update the list, avoiding full redraws.
■ ViewPager2 + Fragment	Used in LoginActivity (Email/Phone tabs) and RegisterActivity (3-step form). Provides smooth swipe navigation between fragments.

Project Structure

The JobLinker project follows standard Android Studio project conventions with a clear separation of concerns. The main source code is organized under **app/src/main/java/com/example/joblinker/** in the following packages:

activities/	All Activity classes (screens that occupy the full window). Includes SplashActivity, LoginActivity, RegisterActivity, MainActivity, ChatActivity, CallActivity, PostJobActivity, JobDetailsActivity, EditProfileActivity, FilterActivity, SettingActivity, SavedJobsActivity.
fragments/	Fragment classes hosted inside activities. JobsFragment, ChatsFragment, ProfileFragment, LoginEmailFragment, LoginPhoneFragment, and the 3 RegisterStep fragments.
adapters/	RecyclerView and ViewPager adapters. JobAdapter, ConversationAdapter, MessageAdapter, LoginPagerAdapter, RegisterPagerAdapter.
models/	Plain Java data model classes (POJOs) mapped to Firestore documents. User, Job, Message, Conversation, Application, Call.
firebase/	Firebase abstraction layer. AuthManager (authentication), DatabaseManager (Firestore helpers), JobLinkerFirebaseManager (main singleton manager).
services/	Android services. CallService (foreground service for calls), JobLinkerFireBaseMessagingServices (FCM handler).
receivers/	Broadcast receivers. NetworkChangeReceiver, BootReceiver.
utils/	Utility/helper classes. ValidationHelper, ImageUtils, DateTimeHelper, LocaleHelper, LogoutHelper, SharedPreferencesManager, ProgressDialogHelper.

Navigation Flow

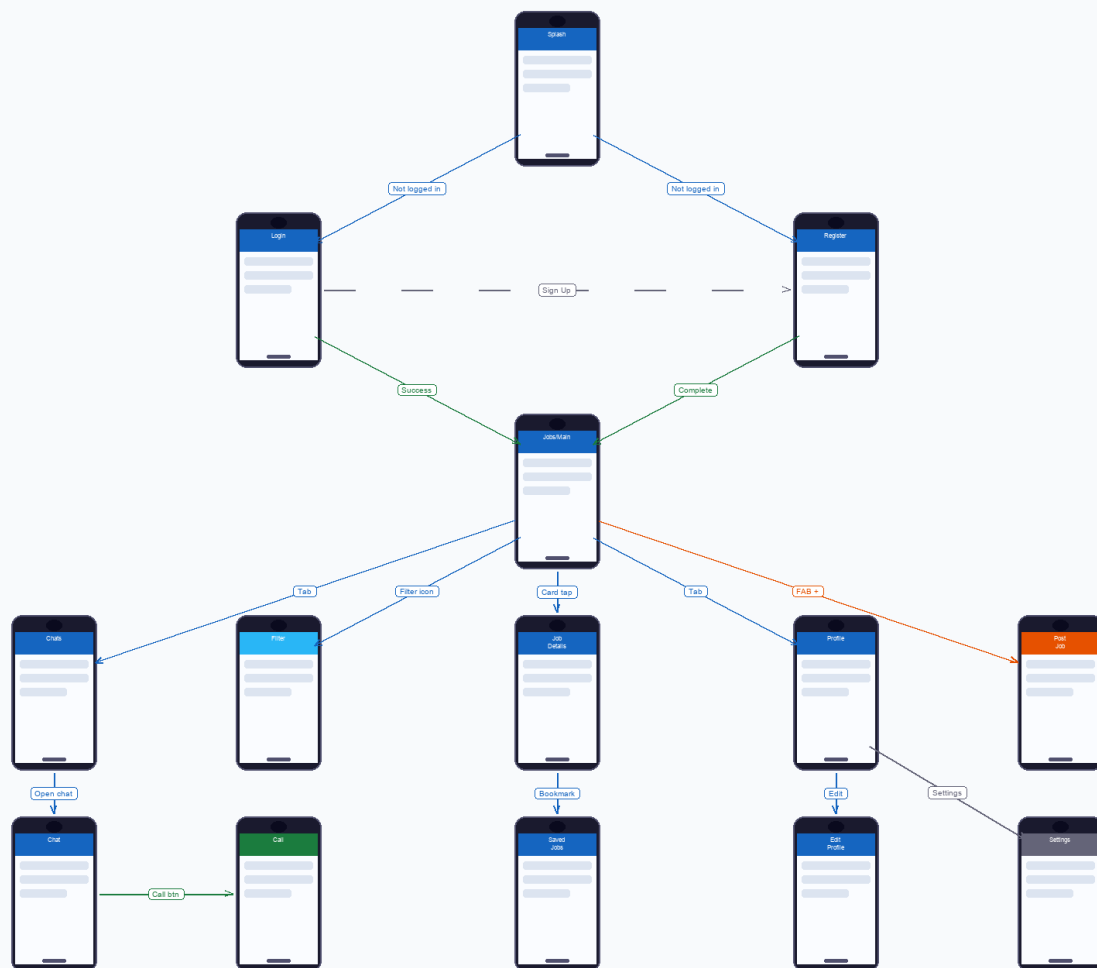
The diagram below shows how users navigate between screens in the app.

From	To	Trigger
SplashActivity	MainActivity	User is logged in (SharedPreferences)
SplashActivity	LoginActivity	User not logged in
LoginActivity	RegisterActivity	'Sign Up' link tapped
LoginActivity	MainActivity	Login success
MainActivity – Jobs	JobDetailsActivity	Job card tapped
MainActivity – Jobs	PostJobActivity	FAB tapped (Employer only)
MainActivity – Jobs	FilterActivity	Filter icon tapped

JobDetailsActivity	ChatActivity	'Contact Employer' button
ChatActivity	CallActivity	Call button in toolbar
MainActivity – Profile	EditProfileActivity	Edit Profile button
MainActivity – Profile	SettingActivity	Settings button
SettingActivity	SavedJobsActivity	'Saved Jobs' row tapped

Screen Navigation Flow Diagram

The diagram below shows how users navigate between all screens in the JobLinker app. Blue arrows represent tab/button navigation, green arrows represent success flows, and orange arrows represent employer-only actions.



Legend:

→ Tab / button navigation

→ Success / call flow

→ Employer only

→ Settings / secondary

UML Class Diagram

Part 1 – System Architecture Overview

The diagram below shows ALL 48 classes in the JobLinker project, organized by package. Dashed arrows indicate dependency relationships between packages. Each package is color-coded by its architectural role: **Blue** = Activities (Controllers), **Green** = Fragments (Views), **Purple** = Models (Entities), **Orange** = Firebase Layer, **Teal** = Adapters, **Red** = Services, **Violet** = Receivers, **Steel Blue** = Utilities.

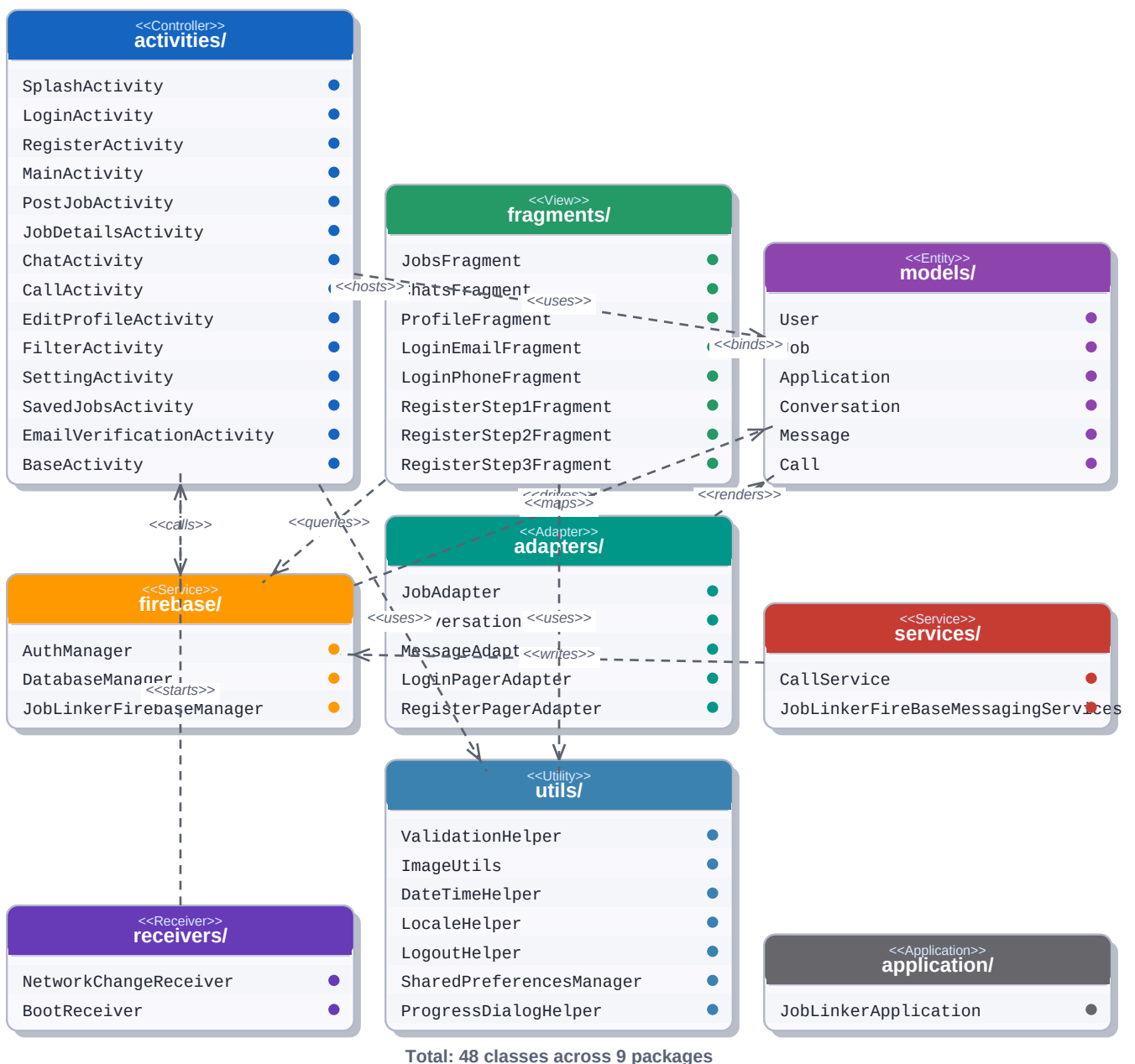


Figure 1: JobLinker Full System Architecture – All 48 Classes (9 packages)

Part 2 – Full Class Diagram

The diagram below shows all major classes across every layer: Firebase/Application, Activities, Fragments, and Data Models. Each box lists the class stereotype, fields (name : type), and methods. **Blue solid lines** = inheritance; **Purple dashed** = dependency/uses; **Orange dashed** = association/manages; **Green dashed** = hosting/contains.

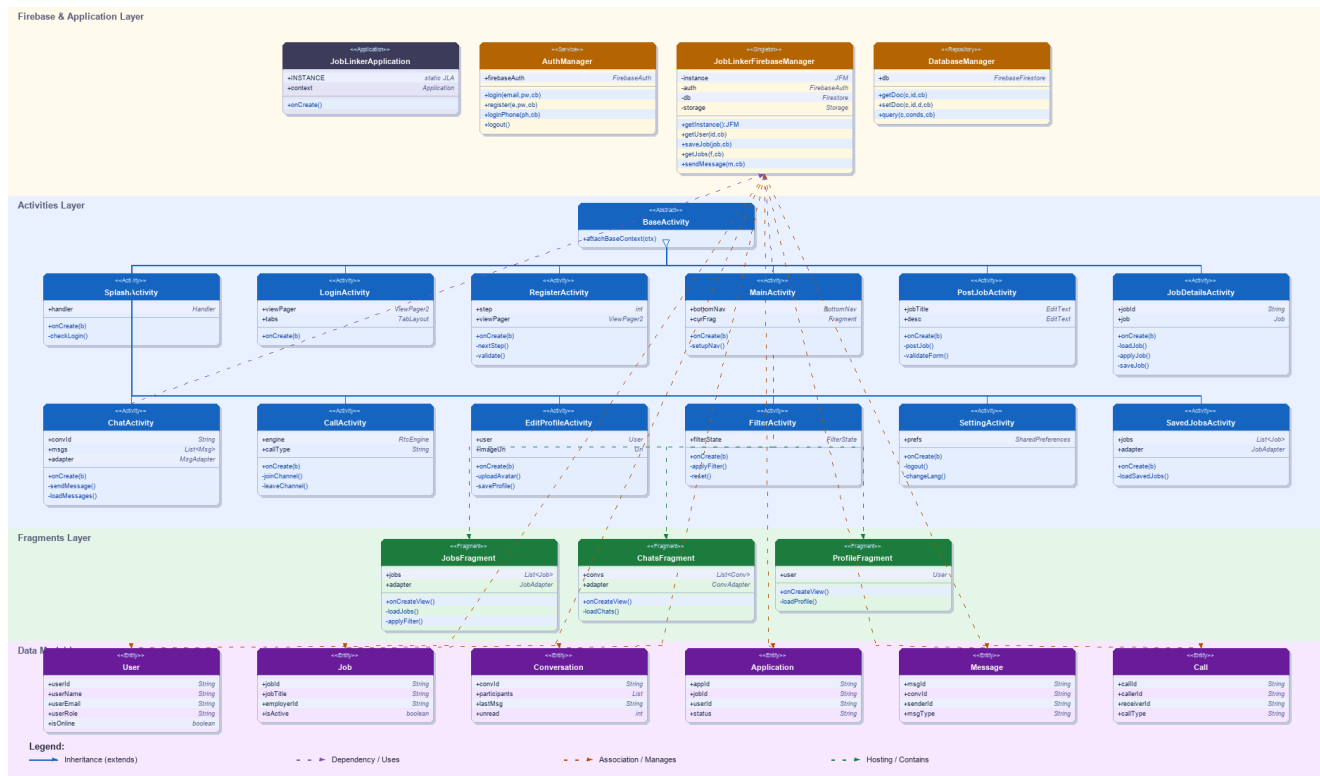


Figure 2: JobLinker Complete UML Class Diagram — 4 Layers, 24 Key Classes

App Screens

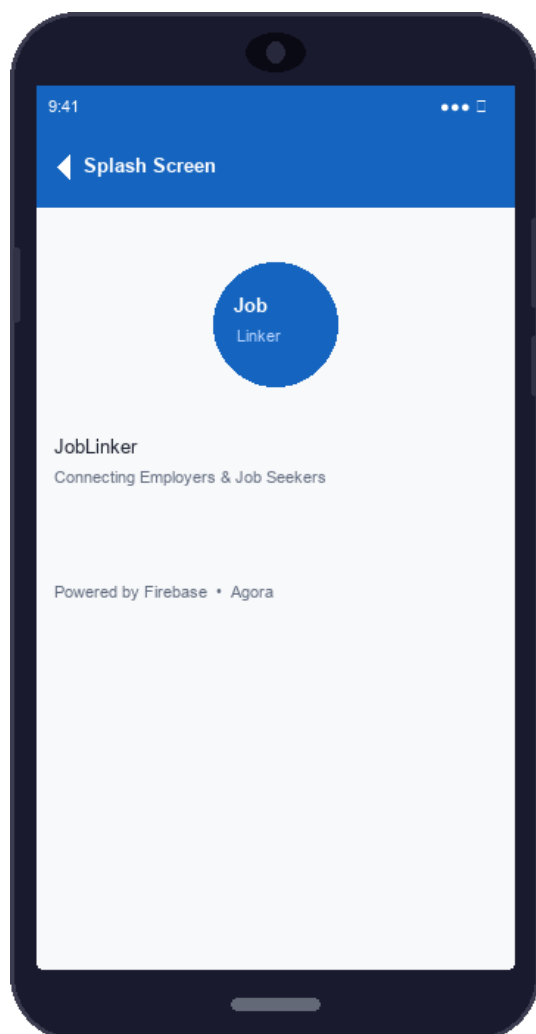
Phone Mockups & UI Element Descriptions

Splash Screen

Source File: `SplashActivity.java`

Screen Overview

The Splash Screen is the first screen displayed when the app launches. It shows the JobLinker logo with a scale and fade-in animation for 2 seconds, then automatically navigates to MainActivity if the user is already logged in, or to LoginActivity if they are not.



UI Element	Description
ImageView – App Logo	Displays the JobLinker logo with scale + fade animation. Centered on screen.
TextView – App Name	Shows 'JobLinker' in large bold white text below the logo.
TextView – Tagline	Subtitle: 'Connecting Employers & Job Seekers' in lighter weight.
Handler.postDelayed (2000ms)	Starts a 2-second timer; on finish, checks SharedPreferences for login status and navigates accordingly.
Background	Gradient background from C_PRIMARY to C_DARK using a GradientDrawable.

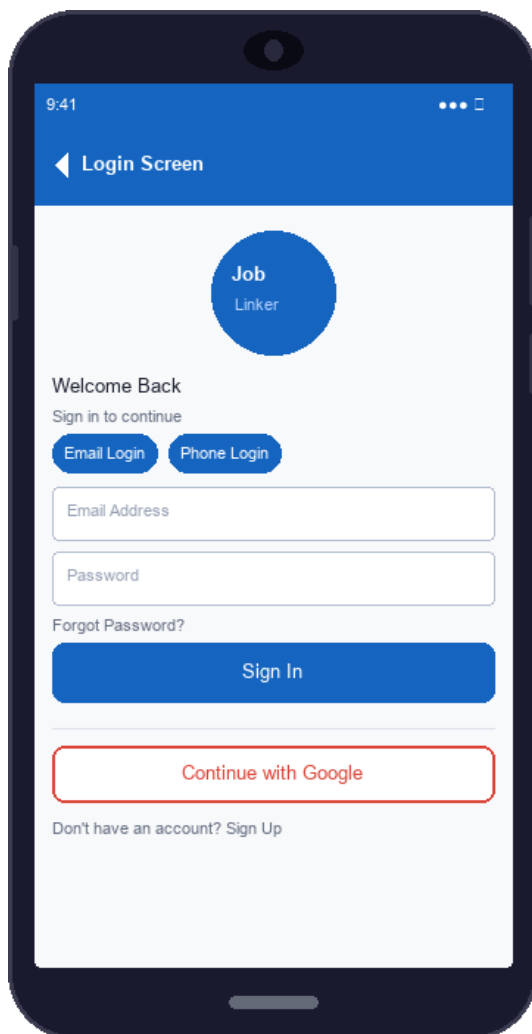
Figure: Splash Screen – Phone Mockup

Login Screen

Source File: `LoginActivity.java`

Screen Overview

The Login Screen provides two login methods accessible via tabs: Email/Password and Phone Number. A `ViewPager2` with a `TabLayout` enables smooth switching between the two methods. A 'Sign Up' link navigates to `RegisterActivity`.



UI Element	Description
ImageView – Logo	App logo at the top for branding.
TabLayout + ViewPager2	Two tabs: 'Email Login' and 'Phone Login'. Hosts <code>LoginEmailFragment</code> and <code>LoginPhoneFragment</code> .
TextInputLayout – Email	Email address input with error validation.
TextInputLayout – Password	Password input with show/hide toggle using <code>PasswordTransformationMethod</code> .
TextView – Forgot Password	Clickable text that triggers <code>Firebase sendPasswordResetEmail()</code> .
Button – Sign In	Calls <code>FirebaseAuth.signInWithEmailAnd Password()</code> on click.
Button – Google Sign-In	Launches Google Sign-In intent via <code>GoogleSignInClient</code> .
TextView – Sign Up	Navigates to <code>RegisterActivity</code> when clicked.
ProgressBar	Shown during authentication; hidden when idle.

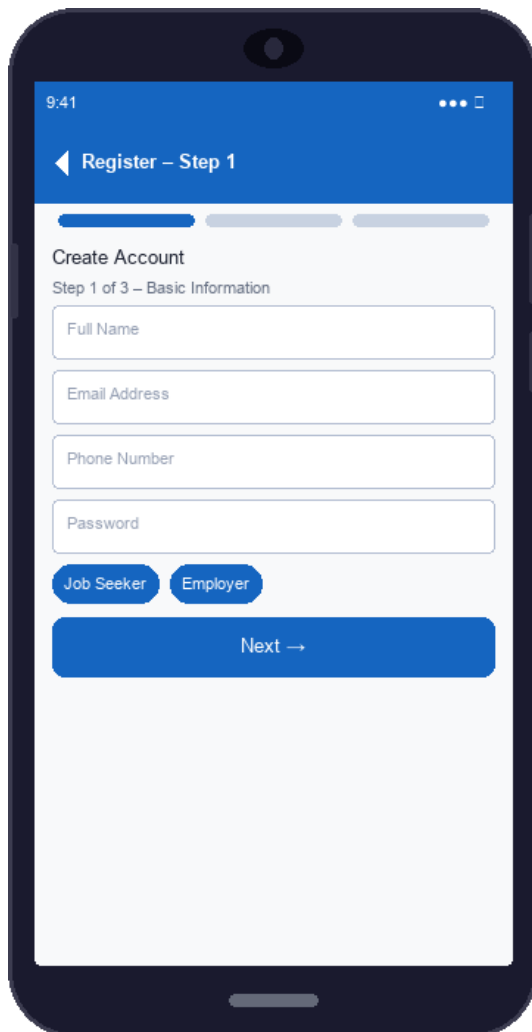
Figure: Login Screen – Phone Mockup

Register – Step 1

Source File: `RegisterActivity.java` / `RegisterStep1Fragment.java`

Screen Overview

The first step of the 3-step registration flow. The user enters their full name, email address, phone number, and password. They also select their account role: Job Seeker or Employer. A progress indicator at the top shows 'Step 1 of 3'.



UI Element	Description
Step Indicator (3 dots)	Visual progress bar showing current registration step (1/3).
TextInputLayout – Full Name	Validated: must not be empty, minimum 2 characters.
TextInputLayout – Email	Validated with Android Patterns.EMAIL_ADDRESS.
TextInputLayout – Phone	Validated: must be 9–15 digits with optional + prefix.
TextInputLayout – Password	Validated: minimum 6 characters. Show/hide toggle.
ToggleButton Group – Role	Job Seeker or Employer selection. Determines which features are shown post-registration.
Button – Next	Validates all fields then calls <code>ViewPager2.setCurrentItem(1)</code> to go to Step 2.

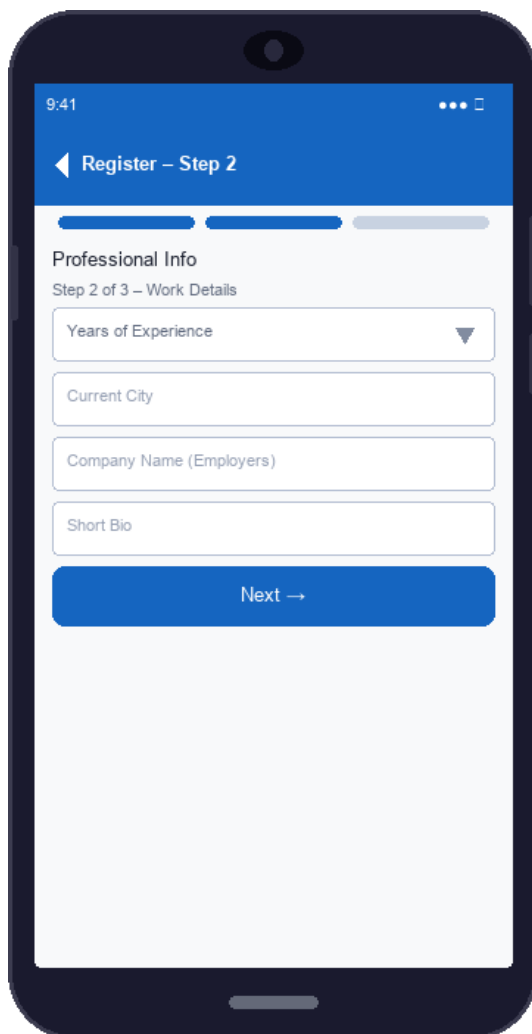
Figure: Register – Step 1 – Phone Mockup

Register – Step 2

Source File: `RegisterStep2Fragment.java`

Screen Overview

The second registration step collects professional information. For Job Seekers: years of experience, current city, and a short bio. For Employers: company name, company website, and industry. The fields shown are dynamic based on the role selected in Step 1.



UI Element	Description
Step Indicator	Shows step 2 of 3 highlighted.
AutoCompleteTextView – Experience	Dropdown for years of experience (e.g., 1–2 years, 3–5 years, etc.).
TextInputLayout – City	User's current city for location-based job filtering.
TextInputLayout – Company Name	Shown only for Employer role. Required if employer.
TextInputLayout – Website	Optional company website URL for employers.
TextInputLayout – Bio	Short biography (max 200 chars). Multi-line input.
Button – Next	Moves to Step 3 (confirmation + Firebase user creation).
Button – Back	Returns to Step 1 without losing entered data.

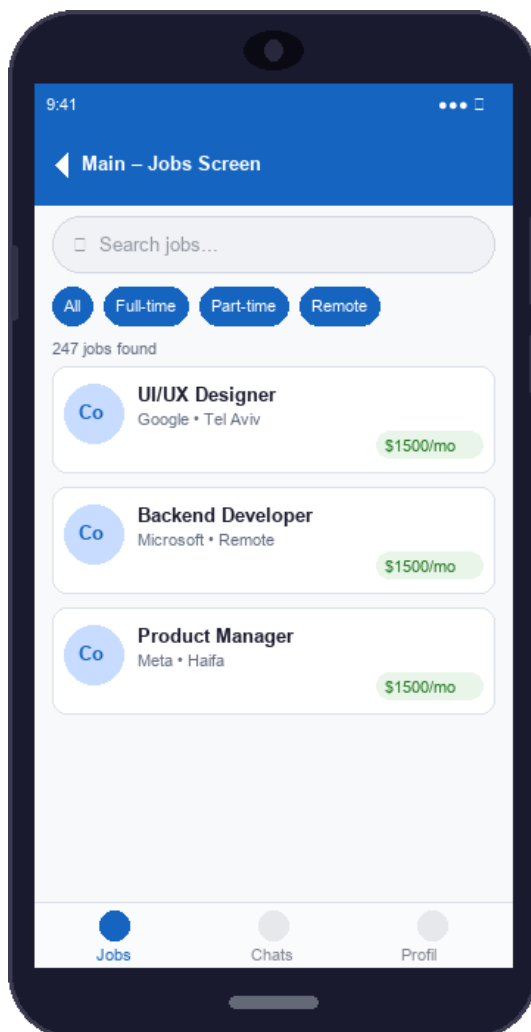
Figure: Register – Step 2 – Phone Mockup

Main – Jobs Screen

Source File: `JobsFragment.java`

Screen Overview

The main browsing screen for job listings. Users can search by keyword, apply quick-filter chips (All, Full-time, Part-time, Remote, etc.), or open the advanced filter dialog. Results update in real-time from Firestore. Employers see a FAB to post a new job.



UI Element	Description
SearchView / TextInputLayout	Search bar at top. Uses 300ms debounce to avoid excessive Firestore queries while typing.
ImageButton – Filter	Opens FilterActivity for advanced filters (category, salary range, location, sort).
ChipGroup – Quick Filters	Chips: All / Full-time / Part-time / Remote / Contract / Internship. Single-select chips that filter job type.
TextView – Result Count	Shows 'X jobs found' dynamically based on active filters.
RecyclerView – Job List	Displays job cards using JobAdapter. Each card shows: company logo, title, company name, location, salary range, bookmark icon.
FloatingActionButton – Post Job	Visible only for Employer role. Opens PostJobActivity.
BottomNavigationView	Three tabs: Jobs (home), Chats (messages icon with badge), Profile.
SwipeRefreshLayout	Pull-to-refresh re-fetches jobs from Firestore.

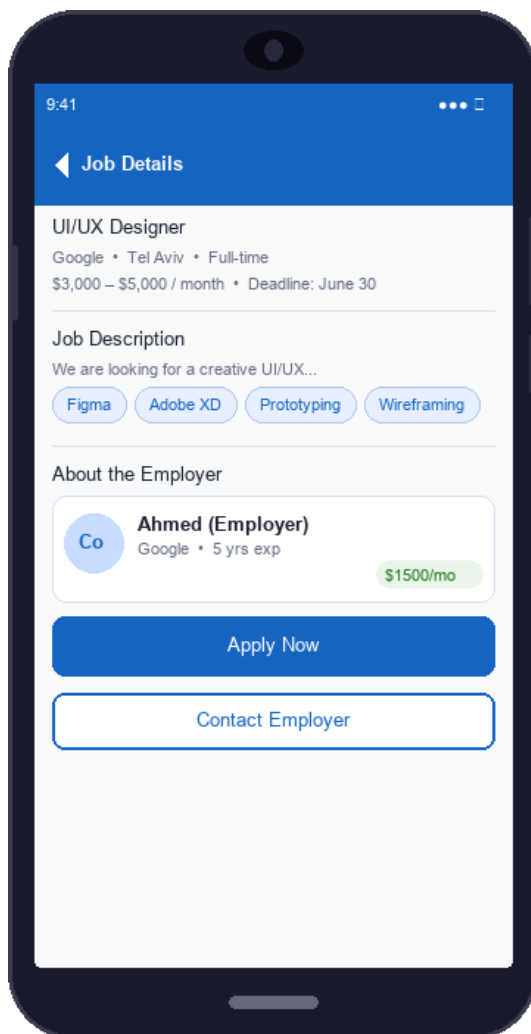
Figure: Main – Jobs Screen – Phone Mockup

Job Details

Source File: `JobDetailsActivity.java`

Screen Overview

Detailed view of a single job posting. Shows company logo, job title, employment type, location, salary range, full description, required skills, and deadline. Job seekers can apply (with cover letter), save the job, or contact the employer directly. View count is tracked.



UI Element	Description
ImageView – Company Logo	Loaded from Firebase Storage using Glide. Shown in a <code>CircleImageView</code> .
TextView – Job Title	Large bold title at the top.
Chip – Employment Type	Badge showing Full-time / Part-time / Remote / etc.
TextView – Location	City and country with a location pin icon.
TextView – Salary Range	Min–Max salary with currency symbol.
TextView – Description	Full multi-line job description. Expandable on 'Read More' tap.
ChipGroup – Required Skills	Dynamic skill chips populated from <code>job.jobSkills</code> list.
CardView – Employer Info	Shows employer avatar, name, company, and experience.
Button – Apply Now	Opens a <code>BottomSheetDialog</code> with a cover letter text field. On submit, writes to 'applications' collection.
Button – Contact	Opens <code>ChatActivity</code> with the employer.
ImageButton – Bookmark	Saves/unsaves the job to 'savedJobs' collection. Toggles icon fill.
MenuItem – Share	Opens Android share sheet with job details as plain text.
MenuItem – Report	Opens a dialog with report reasons; writes to 'reports' collection.

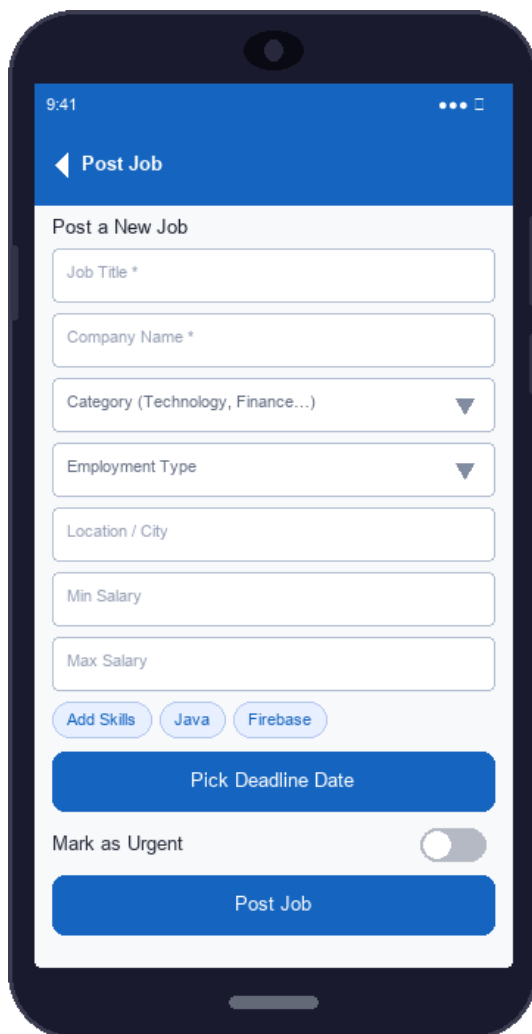
Figure: Job Details – Phone Mockup

Post Job

Source File: `PostJobActivity.java`

Screen Overview

Employers use this screen to post a new job vacancy. Required fields include job title, company name, category, employment type, location, salary range, skills, and deadline. An 'Urgent' toggle marks high-priority listings.



UI Element	Description
TextInputLayout – Job Title	Required. Validated: not empty.
TextInputLayout – Company Name	Required. Pre-filled from the employer's profile if available.
AutoCompleteTextView – Category	Dropdown populated from <code>R.array.job_categories</code> (Technology, Finance, Healthcare, etc.).
AutoCompleteTextView – Type	Employment type: Full-time, Part-time, Remote, Contract, Internship.
TextInputLayout – Location	City or 'Remote'.
TextInputLayout – Min/Max Salary	Numeric fields for salary range. Currency selector next to them.
ChipGroup – Skills	User types a skill and presses Add; a chip with X icon is created dynamically.
Button – Pick Deadline	Opens <code>DatePickerDialog</code> . Minimum date = today.
SwitchCompat – Urgent	Marks job as urgent; shown with a red badge in job cards.
Button – Post Job	Validates all required fields, then writes a new Job document to Firestore. Shows <code>ProgressBar</code> overlay while saving.

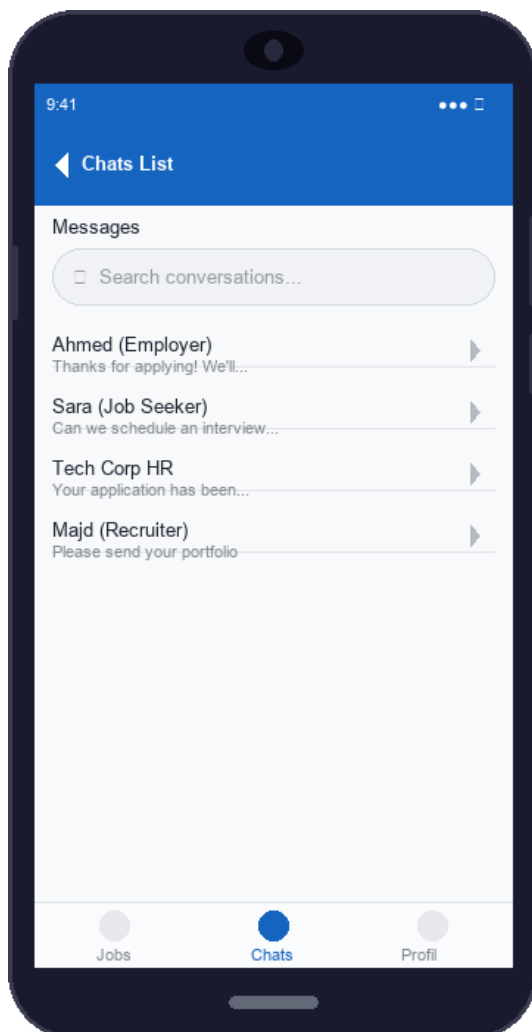
Figure: Post Job – Phone Mockup

Chats List

Source File: `ChatsFragment.java`

Screen Overview

Displays all the user's conversations in a list. Each item shows the other user's avatar, name, online status, last message preview, timestamp, and unread message count badge. A search bar filters conversations by contact name.



UI Element	Description
SearchView	Filters conversation list by the other user's name in real-time.
RecyclerView – Conversation List	Uses ConversationAdapter. Each item is item_conversation.xml.
CircleImageView – Avatar	Other user's profile photo. Default shown if none uploaded.
TextView – Online Indicator	Green dot shown when <code>isOnline == true</code> in Firestore.
TextView – Last Message	Preview of the last message text (truncated to ~40 chars).
TextView – Timestamp	Relative time: 'just now', '5 min ago', 'Yesterday', or date.
Badge – Unread Count	Red bubble with unread message count. Hidden when 0.
BottomNavigationView Badge	Total unread messages shown as a badge on the Chats tab icon.
Empty State View	Shown when no conversations exist yet.

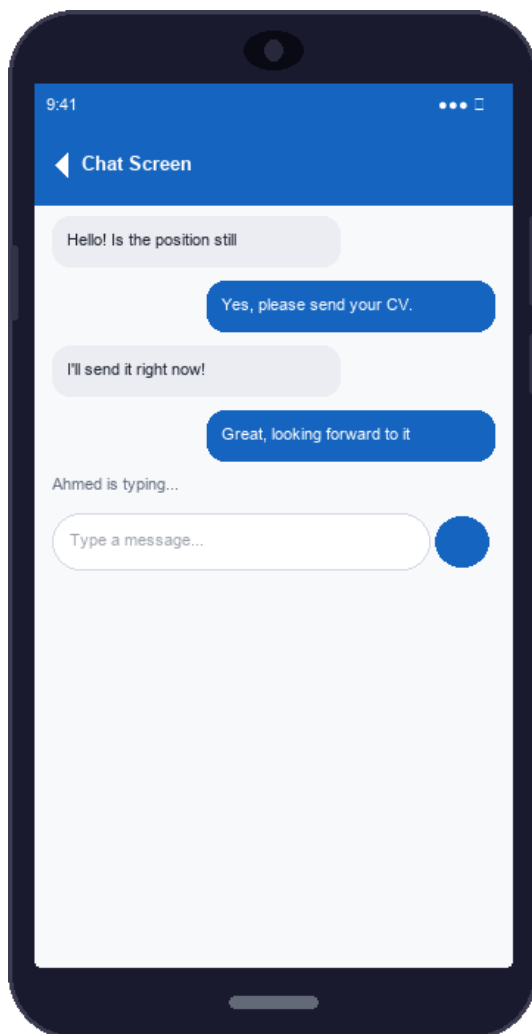
Figure: Chats List – Phone Mockup

Chat Screen

Source File: `ChatActivity.java`

Screen Overview

Full-featured real-time messaging screen between two users. Supports text, images, voice recordings, documents, location, and contact messages. Shows typing indicators, read receipts (single/double checkmarks), and online/offline status in the toolbar.



UI Element	Description
RecyclerView – Messages	Uses MessageAdapter with DiffUtil for efficient updates. Scrolls to bottom on new message.
item_message_send.xml	Outgoing message bubble (blue, right-aligned). Shows timestamp + read receipt checkmarks.
item_message_received.xml	Incoming message bubble (gray, left-aligned). Shows sender avatar + timestamp.
TextInputLayout – Message Input	Message text field with auto-resize and 300-char limit.
ImageButton – Send	Sends a text message to Firestore 'messages' collection.
ImageButton – Attach	Opens attachment tray: Camera, Gallery, Document, Location, Contact.
ImageButton – Voice Record	Hold to record audio via MediaRecorder. Release to send. Shows recording timer.
TextView – Typing Indicator	Updates Firestore 'isTyping' field; other user's client shows animated dots.
Toolbar – Online Status	Shows online/offline dot. Updates via Firestore real-time listener.

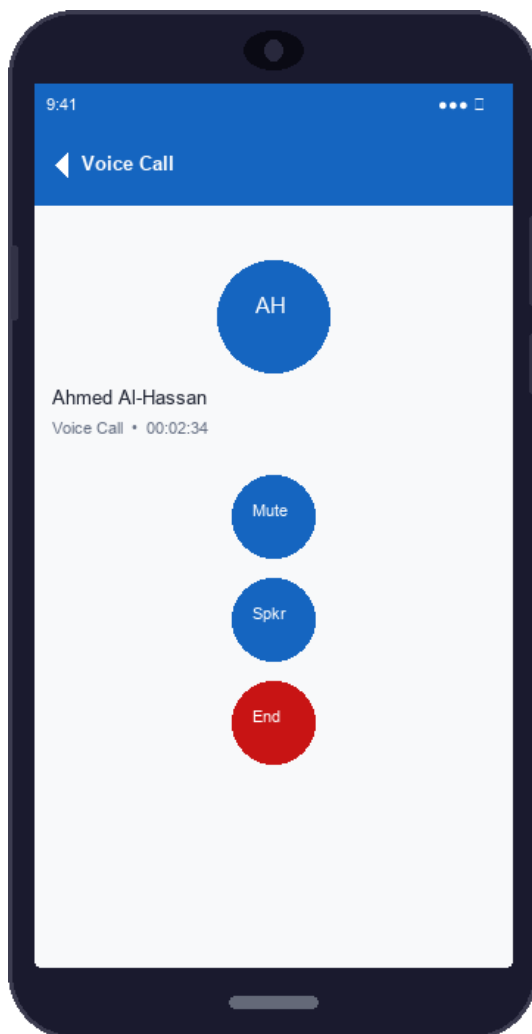
Figure: Chat Screen – Phone Mockup

Voice Call

Source File: `CallActivity.java`

Screen Overview

The call interface for voice and video calls using the Agora RTC SDK. Shows the remote user's avatar and name, a live call timer, and control buttons: Mute, Speaker, End Call, and (for video calls) Camera Toggle. Call details are stored in Firestore for call history.



UI Element	Description
CircleImageView – Remote Avatar	Shows the other user's profile photo (or initials placeholder).
TextView – Call Name	Displays the other user's full name.
TextView – Call Timer	Counts up in HH:MM:SS format using a Handler posted every second.
ImageButton – Mute	Toggles microphone mute. Calls <code>AgoraRtcEngine.muteLocalAudioStream()</code> .
ImageButton – Speaker	Toggles speakerphone. Calls <code>AgoraRtcEngine.setEnableSpeakerphone()</code> .
ImageButton – End Call	Calls <code>AgoraRtcEngine.leaveChannel()</code> , updates call record in Firestore with duration, then finishes the activity.
ImageButton – Camera Toggle	Visible only for video calls. Toggles front/rear camera.
SurfaceView – Local/Remote Video	Used for video calls. Initialized with <code>AgoraRtcEngine.setupLocalVideo()</code> and <code>setupRemoteVideo()</code> .

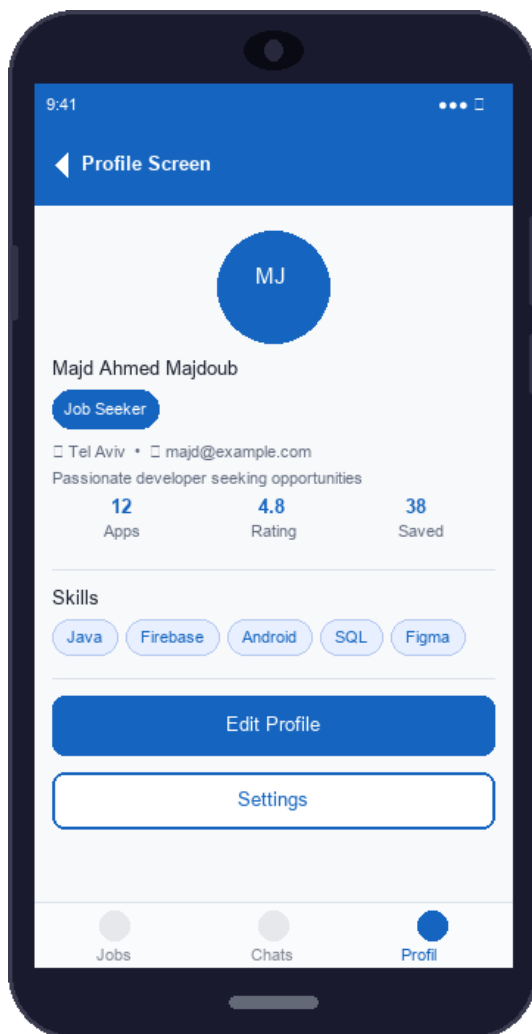
Figure: Voice Call – Phone Mockup

Profile Screen

Source File: **ProfileFragment.java**

Screen Overview

The user's public profile page. Shows avatar, full name, role badge, online indicator, contact info (email, phone, city), a short bio, statistics (jobs posted / applications / connections / rating), and skills chips for Job Seekers. Buttons for Edit Profile, Settings, and Logout.



UI Element	Description
CircleImageView – Avatar	Profile photo with online indicator (green dot overlay).
TextView – Name	User's full name in bold.
Chip – Role Badge	Shows 'Job Seeker' or 'Employer' with corresponding icon and color.
TextView – Bio	Short bio text. Shown only if user has set one.
GridLayout – Stats	3 stats: Applications Sent / Rating / Saved Jobs (job seekers) or Jobs Posted / Applicants / Rating (employers).
RecyclerView – My Jobs	Shown for Employer role only. Lists all jobs posted by this employer.
ChipGroup – Skills	Shown for Job Seeker role. Displays user.userSkills list as chips.
Button – Edit Profile	Opens EditProfileActivity.
Button – Settings	Opens SettingActivity.
Button – Logout	Shows LogoutHelper confirmation dialog. Clears SharedPreferences and returns to LoginActivity.

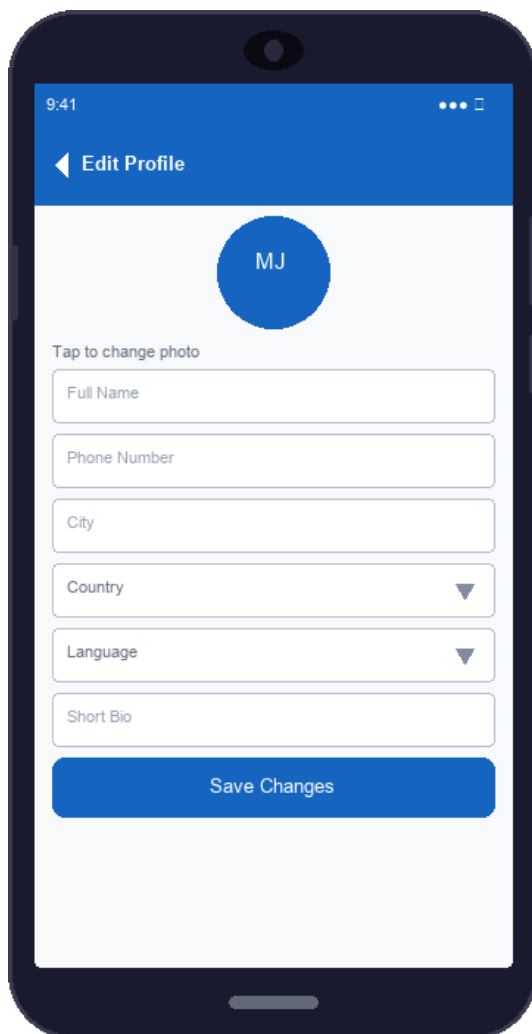
Figure: Profile Screen – Phone Mockup

Edit Profile

Source File: `EditProfileActivity.java`

Screen Overview

Allows users to update their profile information including avatar, name, phone, city, country, language, currency, bio, and skills. The avatar can be changed via camera or gallery. Changes are saved to Firestore and the avatar image is uploaded to Firebase Storage.



UI Element	Description
CircleImageView + FAB	Profile photo with a camera FAB overlay. Tapping opens a BottomSheet: 'Take Photo' or 'Choose from Gallery'.
TextInputLayout – Name	Pre-filled with current name. Validated on save.
TextInputLayout – Phone	Phone number. Validated for correct format.
TextInputLayout – City	Current city. Used in job location matching.
AutoCompleteTextView – Country	Dropdown for country selection from <code>R.array.countries</code> .
AutoCompleteTextView – Language	Preferred language: English, Arabic, Hebrew, etc.
AutoCompleteTextView – Currency	Preferred currency for salary display: ILS, USD, EUR.
TextInputLayout – Bio	Multi-line, max 200 characters.
Button – Save Changes	Validates, uploads photo if changed (shows <code>ProgressBar</code>), then updates Firestore user document.

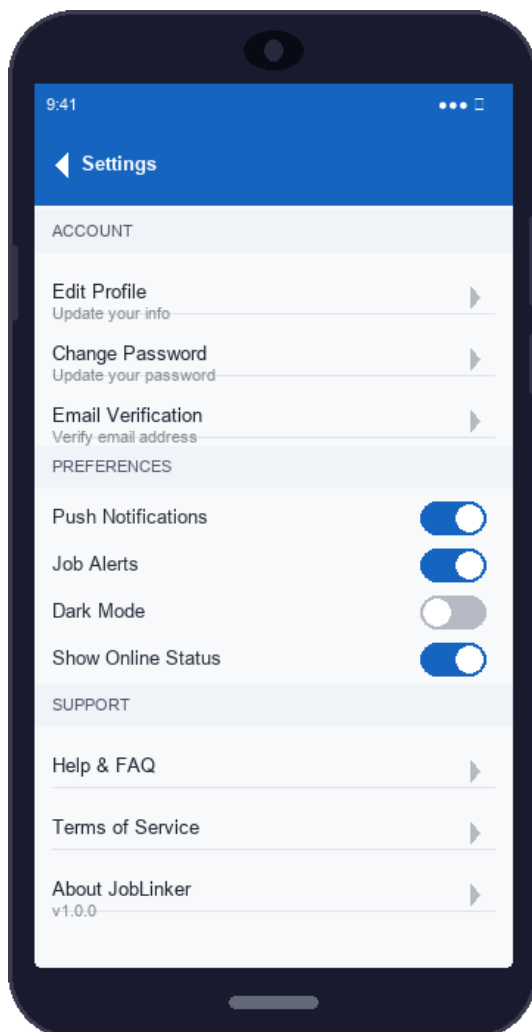
Figure: Edit Profile – Phone Mockup

Settings

Source File: `SettingActivity.java`

Screen Overview

The Settings screen is organized into sections: Account (edit profile, change password, email verification), Job Tools (saved jobs, my applications), Preferences (notifications, dark mode, online status), Privacy, and Support. A Danger Zone section at the bottom offers Delete Account and Logout.



UI Element	Description
Section Headers	Visually separate setting groups: Account / Job Tools / Preferences / Privacy / Support / Danger Zone.
TextView – Edit Profile	Navigates to EditProfileActivity.
TextView – Change Password	Opens dialog_change_password.xml: current password + new password fields.
TextView – Email Verification	Sends Firebase verification email. Disabled if already verified.
TextView – Saved Jobs	Navigates to SavedJobsActivity (Job Seeker only).
SwitchCompat – Notifications	Toggle push notifications. Stored in SharedPreferences.
SwitchCompat – Job Alerts	Toggle job-match notifications from FCM.
SwitchCompat – Dark Mode	Calls AppCompatActivity.setDefaultNightMode(). Instantly applies.
SwitchCompat – Online Status	When off, sets user.isOnline = false in Firestore.
TextView – Delete Account	Shows delete_confirmation_dialog.xml. On confirm, calls FirebaseAuth.deleteUser() and removes Firestore data.
TextView – Logout	Calls LogoutHelper.showLogoutDialog() for confirmation.

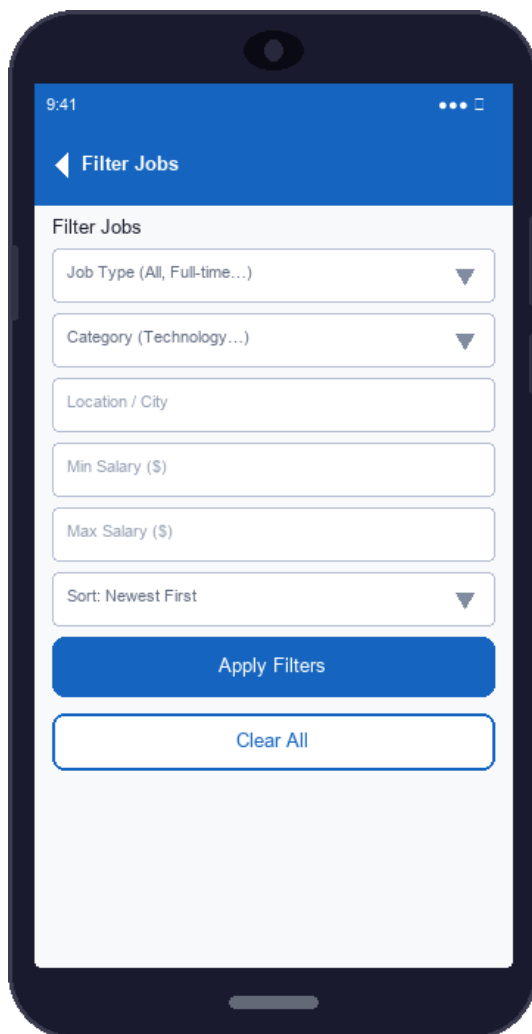
Figure: Settings – Phone Mockup

Filter Jobs

Source File: `FilterActivity.java`

Screen Overview

Advanced filter screen launched from the Jobs screen. Users can filter by job type (dropdown), category (dropdown), location (text input), minimum and maximum salary, and sort order. Results are passed back to JobsFragment via `ActivityResultLauncher`.



UI Element	Description
AutoCompleteTextView – Job Type	Filter by All / Full-time / Part-time / Remote / Contract / Internship.
AutoCompleteTextView – Category	Filter by job category from <code>R.array.job_categories</code> .
TextInputLayout – Location	Free-text location filter (case-insensitive substring match).
TextInputLayout – Min Salary	Numeric filter. Jobs with salary $\geq$ this value are shown.
TextInputLayout – Max Salary	Numeric filter. Jobs with salary $\leq$ this value are shown.
AutoCompleteTextView – Sort	Sort order: Newest First or Oldest First.
Button – Apply	Builds a <code>FilterState</code> object and returns it to JobsFragment via <code>setResult(RESULT_OK, intent)</code> .
Button – Clear	Resets all filter fields to default values.

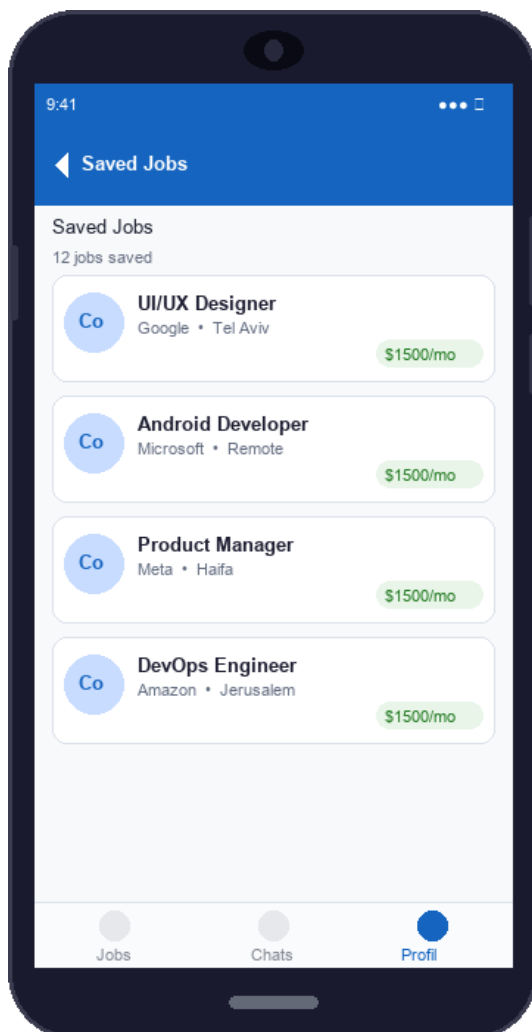
Figure: Filter Jobs – Phone Mockup

Saved Jobs

Source File: `SavedJobsActivity.java`

Screen Overview

Displays all jobs the user has bookmarked using the save (bookmark) icon in job cards or on the Job Details screen. Each item shows the job title, company, location, and salary. Tapping a card opens `JobDetailsActivity`. An empty-state view is shown when no jobs have been saved yet.



UI Element	Description
Toolbar – 'Saved Jobs'	Back arrow + title. Standard AppCompatActivity toolbar.
TextView – Count	Shows 'X jobs saved' subtitle below the toolbar.
RecyclerView – Job List	Uses the same JobAdapter as the main Jobs screen. Displays bookmarked jobs fetched from Firestore 'savedJobs' collection joined with 'jobs'.
item_job.xml – Job Card	Company logo (Glide), job title, company name, location, salary range, bookmark icon (pre-filled).
Empty State View	Shown when the savedJobs list is empty. ImageView + 'No saved jobs yet' message + 'Browse Jobs' button.
JobAdapter.OnJob ClickListener	Opens JobDetailsActivity when a card is tapped.

Figure: Saved Jobs – Phone Mockup

User Guide

Step-by-Step Instructions for Job Seekers & Employers

User Guide

This section explains how to use JobLinker for both **Job Seekers** and **Employers**. Each diagram shows every screen as a phone mockup with a **unique-colored arrow** for each navigation path. The legend below each diagram identifies every arrow.

Job Seeker — Screen Navigation Flow

Follow the numbered screens from Splash to Call. Each arrow has a different color — see the legend at the bottom.

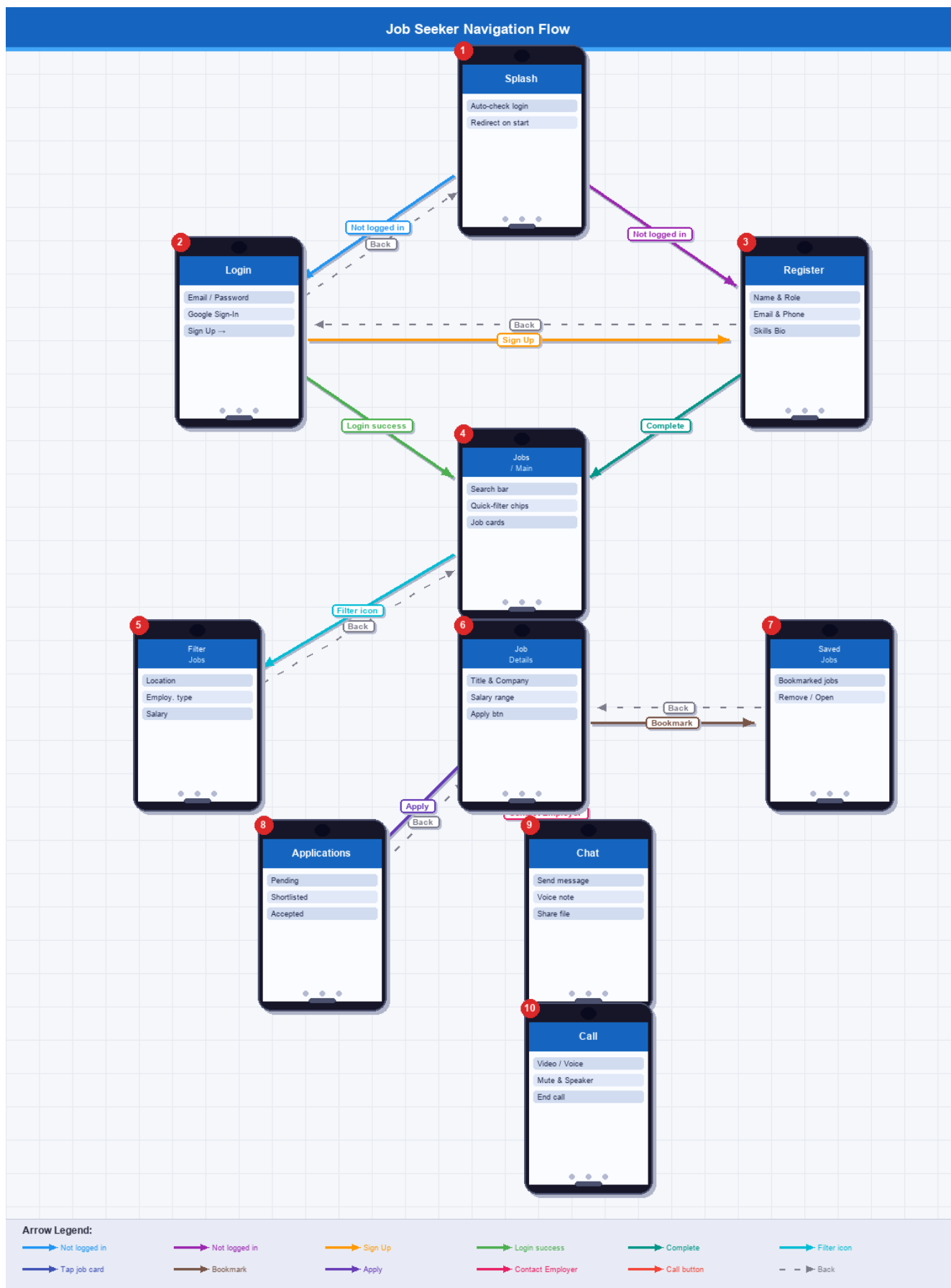


Figure: Job Seeker flow — 10 screens, 11 uniquely colored arrows. Numbers = step order. See arrow legend for transition labels.

1. Splash Screen

App launches SplashActivity. It automatically checks if you are already logged in (stored in SharedPreferences). If yes → skips to Jobs/Main. If not → redirects to Login.

2. Login / 3. Register

LoginActivity offers Email+Password or Google Sign-In. New users tap 'Sign Up' (orange arrow) to open RegisterActivity — a 3-step wizard: name & role → email & phone → skills & bio.

4. Jobs / Main

MainActivity opens on the Jobs tab. A real-time search bar and quick-filter chips let you narrow results instantly. Pull to refresh for new listings.

5. Filter Jobs

Tap the filter icon (cyan arrow) to open FilterActivity. Select location, employment type, and salary range, then tap Apply. Results update immediately in JobsFragment.

6. Job Details

Tap any job card (indigo arrow) to open JobDetailsActivity. Read the full description and required skills. Tap 'Save' (brown arrow) to bookmark, or 'Apply' (deep purple) to submit.

7. Applications

After applying, your application is tracked. Employers update the status to Shortlisted, Accepted, or Rejected. You receive a push notification on each status change.

8. Chat → 9. Call

Tap 'Contact Employer' (pink arrow) on Job Details to open ChatActivity. Send text, voice notes, images, or files. Tap the call icon (red arrow) to start a voice or video call via Agora RTC.

Employer — Screen Navigation Flow

Employers share Login, Register, and Chat/Call screens with Job Seekers, but gain exclusive access to Post Job and Applications Review.

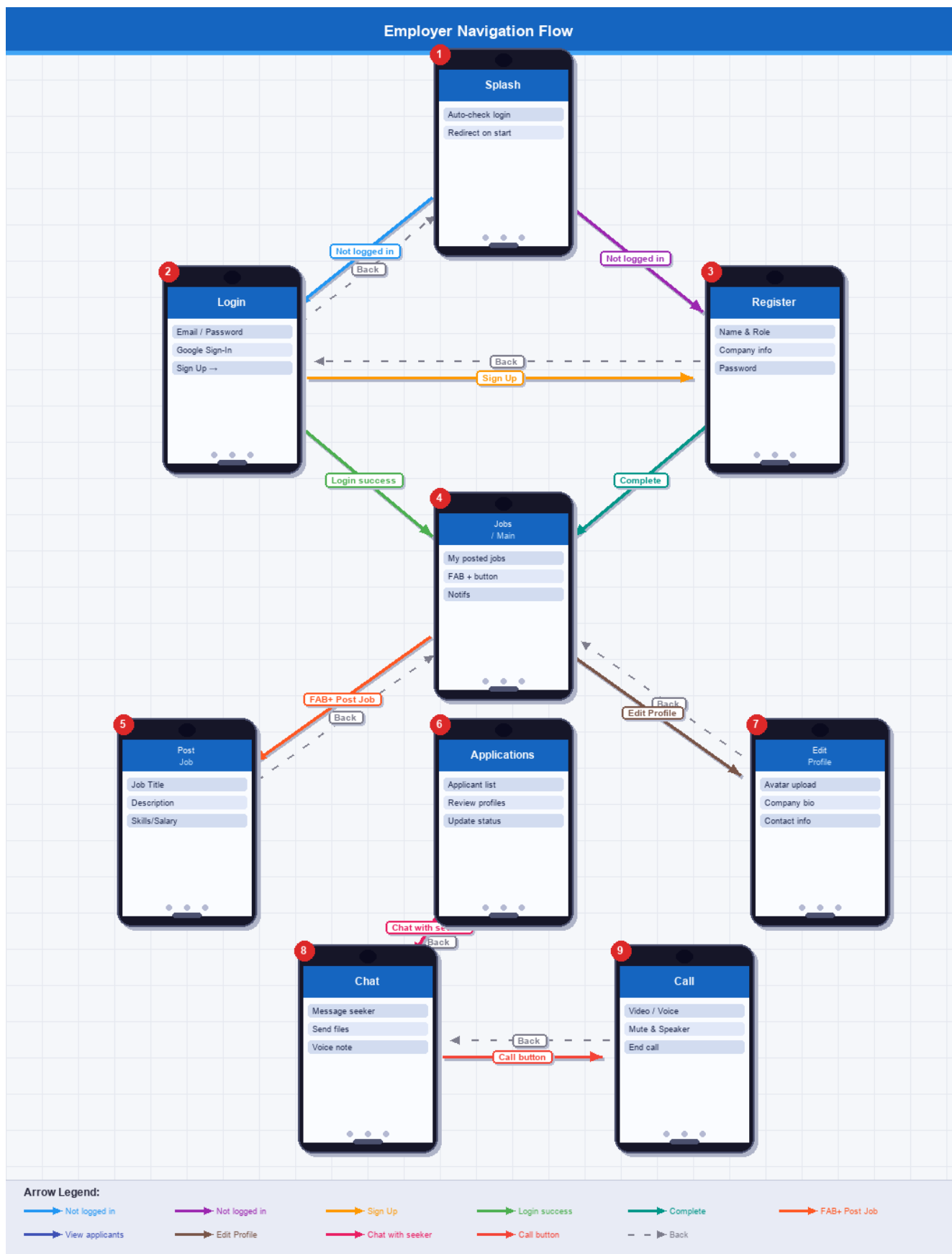


Figure: Employer flow — 9 screens, 10 uniquely colored arrows. Deep orange = Employer-only FAB. See arrow legend for all transition labels.

1. Splash → Login / Register

Same entry flow as Job Seeker. On RegisterActivity Step 1, select role = Employer to unlock employer-specific features.

2. Jobs / Main — Employer View

MainActivity shows your posted jobs list instead of all jobs. An orange FAB '+' button is visible only to Employers. Notification badges show new applicants on each job card.

3. Post a Job

Tap FAB '+' (deep orange arrow) to open PostJobActivity. Fill in title, description, required skills (chips), employment type, salary range, and location. Tap 'Post Job' — published instantly to Firestore.

4. Review Applications

Tap any job card to see its Applications list (indigo arrow). Review each applicant's profile, skills, and cover letter. Update status to Shortlisted, Accepted, or Rejected — applicant is notified.

5. Chat & Call Applicants

Tap 'Chat' on any applicant (pink arrow) to open ChatActivity. Discuss the role, request documents, or send voice notes. Tap the call icon (red arrow) to start a voice or video call.

6. Edit Profile

Tap Edit Profile (brown arrow) from Jobs/Main to open EditProfileActivity. Upload a company logo, update the bio, and save contact details.

Features Available by Role

Feature	Job Seeker	Employer
Browse Job Listings	Yes	Yes
Apply for Jobs	Yes	No
Save / Bookmark Jobs	Yes	No
Post Job Listings	No	Yes
Review Applications	No	Yes
Real-time Chat	Yes	Yes
Voice / Video Calls	Yes	Yes
Edit Profile & Avatar	Yes	Yes
Filter Job Search	Yes	Yes
Push Notifications	Yes	Yes

Source Code

Models – Data Classes

User

Represents a registered user in the system. Used for both Job Seeker and Employer accounts. Stored in the 'users' Firestore collection.

User.java (Java)

```
package com.example.joblinker.models;
import com.google.firebase.firestore.PropertyName;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;
public class User implements Serializable {
    private String userId;
    private String userName;
    private String userEmail;
    private String userPhone;
    private String userRole; // "JobSeeker" or "Employer"
    private String userCountry;
    private String userCity;
    private String userLanguage; // "en", "ar", "he"
    private String userCurrency; // "USD", "EUR", "ILS", etc.
    private String userBio;
    private List<String> userSkills;
    private String userExperience;
    private String companyName;
    private String companyWebsite;
    private String dateOfBirth;
    private String avatarUrl;
    private boolean emailVerified;
    private boolean phoneVerified;
    private boolean isOnline;
    private long lastSeen;
    private long createdAt;
    public User() {
        this.userSkills = new ArrayList<>();
        this.createdAt = System.currentTimeMillis();
        this.lastSeen = System.currentTimeMillis();
        this.emailVerified = false;
        this.phoneVerified = false;
        this.isOnline = false;
        this.userRole = "JobSeeker";
        this.userLanguage = "en";
        this.userCurrency = "USD";
    }
    public User(String userId, String userName, String userEmail, String userRole) {
        this();
        this.userId = userId;
        this.userName = userName;
        this.userEmail = userEmail;
        this.userRole = userRole;
    }
    // Getters and Setters with PropertyName annotations
    @PropertyName("userId")
    public String getUserId() {
        return userId;
    }
    @PropertyName("userId")
    public void setUserId(String userId) {
        this.userId = userId;
    }
}
```

```
}
@PropertyName("userName")
public String getUserName() {
    return userName;
}
@PropertyName("userName")
public void setUserName(String userName) {
    this.userName = userName;
}
@PropertyName("userEmail")
public String getUserEmail() {
    return userEmail;
}
@PropertyName("userEmail")
public void setUserEmail(String userEmail) {
    this.userEmail = userEmail;
}
@PropertyName("userPhone")
public String getUserPhone() {
    return userPhone;
}
@PropertyName("userPhone")
public void setUserPhone(String userPhone) {
    this.userPhone = userPhone;
}
@PropertyName("userRole")
public String getUserRole() {
    return userRole;
}
@PropertyName("userRole")
public void setUserRole(String userRole) {
    this.userRole = userRole;
}
@PropertyName("userCountry")
public String getUserCountry() {
    return userCountry;
}
@PropertyName("userCountry")
public void setUserCountry(String userCountry) {
    this.userCountry = userCountry;
}
@PropertyName("userCity")
public String getUserCity() {
    return userCity;
}
@PropertyName("userCity")
public void setUserCity(String userCity) {
    this.userCity = userCity;
}
@PropertyName("userLanguage")
public String getUserLanguage() {
    return userLanguage;
}
@PropertyName("userLanguage")
public void setUserLanguage(String userLanguage) {
    this.userLanguage = userLanguage;
}
@PropertyName("userCurrency")
public String getUserCurrency() {
    return userCurrency;
}
@PropertyName("userCurrency")
public void setUserCurrency(String userCurrency) {
    this.userCurrency = userCurrency;
}
```

```
}
@PropertyName("userBio")
public String getUserBio() {
    return userBio;
}
@PropertyName("userBio")
public void setUserBio(String userBio) {
    this.userBio = userBio;
}
@PropertyName("userSkills")
public List<String> getUserSkills() {
    return userSkills;
}
@PropertyName("userSkills")
public void setUserSkills(List<String> userSkills) {
    this.userSkills = userSkills;
}
@PropertyName("userExperience")
public String getUserExperience() {
    return userExperience;
}
@PropertyName("userExperience")
public void setUserExperience(String userExperience) {
    this.userExperience = userExperience;
}
@PropertyName("companyName")
public String getCompanyName() {
    return companyName;
}
@PropertyName("companyName")
public void setCompanyName(String companyName) {
    this.companyName = companyName;
}
@PropertyName("companyWebsite")
public String getCompanyWebsite() {
    return companyWebsite;
}
@PropertyName("companyWebsite")
public void setCompanyWebsite(String companyWebsite) {
    this.companyWebsite = companyWebsite;
}
@PropertyName("dateOfBirth")
public String getDateOfBirth() {
    return dateOfBirth;
}
@PropertyName("dateOfBirth")
public void setDateOfBirth(String dateOfBirth) {
    this.dateOfBirth = dateOfBirth;
}
@PropertyName("avatarUrl")
public String getAvatarUrl() {
    return avatarUrl;
}
@PropertyName("avatarUrl")
public void setAvatarUrl(String avatarUrl) {
    this.avatarUrl = avatarUrl;
}
@PropertyName("emailVerified")
public boolean isEmailVerified() {
    return emailVerified;
}
@PropertyName("emailVerified")
public void setEmailVerified(boolean emailVerified) {
    this.emailVerified = emailVerified;
}
```

```

    }
    @PropertyName("phoneVerified")
    public boolean isPhoneVerified() {
        return phoneVerified;
    }
    @PropertyName("phoneVerified")
    public void setPhoneVerified(boolean phoneVerified) {
        this.phoneVerified = phoneVerified;
    }
    @PropertyName("isOnline")
    public boolean isOnline() {
        return isOnline;
    }
    @PropertyName("isOnline")
    public void setOnline(boolean online) {
        isOnline = online;
    }
    @PropertyName("lastSeen")
    public long getLastSeen() {
        return lastSeen;
    }
    @PropertyName("lastSeen")
    public void setLastSeen(long lastSeen) {
        this.lastSeen = lastSeen;
    }
    @PropertyName("createdAt")
    public long getCreatedAt() {
        return createdAt;
    }
    @PropertyName("createdAt")
    public void setCreatedAt(long createdAt) {
        this.createdAt = createdAt;
    }
    // Helper methods
    public boolean isEmployer() {
        return "Employer".equals(userRole);
    }
    public boolean isJobSeeker() {
        return "JobSeeker".equals(userRole);
    }
    public void addSkill(String skill) {
        if (userSkills == null) {
            userSkills = new ArrayList<>();
        }
        if (!userSkills.contains(skill)) {
            userSkills.add(skill);
        }
    }
    public void removeSkill(String skill) {
        if (userSkills != null) {
            userSkills.remove(skill);
        }
    }
    public String getLocation() {
        if (userCity != null && userCountry != null) {
            return userCity + ", " + userCountry;
        } else if (userCountry != null) {
            return userCountry;
        } else if (userCity != null) {
            return userCity;
        }
        return "";
    }
    @Override

```

```
public String toString() {  
    return "User{" +  
        "userId='" + userId + '\'' +  
        ", userName='" + userName + '\'' +  
        ", userEmail='" + userEmail + '\'' +  
        ", userRole='" + userRole + '\'' +  
        '}';  
}  
}
```

Job

Represents a job posting created by an Employer. Contains all job details including skills, salary range, and applicant tracking. Stored in 'jobs' collection.

Job.java (Java)

```
package com.example.joblinker.models;
import java.io.Serializable;
import java.util.List;
public class Job implements Serializable {
    private String jobId;
    private String jobTitle;
    private String jobCompany;
    private String jobEmployerId;
    private String jobCategory;
    private String jobType; // Full-time, Part-time, Remote, Contract, Internship
    private String employmentType; // On-site, Hybrid, Remote (work arrangement)
    private String jobCountry;
    private String jobCity;
    private String jobDescription;
    private List<String> jobSkills;
    private double jobSalaryMin;
    private double jobSalaryMax;
    private String salaryCurrency;
    private long deadline; // Application deadline timestamp
    private long createdAt;
    private long updatedAt;
    private boolean jobActive;
    private int viewCount;
    private int applicantCount;
    private List<String> applicants; // List of user IDs who applied
    private String companyLogoUrl; // Company logo image URL
    private String companyWebsite;
    private String contactEmail;
    private String contactPhone;
    // Default Constructor (Required for Firebase)
    public Job() {
        this.jobActive = true;
        this.createdAt = System.currentTimeMillis();
        this.updatedAt = System.currentTimeMillis();
        this.viewCount = 0;
        this.applicantCount = 0;
        this.salaryCurrency = "USD";
    }
    // Constructor with essential fields
    public Job(String jobTitle, String jobCompany, String jobEmployerId,
        String jobCategory, String jobType) {
        this();
        this.jobTitle = jobTitle;
        this.jobCompany = jobCompany;
        this.jobEmployerId = jobEmployerId;
        this.jobCategory = jobCategory;
        this.jobType = jobType;
    }
    // ===== GETTERS =====
    public String getJobId() {
        return jobId;
    }
    public String getJobTitle() {
```

```
return jobTitle;
}
public String getJobCompany() {
return jobCompany;
}
public String getJobEmployerId() {
return jobEmployerId;
}
public String getJobCategory() {
return jobCategory;
}
public String getJobType() {
return jobType;
}
public String getEmploymentType() {
return employmentType;
}
public String getJobCountry() {
return jobCountry;
}
public String getJobCity() {
return jobCity;
}
public String getJobDescription() {
return jobDescription;
}
public List<String> getJobSkills() {
return jobSkills;
}
public double getJobSalaryMin() {
return jobSalaryMin;
}
public double getJobSalaryMax() {
return jobSalaryMax;
}
public String getSalaryCurrency() {
return salaryCurrency;
}
public long getDeadline() {
return deadline;
}
public long getCreatedAt() {
return createdAt;
}
public long getUpdatedAt() {
return updatedAt;
}
public boolean isJobActive() {
return jobActive;
}
public int getViewCount() {
return viewCount;
}
public int getApplicantCount() {
return applicantCount;
}
public List<String> getApplicants() {
return applicants;
}
public String getCompanyLogoUrl() {
return companyLogoUrl;
}
public String getCompanyWebsite() {
return companyWebsite;
}
```

```

    }
    public String getContactEmail() {
        return contactEmail;
    }
    public String getContactPhone() {
        return contactPhone;
    }
    // ===== SETTERS =====
    public void setJobId(String jobId) {
        this.jobId = jobId;
    }
    public void setJobTitle(String jobTitle) {
        this.jobTitle = jobTitle;
    }
    public void setJobCompany(String jobCompany) {
        this.jobCompany = jobCompany;
    }
    public void setJobEmployerId(String jobEmployerId) {
        this.jobEmployerId = jobEmployerId;
    }
    public void setJobCategory(String jobCategory) {
        this.jobCategory = jobCategory;
    }
    public void setJobType(String jobType) {
        this.jobType = jobType;
    }
    public void setEmploymentType(String employmentType) {
        this.employmentType = employmentType;
    }
    public void setJobCountry(String jobCountry) {
        this.jobCountry = jobCountry;
    }
    public void setJobCity(String jobCity) {
        this.jobCity = jobCity;
    }
    public void setJobDescription(String jobDescription) {
        this.jobDescription = jobDescription;
    }
    public void setJobSkills(List<String> jobSkills) {
        this.jobSkills = jobSkills;
    }
    public void setJobSalaryMin(double jobSalaryMin) {
        this.jobSalaryMin = jobSalaryMin;
    }
    public void setJobSalaryMax(double jobSalaryMax) {
        this.jobSalaryMax = jobSalaryMax;
    }
    public void setSalaryCurrency(String salaryCurrency) {
        this.salaryCurrency = salaryCurrency;
    }
    public void setDeadline(long deadline) {
        this.deadline = deadline;
    }
    public void setCreatedAt(long createdAt) {
        this.createdAt = createdAt;
    }
    public void setUpdatedAt(long updatedAt) {
        this.updatedAt = updatedAt;
    }
    public void setJobActive(boolean jobActive) {
        this.jobActive = jobActive;
    }
    public void setViewCount(int viewCount) {
        this.viewCount = viewCount;
    }

```



```

}
public void setApplicantCount(int applicantCount) {
    this.applicantCount = applicantCount;
}
public void setApplicants(List<String> applicants) {
    this.applicants = applicants;
}
public void setCompanyLogoUrl(String companyLogoUrl) {
    this.companyLogoUrl = companyLogoUrl;
}
public void setCompanyWebsite(String companyWebsite) {
    this.companyWebsite = companyWebsite;
}
public void setContactEmail(String contactEmail) {
    this.contactEmail = contactEmail;
}
public void setContactPhone(String contactPhone) {
    this.contactPhone = contactPhone;
}
// ===== HELPER METHODS =====
/**
 * Get formatted location string (City, Country)
 */
public String getLocation() {
    if (jobCity != null && !jobCity.isEmpty() &&
        jobCountry != null && !jobCountry.isEmpty()) {
        return jobCity + ", " + jobCountry;
    } else if (jobCity != null && !jobCity.isEmpty()) {
        return jobCity;
    } else if (jobCountry != null && !jobCountry.isEmpty()) {
        return jobCountry;
    }
    return "Location not specified";
}
/**
 * Get formatted salary range string
 */
public String getSalaryRange() {
    if (jobSalaryMin > 0 && jobSalaryMax > 0) {
        return formatCurrency(jobSalaryMin) + " - " + formatCurrency(jobSalaryMax);
    } else if (jobSalaryMin > 0) {
        return "From " + formatCurrency(jobSalaryMin);
    } else if (jobSalaryMax > 0) {
        return "Up to " + formatCurrency(jobSalaryMax);
    }
    return "Salary not specified";
}
/**
 * Format currency with symbol
 */
private String formatCurrency(double amount) {
    String symbol = getCurrencySymbol();
    // Format large numbers (e.g., 120000 -> 120K)
    if (amount >= 1000) {
        return symbol + String.format("%.0fK", amount / 1000);
    }
    return symbol + String.format("%.0f", amount);
}
/**
 * Get currency symbol
 */
private String getCurrencySymbol() {
    if (salaryCurrency == null) {
        return "$";
    }

```

```

    }
    switch (salaryCurrency) {
    case "USD":
    return "$";
    case "EUR":
    return "€";
    case "GBP":
    return "£";
    case "JPY":
    return "¥";
    case "INR":
    return "₹";
    case "AUD":
    return "A$";
    case "CAD":
    return "C$";
    case "CHF":
    return "CHF";
    case "CNY":
    return "¥";
    case "ILS":
    return "₪";
    default:
    return salaryCurrency + " ";
    }
    }
    /**
    * Check if application deadline has passed
    */
    public boolean isDeadlinePassed() {
    if (deadline <= 0) {
    return false;
    }
    return System.currentTimeMillis() > deadline;
    }
    /**
    * Check if user has applied
    */
    public boolean hasUserApplied(String userId) {
    return applicants != null && applicants.contains(userId);
    }
    /**
    * Get days remaining until deadline
    */
    public int getDaysUntilDeadline() {
    if (deadline <= 0) {
    return -1;
    }
    long diff = deadline - System.currentTimeMillis();
    return (int) (diff / (1000 * 60 * 60 * 24));
    }
    /**
    * Increment view count
    */
    public void incrementViewCount() {
    this.viewCount++;
    }
    /**
    * Increment applicant count
    */
    public void incrementApplicantCount() {
    this.applicantCount++;
    }
    // ===== VALIDATION METHODS =====

```

[illegible]

Application

Represents a Job Seeker's application to a specific job. Tracks status (Applied → Shortlisted → Accepted/Rejected). Stored in 'applications' collection.

Application.java (Java)

```
package com.example.joblinker.models;
public class Application {
    private String applicationId;
    private String jobId;
    private String jobSeekerUserId;
    private String employerUserId;
    private String status; // "Applied", "Shortlisted", "Rejected", "Accepted"
    private String coverLetter;
    private long appliedAt;
    private long updatedAt;
    private String jobTitle;
    private String jobSeekerName;
    // Empty constructor (required for Firebase)
    public Application() {}
    // Constructor with main fields
    public Application(String applicationId, String jobId, String jobSeekerUserId,
        String employerUserId, String coverLetter) {
        this.applicationId = applicationId;
        this.jobId = jobId;
        this.jobSeekerUserId = jobSeekerUserId;
        this.employerUserId = employerUserId;
        this.status = "Applied";
        this.coverLetter = coverLetter;
        this.appliedAt = System.currentTimeMillis();
        this.updatedAt = System.currentTimeMillis();
    }
    // Getters and Setters
    public String getApplicationId() { return applicationId; }
    public void setApplicationId(String applicationId) { this.applicationId = applicationId; [...]}
    public String getJobId() { return jobId; }
    public void setJobId(String jobId) { this.jobId = jobId; }
    public String getJobSeekerUserId() { return jobSeekerUserId; }
    public void setJobSeekerUserId(String jobSeekerUserId) { this.jobSeekerUserId = jobSeekerUser [...]}
    public String getEmployerUserId() { return employerUserId; }
    public void setEmployerUserId(String employerUserId) { this.employerUserId = employerUse [...]}
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public String getCoverLetter() { return coverLetter; }
    public void setCoverLetter(String coverLetter) { this.coverLetter = coverLetter; }
    public long getAppliedAt() { return appliedAt; }
    public void setAppliedAt(long appliedAt) { this.appliedAt = appliedAt; }
    public long getUpdatedAt() { return updatedAt; }
    public void setUpdatedAt(long updatedAt) { this.updatedAt = updatedAt; }
    public String getJobTitle() { return jobTitle; }
    public void setJobTitle(String jobTitle) { this.jobTitle = jobTitle; }
    public String getJobSeekerName() { return jobSeekerName; }
    public void setJobSeekerName(String jobSeekerName) { this.jobSeekerName = jobSeekerName; [...]}
}
```

Conversation

Represents a one-to-one chat thread between two users. Stores the last message for preview and unread count. Stored in 'conversations' collection.

Conversation.java (Java)

```
package com.example.joblinker.models;
import com.google.firebase.firestore.PropertyName;
import java.io.Serializable;
import java.util.List;
public class Conversation implements Serializable {
    private String conversationId;
    private List<String> participants;
    private String lastMessage;
    private long lastMessageTime;
    private String lastMessageSenderId;
    private int unreadCount;
    private String otherUserName;
    private String otherUserAvatarUrl;
    private boolean otherUserOnline;
    public Conversation() {
        this.lastMessageTime = System.currentTimeMillis();
        this.unreadCount = 0;
        this.otherUserOnline = false;
    }
    @PropertyName("conversationId")
    public String getConversationId() {
        return conversationId;
    }
    @PropertyName("conversationId")
    public void setConversationId(String conversationId) {
        this.conversationId = conversationId;
    }
    @PropertyName("participants")
    public List<String> getParticipants() {
        return participants;
    }
    @PropertyName("participants")
    public void setParticipants(List<String> participants) {
        this.participants = participants;
    }
    @PropertyName("lastMessage")
    public String getLastMessage() {
        return lastMessage;
    }
    @PropertyName("lastMessage")
    public void setLastMessage(String lastMessage) {
        this.lastMessage = lastMessage;
    }
    @PropertyName("lastMessageTime")
    public long getLastMessageTime() {
        return lastMessageTime;
    }
    @PropertyName("lastMessageTime")
    public void setLastMessageTime(long lastMessageTime) {
        this.lastMessageTime = lastMessageTime;
    }
    @PropertyName("lastMessageSenderId")
    public String getLastMessageSenderId() {
```

```
return lastMessageSenderId;
}
@PropertyName("lastMessageSenderId")
public void setLastMessageSenderId(String lastMessageSenderId) {
    this.lastMessageSenderId = lastMessageSenderId;
}
@PropertyName("unreadCount")
public int getUnreadCount() {
    return unreadCount;
}
@PropertyName("unreadCount")
public void setUnreadCount(int unreadCount) {
    this.unreadCount = unreadCount;
}
@PropertyName("otherUserName")
public String getOtherUserName() {
    return otherUserName;
}
@PropertyName("otherUserName")
public void setOtherUserName(String otherUserName) {
    this.otherUserName = otherUserName;
}
@PropertyName("otherUserAvatarUrl")
public String getOtherUserAvatarUrl() {
    return otherUserAvatarUrl;
}
@PropertyName("otherUserAvatarUrl")
public void setOtherUserAvatarUrl(String otherUserAvatarUrl) {
    this.otherUserAvatarUrl = otherUserAvatarUrl;
}
@PropertyName("otherUserOnline")
public boolean isOtherUserOnline() {
    return otherUserOnline;
}
@PropertyName("otherUserOnline")
public void setOtherUserOnline(boolean otherUserOnline) {
    this.otherUserOnline = otherUserOnline;
}
public boolean hasUnreadMessages() {
    return unreadCount > 0;
}
@Override
public String toString() {
    return "Conversation{" +
        "conversationId='" + conversationId + '\'' +
        ", lastMessage='" + lastMessage + '\'' +
        ", unreadCount=" + unreadCount +
        '}';
}
}
```

Message

Represents a single message within a conversation. Supports multiple message types: text, image, audio, document, location, contact. Stored in 'messages' collection.

Message.java (Java)

```
package com.example.joblinker.models;
import com.google.firebase.firestore.PropertyName;
import java.io.Serializable;
public class Message implements Serializable {
    private String messageId;
    private String conversationId;
    private String messageSenderId;
    private String messageReceiverId;
    private String messageText;
    private String messageType; // "text", "image", "voice"
    private long messageTimestamp;
    private boolean messageRead;
    private String senderName;
    private String senderAvatarUrl;
    private String imageUrl;
    private String voiceUrl;
    public Message() {
        this.messageTimestamp = System.currentTimeMillis();
        this.messageRead = false;
        this.messageType = "text";
    }
    public Message(String conversationId, String senderId, String receiverId, String text) {
        this();
        this.conversationId = conversationId;
        this.messageSenderId = senderId;
        this.messageReceiverId = receiverId;
        this.messageText = text;
    }
    // Getters and Setters with PropertyName annotations
    @PropertyName("messageId")
    public String getMessageId() {
        return messageId;
    }
    @PropertyName("messageId")
    public void setMessageId(String messageId) {
        this.messageId = messageId;
    }
    @PropertyName("conversationId")
    public String getConversationId() {
        return conversationId;
    }
    @PropertyName("conversationId")
    public void setConversationId(String conversationId) {
        this.conversationId = conversationId;
    }
    @PropertyName("messageSenderId")
    public String getMessageSenderId() {
        return messageSenderId;
    }
    @PropertyName("messageSenderId")
    public void setMessageSenderId(String messageSenderId) {
        this.messageSenderId = messageSenderId;
    }
}
```

```
@PropertyName("messageReceiverId")
public String getMessageReceiverId() {
    return messageReceiverId;
}
@PropertyName("messageReceiverId")
public void setMessageReceiverId(String messageReceiverId) {
    this.messageReceiverId = messageReceiverId;
}
@PropertyName("messageText")
public String getMessageText() {
    return messageText;
}
@PropertyName("messageText")
public void setMessageText(String messageText) {
    this.messageText = messageText;
}
@PropertyName("messageType")
public String getMessageType() {
    return messageType;
}
@PropertyName("messageType")
public void setMessageType(String messageType) {
    this.messageType = messageType;
}
@PropertyName("messageTimestamp")
public long getMessageTimestamp() {
    return messageTimestamp;
}
@PropertyName("messageTimestamp")
public void setMessageTimestamp(long messageTimestamp) {
    this.messageTimestamp = messageTimestamp;
}
@PropertyName("messageRead")
public boolean isMessageRead() {
    return messageRead;
}
@PropertyName("messageRead")
public void setMessageRead(boolean messageRead) {
    this.messageRead = messageRead;
}
@PropertyName("senderName")
public String getSenderName() {
    return senderName;
}
@PropertyName("senderName")
public void setSenderName(String senderName) {
    this.senderName = senderName;
}
@PropertyName("senderAvatarUrl")
public String getSenderAvatarUrl() {
    return senderAvatarUrl;
}
@PropertyName("senderAvatarUrl")
public void setSenderAvatarUrl(String senderAvatarUrl) {
    this.senderAvatarUrl = senderAvatarUrl;
}
@PropertyName("imageUrl")
public String getImageUrl() {
    return imageUrl;
}
@PropertyName("imageUrl")
public void setImageUrl(String imageUrl) {
    this.imageUrl = imageUrl;
}
}
```



```
@PropertyName("voiceUrl")
public String getVoiceUrl() {
    return voiceUrl;
}
@PropertyName("voiceUrl")
public void setVoiceUrl(String voiceUrl) {
    this.voiceUrl = voiceUrl;
}
// Helper methods
public boolean isTextMessage() {
    return "text".equals(messageType);
}
public boolean isImageMessage() {
    return "image".equals(messageType);
}
public boolean isVoiceMessage() {
    return "audio".equals(messageType) || "voice".equals(messageType);
}
@Override
public String toString() {
    return "Message{" +
        "messageId='" + messageId + '\'' +
        ", conversationId='" + conversationId + '\'' +
        ", messageType='" + messageType + '\'' +
        ", messageTimestamp='" + messageTimestamp +
        ", messageRead='" + messageRead +
        '\'';
}
}
```

Call

Represents a voice or video call record. Stores call type, status, and duration. Stored in 'calls' collection.

Call.java (Java)

```
package com.example.joblinker.models;
import com.google.firebase.firestore.PropertyName;
import java.io.Serializable;
public class Call implements Serializable {
    private String callId;
    private String callerId;
    private String receiverId;
    private String callType; // "voice" or "video"
    private String callStatus; // "initiated", "ringing", "connected", "ended", "rejected"
    private long startTime;
    private long endTime;
    private long duration; // in seconds
    private String callerName;
    private String callerAvatarUrl;
    private String receiverName;
    private String receiverAvatarUrl;
    private String agoraChannelName;
    private String agoraToken;
    public Call() {
        this.startTime = System.currentTimeMillis();
        this.callStatus = "initiated";
        this.duration = 0;
    }
    public Call(String callerId, String receiverId, String callType) {
        this();
        this.callerId = callerId;
        this.receiverId = receiverId;
        this.callType = callType;
    }
    // Getters and Setters with PropertyName annotations
    @PropertyName("callId")
    public String getCallId() {
        return callId;
    }
    @PropertyName("callId")
    public void setCallId(String callId) {
        this.callId = callId;
    }
    @PropertyName("callerId")
    public String getCallerId() {
        return callerId;
    }
    @PropertyName("callerId")
    public void setCallerId(String callerId) {
        this.callerId = callerId;
    }
    @PropertyName("receiverId")
    public String getReceiverId() {
        return receiverId;
    }
    @PropertyName("receiverId")
    public void setReceiverId(String receiverId) {
        this.receiverId = receiverId;
    }
}
```

```
@PropertyName("callType")
public String getCallType() {
    return callType;
}
@PropertyName("callType")
public void setCallType(String callType) {
    this.callType = callType;
}
@PropertyName("callStatus")
public String getCallStatus() {
    return callStatus;
}
@PropertyName("callStatus")
public void setCallStatus(String callStatus) {
    this.callStatus = callStatus;
}
@PropertyName("startTime")
public long getStartTime() {
    return startTime;
}
@PropertyName("startTime")
public void setStartTime(long startTime) {
    this.startTime = startTime;
}
@PropertyName("endTime")
public long getEndTime() {
    return endTime;
}
@PropertyName("endTime")
public void setEndTime(long endTime) {
    this.endTime = endTime;
}
@PropertyName("duration")
public long getDuration() {
    return duration;
}
@PropertyName("duration")
public void setDuration(long duration) {
    this.duration = duration;
}
@PropertyName("callerName")
public String getCallerName() {
    return callerName;
}
@PropertyName("callerName")
public void setCallerName(String callerName) {
    this.callerName = callerName;
}
@PropertyName("callerAvatarUrl")
public String getCallerAvatarUrl() {
    return callerAvatarUrl;
}
@PropertyName("callerAvatarUrl")
public void setCallerAvatarUrl(String callerAvatarUrl) {
    this.callerAvatarUrl = callerAvatarUrl;
}
@PropertyName("receiverName")
public String getReceiverName() {
    return receiverName;
}
@PropertyName("receiverName")
public void setReceiverName(String receiverName) {
    this.receiverName = receiverName;
}
}
```

```

@PropertyName("receiverAvatarUrl")
public String getReceiverAvatarUrl() {
    return receiverAvatarUrl;
}

@PropertyName("receiverAvatarUrl")
public void setReceiverAvatarUrl(String receiverAvatarUrl) {
    this.receiverAvatarUrl = receiverAvatarUrl;
}

@PropertyName("agoraChannelName")
public String getAgoraChannelName() {
    return agoraChannelName;
}

@PropertyName("agoraChannelName")
public void setAgoraChannelName(String agoraChannelName) {
    this.agoraChannelName = agoraChannelName;
}

@PropertyName("agoraToken")
public String getAgoraToken() {
    return agoraToken;
}

@PropertyName("agoraToken")
public void setAgoraToken(String agoraToken) {
    this.agoraToken = agoraToken;
}

// Helper methods
public boolean isVoiceCall() {
    return "voice".equals(callType);
}

public boolean isVideoCall() {
    return "video".equals(callType);
}

public boolean isConnected() {
    return "connected".equals(callStatus);
}

public boolean isEnded() {
    return "ended".equals(callStatus);
}

public boolean isRejected() {
    return "rejected".equals(callStatus);
}

public void calculateDuration() {
    if (endTime > 0 && startTime > 0) {
        this.duration = (endTime - startTime) / 1000;
    }
}

public String getFormattedDuration() {
    long hours = duration / 3600;
    long minutes = (duration % 3600) / 60;
    long seconds = duration % 60;
    if (hours > 0) {
        return String.format("%02d:%02d:%02d", hours, minutes, seconds);
    } else {
        return String.format("%02d:%02d", minutes, seconds);
    }
}

@Override
public String toString() {
    return "Call{" +
        "callId='" + callId + '\'' +
        ", callType='" + callType + '\'' +
        ", callStatus='" + callStatus + '\'' +
        ", duration=" + duration +
        '}';
}

```

```
}
```


Source Code

Firebase – Backend Integration

AuthManager

Thin wrapper around FirebaseAuth providing sign-in, registration, Google auth, phone OTP, and password reset methods.

AuthManager.java (Java)

```
package com.example.joblinker.firebase;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.AuthResult;
import com.google.android.gms.tasks.Task;
public class AuthManager {
    private FirebaseAuth auth;
    public AuthManager() {
        this.auth = FirebaseAuth.getInstance();
    }
    // Register new user
    public Task<AuthResult> registerUser(String email, String password) {
        return auth.createUserWithEmailAndPassword(email, password);
    }
    // Login user
    public Task<AuthResult> loginUser(String email, String password) {
        return auth.signInWithEmailAndPassword(email, password);
    }
    // Logout user
    public void logoutUser() {
        auth.signOut();
    }
    // Send password reset email
    public Task<Void> sendPasswordResetEmail(String email) {
        return auth.sendPasswordResetEmail(email);
    }
    // Get current user ID
    public String getCurrentUserId() {
        if (auth.getCurrentUser() != null) {
            return auth.getCurrentUser().getUid();
        }
        return null;
    }
    // Get current user email
    public String getCurrentUserEmail() {
        if (auth.getCurrentUser() != null) {
            return auth.getCurrentUser().getEmail();
        }
        return null;
    }
    // Check if user is logged in
    public boolean isUserLoggedIn() {
        return auth.getCurrentUser() != null;
    }
    // Delete current user account
    public Task<Void> deleteCurrentUser() {
        if (auth.getCurrentUser() != null) {
            return auth.getCurrentUser().delete();
        }
        return null;
    }
}
```


DatabaseManager

Provides helper methods for common Firestore read/write patterns used across the app.

DatabaseManager.java (Java)

```
package com.example.joblinker.firebaseio;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.Query;
import com.google.android.gms.tasks.Task;
import com.example.joblinker.models.User;
import com.example.joblinker.models.Job;
import com.example.joblinker.models.Application;
public class DatabaseManager {
    private FirebaseFirestore firestore;
    private static final String USERS_COLLECTION = "users";
    private static final String JOBS_COLLECTION = "jobs";
    private static final String APPLICATIONS_COLLECTION = "applications";
    public DatabaseManager() {
        this.firestore = FirebaseFirestore.getInstance();
    }
    // ===== USER OPERATIONS =====
    // Create/Update user
    public Task<Void> saveUser(User user) {
        return firestore.collection(USERS_COLLECTION)
            .document(user.getUserId())
            .set(user);
    }
    // Get user by ID
    public Task<com.google.firebase.firestore.DocumentSnapshot> getUser(String userId) {
        return firestore.collection(USERS_COLLECTION)
            .document(userId)
            .get();
    }
    // ===== JOB OPERATIONS =====
    // Create/Update job
    public Task<Void> saveJob(Job job) {
        return firestore.collection(JOBS_COLLECTION)
            .document(job.getJobId())
            .set(job);
    }
    // Get job by ID
    public Task<com.google.firebase.firestore.DocumentSnapshot> getJob(String jobId) {
        return firestore.collection(JOBS_COLLECTION)
            .document(jobId)
            .get();
    }
    // Get all jobs (with pagination)
    public Query getAllJobs() {
        return firestore.collection(JOBS_COLLECTION)
            .whereEqualTo("status", "Open")
            .orderBy("createdAt", Query.Direction.DESENDING);
    }
    // Get jobs by employer
    public Query getJobsByEmployer(String employerId) {
        return firestore.collection(JOBS_COLLECTION)
            .whereEqualTo("employerId", employerId)
            .orderBy("createdAt", Query.Direction.DESENDING);
    }
    // Delete job
```

```
public Task<Void> deleteJob(String jobId) {
    return firestore.collection(JOBS_COLLECTION)
        .document(jobId)
        .delete();
}

// ===== APPLICATION OPERATIONS =====
// Create/Update application
public Task<Void> saveApplication(Application application) {
    return firestore.collection(APPLICATIONS_COLLECTION)
        .document(application.getId())
        .set(application);
}

// Get application by ID
public Task<com.google.firebase.firestore.DocumentSnapshot> getApplication(String applic [...])
    return firestore.collection(APPLICATIONS_COLLECTION)
        .document(applicationId)
        .get();
}

// Get applications for a job seeker
public Query getApplicationsByJobSeeker(String jobSeekerUserId) {
    return firestore.collection(APPLICATIONS_COLLECTION)
        .whereEqualTo("jobSeekerUserId", jobSeekerUserId)
        .orderBy("appliedAt", Query.Direction.DESENDING);
}

// Get applications for a job
public Query getApplicationsByJob(String jobId) {
    return firestore.collection(APPLICATIONS_COLLECTION)
        .whereEqualTo("jobId", jobId);
}

// Update application status
public Task<Void> updateApplicationStatus(String applicationId, String status) {
    return firestore.collection(APPLICATIONS_COLLECTION)
        .document(applicationId)
        .update("status", status);
}

// Delete application
public Task<Void> deleteApplication(String applicationId) {
    return firestore.collection(APPLICATIONS_COLLECTION)
        .document(applicationId)
        .delete();
}
}
```

JobLinkerFirebaseManager

Main singleton Firebase manager. Contains all Firestore operations: CRUD for users, jobs, messages, conversations, applications, and real-time listeners.

JobLinkerFirebaseManager.java (Java)

```
package com.example.joblinker.firebase;
import android.net.Uri;
import android.util.Log;
import androidx.annotation.NonNull;
import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.OnFailureListener;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.auth.AuthResult;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.auth.PhoneAuthCredential;
import com.google.firebase.firestore.DocumentReference;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FieldValue;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.ListenerRegistration;
import com.google.firebase.firestore.Query;
import com.google.firebase.firestore.QueryDocumentSnapshot;
import com.google.firebase.firestore.QuerySnapshot;
import com.google.firebase.storage.FirebaseStorage;
import com.google.firebase.storage.StorageReference;
import com.google.firebase.storage.UploadTask;
import com.example.joblinker.models.Call;
import com.example.joblinker.models.Conversation;
import com.example.joblinker.models.Job;
import com.example.joblinker.models.Message;
import com.example.joblinker.models.User;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID;

public class JobLinkerFirebaseManager {
    private static final String TAG = "FirebaseManager";
    // Firebase instances
    private final FirebaseAuth mAuth;
    private final FirebaseFirestore db;
    private final FirebaseStorage storage;
    // Collection names
    private static final String USERS_COLLECTION = "users";
    private static final String JOBS_COLLECTION = "jobs";
    private static final String MESSAGES_COLLECTION = "messages";
    private static final String CALLS_COLLECTION = "calls";
    private static final String CONVERSATIONS_COLLECTION = "conversations";
    private static final String APPLICATIONS_COLLECTION = "applications";
    private static final String SAVED_JOBS_COLLECTION = "savedJobs";
    // Storage paths
    private static final String AVATARS_PATH = "avatars";
    private static final String DOCUMENTS_PATH = "documents";
    private static final String CHAT_IMAGES_PATH = "chat_images";
    // Singleton instance
```

```

private static JobLinkerFirebaseManager instance;
// Listener registrations for cleanup
private final List<ListenerRegistration> listenerRegistrations;
// ===== CALLBACK INTERFACES =====
/**
 * Authentication callback
 */
public interface AuthCallback {
    void onSuccess(FirebaseUser user);
    void onFailure(String error);
}
/**
 * Data callback for single object
 */
public interface DataCallback<T> {
    void onSuccess(T data);
    void onFailure(String error);
}
/**
 * List callback for multiple objects
 */
public interface ListCallback<T> {
    void onSuccess(List<T> dataList);
    void onFailure(String error);
}
/**
 * Upload callback with progress
 */
public interface UploadCallback {
    void onSuccess(String downloadUrl);
    void onProgress(int progress);
    void onFailure(String error);
}
/**
 * Void callback for operations with no return value
 */
public interface VoidCallback {
    void onSuccess();
    void onFailure(String error);
}
// ===== CONSTRUCTOR & INITIALIZATION =====
/**
 * Private constructor for singleton pattern
 */
private JobLinkerFirebaseManager() {
    mAuth = FirebaseAuth.getInstance();
    db = FirebaseFirestore.getInstance();
    storage = FirebaseStorage.getInstance();
    listenerRegistrations = new ArrayList<>();
}
/**
 * Get singleton instance
 */
public static synchronized JobLinkerFirebaseManager getInstance() {
    if (instance == null) {
        instance = new JobLinkerFirebaseManager();
    }
    return instance;
}
// ===== AUTHENTICATION METHODS =====
/**
 * Register user with email and password
 */
public void registerWithEmail(String email, String password, final AuthCallback callback [...]
```

```

mAuth.createUserWithEmailAndPassword(email, password)
    .addOnCompleteListener(task -> {
        if (task.isSuccessful()) {
            FirebaseUser user = mAuth.getCurrentUser();
            if (user != null) {
                // Send email verification
                sendEmailVerification(new VoidCallback() {
                    @Override
                    public void onSuccess() {
                        Log.d(TAG, "Email verification sent");
                        callback.onSuccess(user);
                    }
                    @Override
                    public void onFailure(String error) {
                        // Still proceed even if verification email fails
                        Log.e(TAG, "Failed to send verification email: " + error [...]);
                        callback.onSuccess(user);
                    }
                });
            }
        } else {
            String error = task.getException() != null ?
            task.getException().getMessage() : "Registration failed";
            callback.onFailure(error);
            Log.e(TAG, "registerWithEmail:failure", task.getException());
        }
    });
}
/**
 * Login with email and password
 */
public void loginWithEmail(String email, String password, final AuthCallback callback) {
    mAuth.signInWithEmailAndPassword(email, password)
        .addOnCompleteListener(task -> {
            if (task.isSuccessful()) {
                FirebaseUser user = mAuth.getCurrentUser();
                if (user != null) {
                    // Update online status
                    updateUserOnlineStatus(user.getUid(), true);
                }
                callback.onSuccess(user);
                Log.d(TAG, "loginWithEmail:success");
            } else {
                String error = task.getException() != null ?
                task.getException().getMessage() : "Login failed";
                callback.onFailure(error);
                Log.e(TAG, "loginWithEmail:failure", task.getException());
            }
        });
}
/**
 * Sign in with phone credential
 */
public void signInWithPhoneCredential(PhoneAuthCredential credential, final AuthCallback [...]) {
    mAuth.signInWithCredential(credential)
        .addOnCompleteListener(task -> {
            if (task.isSuccessful()) {
                FirebaseUser user = mAuth.getCurrentUser();
                if (user != null) {
                    updateUserOnlineStatus(user.getUid(), true);
                }
                callback.onSuccess(user);
                Log.d(TAG, "signInWithPhoneCredential:success");
            } else {

```

```

String error = task.getException() != null ?
task.getException().getMessage() : "Phone verification faile [...]
callback.onFailure(error);
Log.e(TAG, "signInWithPhoneCredential:failure", task.getException()) [...]
}
});
}
/**
 * Send email verification
 */
public void sendEmailVerification(final VoidCallback callback) {
    FirebaseAuth user = mAuth.getCurrentUser();
    if (user != null) {
        user.sendEmailVerification()
            .addOnCompleteListener(task -> {
                if (task.isSuccessful()) {
                    callback.onSuccess();
                    Log.d(TAG, "Email verification sent");
                } else {
                    String error = task.getException() != null ?
                    task.getException().getMessage() : "Failed to send verif [...]
                    callback.onFailure(error);
                    Log.e(TAG, "sendEmailVerification:failure", task.getException()) [...]
                }
            });
    } else {
        callback.onFailure("No user logged in");
    }
}
/**
 * Send password reset email
 */
public void sendPasswordResetEmail(String email, final VoidCallback callback) {
    mAuth.sendPasswordResetEmail(email)
        .addOnCompleteListener(task -> {
            if (task.isSuccessful()) {
                callback.onSuccess();
                Log.d(TAG, "Password reset email sent");
            } else {
                String error = task.getException() != null ?
                task.getException().getMessage() : "Failed to send reset ema [...]
                callback.onFailure(error);
                Log.e(TAG, "sendPasswordResetEmail:failure", task.getException());
            }
        });
}
/**
 * Change password
 */
public void changePassword(String newPassword, final VoidCallback callback) {
    FirebaseAuth user = mAuth.getCurrentUser();
    if (user != null) {
        user.updatePassword(newPassword)
            .addOnCompleteListener(task -> {
                if (task.isSuccessful()) {
                    callback.onSuccess();
                    Log.d(TAG, "Password changed successfully");
                } else {
                    String error = task.getException() != null ?
                    task.getException().getMessage() : "Failed to change pas [...]
                    callback.onFailure(error);
                    Log.e(TAG, "changePassword:failure", task.getException());
                }
            });
    }
}

```

```

    } else {
        callback.onFailure("No user logged in");
    }
}
/**
 * Logout current user
 */
public void logout() {
    String userId = getCurrentUserId();
    if (userId != null) {
        updateUserOnlineStatus(userId, false);
    }
    // Clean up listeners
    for (ListenerRegistration registration : listenerRegistrations) {
        registration.remove();
    }
    listenerRegistrations.clear();
    // Sign out
    mAuth.signOut();
    Log.d(TAG, "User logged out");
}
/**
 * Delete user account
 */
public void deleteAccount(final VoidCallback callback) {
    FirebaseUser user = mAuth.getCurrentUser();
    if (user != null) {
        String userId = user.getUid();
        // First delete user data from Firestore
        deleteUser(userId, new VoidCallback() {
            @Override
            public void onSuccess() {
                // Then delete auth account
                user.delete().addOnCompleteListener(task -> {
                    if (task.isSuccessful()) {
                        callback.onSuccess();
                        Log.d(TAG, "Account deleted successfully");
                    } else {
                        String error = task.getException() != null ?
                            task.getException().getMessage() : "Failed to delete acc [...]";
                        callback.onFailure(error);
                        Log.e(TAG, "deleteAccount:failure", task.getException());
                    }
                });
            }
        });
        @Override
        public void onFailure(String error) {
            callback.onFailure(error);
        }
    }
} else {
    callback.onFailure("No user logged in");
}
}
/**
 * Get current user ID
 */
public String getCurrentUserId() {
    FirebaseUser user = mAuth.getCurrentUser();
    return user != null ? user.getUid() : null;
}
/**
 * Get current Firebase user
 */

```

```

public FirebaseUser getCurrentUser() {
    return mAuth.getCurrentUser();
}
/**
 * Check if email is verified
 */
public boolean isEmailVerified() {
    FirebaseUser user = mAuth.getCurrentUser();
    return user != null && user.isEmailVerified();
}
/**
 * Reload current user to get updated verification status
 */
public void reloadUser(final VoidCallback callback) {
    FirebaseUser user = mAuth.getCurrentUser();
    if (user != null) {
        user.reload().addOnCompleteListener(task -> {
            if (task.isSuccessful()) {
                callback.onSuccess();
            } else {
                String error = task.getException() != null ?
                task.getException().getMessage() : "Failed to reload user";
                callback.onFailure(error);
            }
        });
    } else {
        callback.onFailure("No user logged in");
    }
}
// ===== USER CRUD OPERATIONS =====
/**
 * Create user in Firestore
 */
public void createUser(User user, final VoidCallback callback) {
    if (user.getUserId() == null || user.getUserId().isEmpty()) {
        callback.onFailure("User ID is required");
        return;
    }
    db.collection(USERS_COLLECTION)
        .document(user.getUserId())
        .set(user)
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "User created successfully: " + user.getUserId());
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error creating user", e);
        });
}
/**
 * Get user by ID
 */
public void getUser(String userId, final DataCallback<User> callback) {
    db.collection(USERS_COLLECTION)
        .document(userId)
        .get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                User user = documentSnapshot.toObject(User.class);
                callback.onSuccess(user);
                Log.d(TAG, "User retrieved: " + userId);
            } else {
                callback.onFailure("User not found");
            }
        });
}

```



```

Log.w(TAG, "User not found: " + userId);
}
})
.addOnFailureListener(e -> {
callback.onFailure(e.getMessage());
Log.e(TAG, "Error getting user", e);
});
}
/**
 * Update user
 */
public void updateUser(String userId, Map<String, Object> updates, final VoidCallback ca [...])
db.collection(USERS_COLLECTION)
.document(userId)
.update(updates)
.addOnSuccessListener(aVoid -> {
callback.onSuccess();
Log.d(TAG, "User updated successfully: " + userId);
})
.addOnFailureListener(e -> {
callback.onFailure(e.getMessage());
Log.e(TAG, "Error updating user", e);
});
}
/**
 * Update user online status
 */
public void updateUserOnlineStatus(String userId, boolean isOnline) {
Map<String, Object> updates = new HashMap<>();
updates.put("isOnline", isOnline);
updates.put("lastSeen", System.currentTimeMillis());
db.collection(USERS_COLLECTION)
.document(userId)
.update(updates)
.addOnSuccessListener(aVoid ->
Log.d(TAG, "User online status updated: " + userId + " - " + isOnlin [...])
.addOnFailureListener(e ->
Log.e(TAG, "Error updating online status", e));
}
}
/**
 * Delete user
 */
public void deleteUser(String userId, final VoidCallback callback) {
db.collection(USERS_COLLECTION)
.document(userId)
.delete()
.addOnSuccessListener(aVoid -> {
callback.onSuccess();
Log.d(TAG, "User deleted successfully: " + userId);
})
.addOnFailureListener(e -> {
callback.onFailure(e.getMessage());
Log.e(TAG, "Error deleting user", e);
});
}
}
/**
 * Search users by name or email
 */
public void searchUsers(String query, final ListCallback<User> callback) {
// Get all users and filter in memory – avoids needing index on userName
String lowerQuery = query.toLowerCase();
db.collection(USERS_COLLECTION)
.get()
.addOnSuccessListener(queryDocumentSnapshots -> {

```

```

List<User> users = new ArrayList<>();
for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
    User user = document.toObject(User.class);
    if (user.getUserName() != null &&
        user.getUserName().toLowerCase().contains(lowerQuery)) {
        users.add(user);
    }
}
callback.onSuccess(users);
})
.addOnFailureListener(e -> {
    callback.onFailure(e.getMessage());
    Log.e(TAG, "Error searching users", e);
});
}

// ===== JOB CRUD OPERATIONS =====
/**
 * Create job
 */
public void createJob(Job job, final DataCallback<String> callback) {
    DocumentReference docRef = db.collection(JOBS_COLLECTION).document();
    job.setJobId(docRef.getId());
    job.setCreatedAt(System.currentTimeMillis());
    // Convert to map so we can save BOTH employerId fields for compatibility
    // (old code queried "employerId", new code queries "jobEmployerId")
    java.util.Map<String, Object> jobMap = new java.util.HashMap<>();
    jobMap.put("jobId", job.getJobId());
    jobMap.put("jobTitle", job.getJobTitle());
    jobMap.put("jobCompany", job.getJobCompany());
    jobMap.put("jobEmployerId", job.getJobEmployerId());
    jobMap.put("employerId", job.getJobEmployerId()); // backward compat
    jobMap.put("jobCategory", job.getJobCategory());
    jobMap.put("jobType", job.getJobType());
    jobMap.put("employmentType", job.getEmploymentType());
    jobMap.put("jobCountry", job.getJobCountry());
    jobMap.put("jobCity", job.getJobCity());
    jobMap.put("jobDescription", job.getJobDescription());
    jobMap.put("jobSkills", job.getJobSkills());
    jobMap.put("jobSalaryMin", job.getJobSalaryMin());
    jobMap.put("jobSalaryMax", job.getJobSalaryMax());
    jobMap.put("salaryCurrency", job.getSalaryCurrency());
    jobMap.put("deadline", job.getDeadline());
    jobMap.put("createdAt", job.getCreatedAt());
    jobMap.put("updatedAt", job.getUpdatedAt());
    jobMap.put("jobActive", true);
    jobMap.put("viewCount", job.getViewCount());
    jobMap.put("applicantCount", job.getApplicantCount());
    jobMap.put("urgent", job.isUrgent());
    if (job.getCompanyLogoUrl() != null) jobMap.put("companyLogoUrl", job.getCompanyLogo [...])
    docRef.set(jobMap)
    .addOnSuccessListener(aVoid -> {
        callback.onSuccess(job.getJobId());
        Log.d(TAG, "Job created successfully: " + job.getJobId());
    })
    .addOnFailureListener(e -> {
        callback.onFailure(e.getMessage());
        Log.e(TAG, "Error creating job", e);
    });
}

/**
 * Get job by ID
 */
public void getJob(String jobId, final DataCallback<Job> callback) {
    db.collection(JOBS_COLLECTION)

```

```

        .document(jobId)
        .get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                Job job = documentSnapshot.toObject(Job.class);
                if (job != null) {
                    // Try every possible field name for employer ID
                    // (different versions of the app saved it differently)
                    if (job.getJobEmployerId() == null || job.getJobEmployerId().isEmpty()) {
                        String[] possibleFields = {"employerId", "jobEmployerId", "e [...]
                    for (String field : possibleFields) {
                        String val = documentSnapshot.getString(field);
                        if (val != null && !val.isEmpty()) {
                            job.setJobEmployerId(val);
                            Log.d(TAG, "Found employer ID in field '" + field + [...]
                            break;
                        }
                    }
                }
                // Also ensure jobId is set
                if (job.getJobId() == null || job.getJobId().isEmpty()) {
                    job.setJobId(documentSnapshot.getId());
                }
                callback.onSuccess(job);
                Log.d(TAG, "Job retrieved: " + jobId + " employerId=" + (job != null [...]
            } else {
                callback.onFailure("Job not found");
                Log.w(TAG, "Job not found: " + jobId);
            }
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error getting job", e);
        });
    }
    /**
     * Get all active jobs
     */
    public void getActiveJobs(final ListCallback<Job> callback) {
        db.collection(JOBS_COLLECTION)
            .whereEqualTo("jobActive", true)
            .get()
            .addOnSuccessListener(queryDocumentSnapshots -> {
                List<Job> jobs = new ArrayList<>();
                for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                    Job job = document.toObject(Job.class);
                    if (job != null) {
                        // Try all possible employer ID field names
                        if (job.getJobEmployerId() == null || job.getJobEmployerId().isEmpty()) {
                            for (String f2 : new String[]{"employerId", "jobEmployerId", " [...]
                                String v = document.getString(f2);
                                if (v != null && !v.isEmpty()) { job.setJobEmployerId(v) [...]
                            }
                        }
                        // Ensure jobId is set from document ID
                        if (job.getJobId() == null || job.getJobId().isEmpty()) {
                            job.setJobId(document.getId());
                        }
                        jobs.add(job);
                    }
                }
                jobs.sort((a, b) -> Long.compare(b.getCreatedAt(), a.getCreatedAt()));
                callback.onSuccess(jobs);
            }
        }
    }

```

```

Log.d(TAG, "Retrieved " + jobs.size() + " active jobs");
})
.addOnFailureListener(e -> {
    callback.onFailure(e.getMessage());
    Log.e(TAG, "Error getting active jobs", e);
});
}
/**
 * Get jobs by type
 */
public void getJobsByType(String jobType, final ListCallback<Job> callback) {
    db.collection(JOBS_COLLECTION)
        .whereEqualTo("jobActive", true)
        .whereEqualTo("jobType", jobType)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            List<Job> jobs = new ArrayList<>();
            for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                Job job = document.toObject(Job.class);
                jobs.add(job);
            }
            jobs.sort((a, b) -> Long.compare(b.getCreatedAt(), a.getCreatedAt()));
            callback.onSuccess(jobs);
            Log.d(TAG, "Retrieved " + jobs.size() + " jobs of type: " + jobType);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error getting jobs by type", e);
        });
}
/**
 * Get jobs by employer
 */
public void getJobsByEmployer(String employerId, final ListCallback<Job> callback) {
    db.collection(JOBS_COLLECTION)
        // Field name matches Job model: private String jobEmployerId
        .whereEqualTo("jobEmployerId", employerId)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            List<Job> jobs = new ArrayList<>();
            for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                Job job = document.toObject(Job.class);
                jobs.add(job);
            }
            // Sort by createdAt descending in memory – no composite index needed
            jobs.sort((a, b) -> Long.compare(b.getCreatedAt(), a.getCreatedAt()));
            callback.onSuccess(jobs);
            Log.d(TAG, "Retrieved " + jobs.size() + " jobs for employer: " + employerId);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error getting jobs by employer", e);
        });
}
/**
 * Search jobs by title, company, or description
 */
public void searchJobs(String query, final ListCallback<Job> callback) {
    // Note: Firestore doesn't support full-text search natively
    // This is a basic implementation - consider using Algolia or Elasticsearch for prod [...]
    db.collection(JOBS_COLLECTION)
        .whereEqualTo("jobActive", true)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {

```

```

List<Job> jobs = new ArrayList<>();
String lowerQuery = query.toLowerCase();
for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
    Job job = document.toObject(Job.class);
    String title = job.getJobTitle() != null ? job.getJobTitle() [...]
    String company = job.getJobCompany() != null ? job.getJobCompany [...]
    String desc = job.getJobDescription() != null ? job.getJobDescrip [...]
    if (title.contains(lowerQuery) || company.contains(lowerQuery) || de [...]
    jobs.add(job);
}
}
callback.onSuccess(jobs);
Log.d(TAG, "Search returned " + jobs.size() + " jobs for query: " + quer [...])
})
.addOnFailureListener(e -> {
    callback.onFailure(e.getMessage());
    Log.e(TAG, "Error searching jobs", e);
});
}
/**
 * Update job
 */
public void updateJob(String jobId, Map<String, Object> updates, final VoidCallback call [...])
updates.put("updatedAt", System.currentTimeMillis());
db.collection(JOBS_COLLECTION)
    .document(jobId)
    .update(updates)
    .addOnSuccessListener(aVoid -> {
        callback.onSuccess();
        Log.d(TAG, "Job updated successfully: " + jobId);
    })
    .addOnFailureListener(e -> {
        callback.onFailure(e.getMessage());
        Log.e(TAG, "Error updating job", e);
    });
}
/**
 * Delete job
 */
public void deleteJob(String jobId, final VoidCallback callback) {
    db.collection(JOBS_COLLECTION)
        .document(jobId)
        .delete()
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "Job deleted successfully: " + jobId);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error deleting job", e);
        });
}
/**
 * Listen to active jobs in real-time
 */
public ListenerRegistration listenToActiveJobs(final ListCallback<Job> callback) {
    ListenerRegistration registration = db.collection(JOBS_COLLECTION)
        .whereEqualTo("jobActive", true)
        // No orderBy – sort in memory to avoid composite index
        .addSnapshotListener((queryDocumentSnapshots, e) -> {
            if (e != null) {
                callback.onFailure(e.getMessage());
                Log.e(TAG, "Listen to jobs failed", e);
            }
            return;
        });
}

```

```

    }
    if (queryDocumentSnapshots != null) {
        List<Job> jobs = new ArrayList<>();
        for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
            Job job = document.toObject(Job.class);
            if (job != null) {
                // Try all possible employer ID field names
                if (job.getJobEmployerId() == null || job.getJobEmployerId() [...]
                for (String f2 : new String[]{"employerId", "jobEmployerI [...]
                String v = document.getString(f2);
                if (v != null && !v.isEmpty()) { job.setJobEmployerI [...]
            }
        }
        if (job.getJobId() == null || job.getJobId().isEmpty()) {
            job.setJobId(document.getId());
        }
        jobs.add(job);
    }
    jobs.sort((a, b) -> Long.compare(b.getCreatedAt(), a.getCreatedAt()) [...]
    callback.onSuccess(jobs);
    Log.d(TAG, "Jobs updated via listener: " + jobs.size());
}
});
listenerRegistrations.add(registration);
return registration;
}
/**
 * Increment job view count
 */
public void incrementJobViewCount(String jobId) {
    db.collection(JOBS_COLLECTION)
        .document(jobId)
        .update("viewCount", FieldValue.increment(1))
        .addOnSuccessListener(aVoid ->
            Log.d(TAG, "View count incremented for job: " + jobId))
        .addOnFailureListener(e ->
            Log.e(TAG, "Error incrementing view count", e));
}
/**
 * Add applicant to job
 */
public void addJobApplicant(String jobId, String userId, final VoidCallback callback) {
    db.collection(JOBS_COLLECTION)
        .document(jobId)
        .update("applicants", FieldValue.arrayUnion(userId))
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "Applicant added to job: " + jobId);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error adding applicant", e);
        });
}
// ===== MESSAGE CRUD OPERATIONS =====
/**
 * Send message
 */
public void sendMessage(Message message, final DataCallback<String> callback) {
    DocumentReference docRef = db.collection(MESSAGES_COLLECTION).document();
    message.setMessageId(docRef.getId());
    message.setTimestamp(System.currentTimeMillis());
    docRef.set(message)

```

```

        .addOnSuccessListener(aVoid -> {
            // Update conversation's last message
            updateConversationLastMessage(message);
            callback.onSuccess(message.getMessageId());
            Log.d(TAG, "Message sent successfully: " + message.getMessageId());
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error sending message", e);
        });
    }
    /**
     * Get messages for a conversation
     */
    public void getMessages(String conversationId, final ListCallback<Message> callback) {
        db.collection(MESSAGES_COLLECTION)
            .whereEqualTo("conversationId", conversationId)
            // No orderBy – sort in memory to avoid composite index requirement
            .get()
            .addOnSuccessListener(queryDocumentSnapshots -> {
                List<Message> messages = new ArrayList<>();
                for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                    messages.add(document.toObject(Message.class));
                }
                messages.sort((a, b) ->
                    Long.compare(a.getMessageTimestamp(), b.getMessageTimestamp()));
                callback.onSuccess(messages);
                Log.d(TAG, "Retrieved " + messages.size() + " messages");
            })
            .addOnFailureListener(e -> {
                callback.onFailure(e.getMessage());
                Log.e(TAG, "Error getting messages", e);
            });
    }
    /**
     * Listen to messages in real-time
     */
    public ListenerRegistration listenToMessages(String conversationId, final ListCallback<M [...]
        ListenerRegistration registration = db.collection(MESSAGES_COLLECTION)
            .whereEqualTo("conversationId", conversationId)
            // No orderBy – sort in memory to avoid composite index requirement
            .addSnapshotListener((queryDocumentSnapshots, e) -> {
                if (e != null) {
                    callback.onFailure(e.getMessage());
                    Log.e(TAG, "Listen to messages failed", e);
                    return;
                }
                if (queryDocumentSnapshots != null) {
                    List<Message> messages = new ArrayList<>();
                    for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                        messages.add(document.toObject(Message.class));
                    }
                    messages.sort((a, b) ->
                        Long.compare(a.getMessageTimestamp(), b.getMessageTimestamp()));
                    callback.onSuccess(messages);
                    Log.d(TAG, "Messages updated: " + messages.size());
                }
            });
        listenerRegistrations.add(registration);
        return registration;
    }
    /**
     * Mark message as read
     */

```

```

public void markMessageAsRead(String messageId, final VoidCallback callback) {
    Map<String, Object> updates = new HashMap<>();
    updates.put("messageRead", true);
    db.collection(MESSAGES_COLLECTION)
        .document(messageId)
        .update(updates)
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "Message marked as read: " + messageId);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error marking message as read", e);
        });
}
/**
 * Mark all messages in conversation as read
 */
public void markAllMessagesAsRead(String conversationId, String userId, final VoidCallba [...]
    db.collection(MESSAGES_COLLECTION)
        .whereEqualTo("conversationId", conversationId)
        .whereEqualTo("messageReceiverId", userId)
        .whereEqualTo("messageRead", false)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                document.getReference().update("messageRead", true);
            }
            callback.onSuccess();
            Log.d(TAG, "All messages marked as read for conversation: " + conversati [...])
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error marking all messages as read", e);
        });
}
/**
 * Delete message
 */
public void deleteMessage(String messageId, final VoidCallback callback) {
    db.collection(MESSAGES_COLLECTION)
        .document(messageId)
        .delete()
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "Message deleted: " + messageId);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error deleting message", e);
        });
}
/**
 * Update conversation's last message
 */
private void updateConversationLastMessage(Message message) {
    Map<String, Object> conversationData = new HashMap<>();
    conversationData.put("lastMessage", message.getMessageText());
    conversationData.put("lastMessageTime", message.getMessageTimestamp());
    conversationData.put("lastMessageSenderId", message.getMessageSenderId());
    conversationData.put("participants", Arrays.asList(
        message.getMessageSenderId(),
        message.getMessageReceiverId()
    ));
}

```



```

db.collection(CONVERSATIONS_COLLECTION)
  .document(message.getConversationId())
  .set(conversationData)
  .addOnSuccessListener(aVoid ->
    Log.d(TAG, "Conversation updated: " + message.getConversationId()))
  .addOnFailureListener(e ->
    Log.e(TAG, "Error updating conversation", e));
}
/**
 * Get user conversations
 */
public void getUserConversations(String userId, final ListCallback<Conversation> callbac [...])
db.collection(CONVERSATIONS_COLLECTION)
  .whereArrayContains("participants", userId)
  // No orderBy here – Firestore requires a composite index for
  // whereArrayContains + orderBy. We sort in memory instead.
  .get()
  .addOnSuccessListener(queryDocumentSnapshots -> {
    List<Conversation> conversations = new ArrayList<>();
    for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
      Conversation conversation = document.toObject(Conversation.class);
      conversation.setConversationId(document.getId());
      conversations.add(conversation);
    }
    // Sort by lastMessageTime descending (newest first)
    conversations.sort((a, b) ->
      Long.compare(b.getLastMessageTime(), a.getLastMessageTime()));
    callback.onSuccess(conversations);
    Log.d(TAG, "Retrieved " + conversations.size() + " conversations for use [...]");
  })
  .addOnFailureListener(e -> {
    callback.onFailure(e.getMessage());
    Log.e(TAG, "Error getting conversations", e);
  });
}
/**
 * Listen to conversations in real-time
 */
public ListenerRegistration listenToConversations(String userId, final ListCallback<Conv [...])
ListenerRegistration registration = db.collection(CONVERSATIONS_COLLECTION)
  .whereArrayContains("participants", userId)
  // No orderBy – avoids composite index requirement, sort in memory
  .addSnapshotListener((queryDocumentSnapshots, e) -> {
    if (e != null) {
      callback.onFailure(e.getMessage());
      Log.e(TAG, "Listen to conversations failed", e);
      return;
    }
    if (queryDocumentSnapshots != null) {
      List<Conversation> conversations = new ArrayList<>();
      for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
        Conversation conversation = document.toObject(Conversation.class [...])
        conversation.setConversationId(document.getId());
        conversations.add(conversation);
      }
      // Sort newest first in memory
      conversations.sort((a, b) ->
        Long.compare(b.getLastMessageTime(), a.getLastMessageTime()));
      callback.onSuccess(conversations);
      Log.d(TAG, "Conversations updated via listener: " + conversations.si [...])
    }
  });
listenerRegistrations.add(registration);
return registration;

```

```

}
// ===== CALL OPERATIONS =====
/**
 * Create call record
 */
public void createCall(Call call, final DataCallback<String> callback) {
    DocumentReference docRef = db.collection(CALLS_COLLECTION).document();
    call.setCallId(docRef.getId());
    call.setStartTime(System.currentTimeMillis());
    docRef.set(call)
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess(call.getCallId());
            Log.d(TAG, "Call created successfully: " + call.getCallId());
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error creating call", e);
        });
}
/**
 * Update call status
 */
public void updateCallStatus(String callId, String status, final VoidCallback callback) [...]
    Map<String, Object> updates = new HashMap<>();
    updates.put("callStatus", status);
    if ("ended".equals(status)) {
        updates.put("endTime", System.currentTimeMillis());
    } else if ("connected".equals(status)) {
        updates.put("startTime", System.currentTimeMillis());
    }
    db.collection(CALLS_COLLECTION)
        .document(callId)
        .update(updates)
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "Call status updated: " + callId + " - " + status);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error updating call status", e);
        });
}
/**
 * Get call by ID
 */
public void getCall(String callId, final DataCallback<Call> callback) {
    db.collection(CALLS_COLLECTION)
        .document(callId)
        .get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                Call call = documentSnapshot.toObject(Call.class);
                callback.onSuccess(call);
                Log.d(TAG, "Call retrieved: " + callId);
            } else {
                callback.onFailure("Call not found");
                Log.w(TAG, "Call not found: " + callId);
            }
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error getting call", e);
        });
}
}

```

```

/**
 * Get call history for user
 */
public void getCallHistory(String userId, final ListCallback<Call> callback) {
    List<Call> allCalls = new ArrayList<>();
    // Get calls where user is caller
    db.collection(CALLS_COLLECTION)
        .whereEqualTo("callerId", userId)
    // No orderBy – sort in memory
    .limit(50)
    .get()
    .addOnSuccessListener(queryDocumentSnapshots -> {
        for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
            Call call = document.toObject(Call.class);
            allCalls.add(call);
        }
    // Get calls where user is receiver
    db.collection(CALLS_COLLECTION)
        .whereEqualTo("receiverId", userId)
    // No orderBy – sort in memory
    .limit(50)
    .get()
    .addOnSuccessListener(receivedCallsSnapshot -> {
        for (QueryDocumentSnapshot document : receivedCallsSnapshot) [...]
            Call call = document.toObject(Call.class);
            allCalls.add(call);
        }
    callback.onSuccess(allCalls);
    Log.d(TAG, "Retrieved " + allCalls.size() + " calls for user [...]");
    })
    .addOnFailureListener(e -> {
        callback.onFailure(e.getMessage());
        Log.e(TAG, "Error getting received calls", e);
    });
    })
    .addOnFailureListener(e -> {
        callback.onFailure(e.getMessage());
        Log.e(TAG, "Error getting call history", e);
    });
    })
    // ===== STORAGE OPERATIONS =====
    /**
     * Upload image to Firebase Storage
     */
    public void uploadImage(Uri imageUri, String path, final UploadCallback callback) {
        if (imageUri == null) { callback.onFailure("Image URI is null"); return; }
        // NOTE: This uses Application Context which may fail for FileProvider URIs.
        // Callers should read bytes first using their Activity context, then call uploadByt [...]
        byte[] bytes = readBytesFromUri(imageUri, storage.getApp().getApplicationContext());
        if (bytes == null || bytes.length == 0) {
            callback.onFailure("Cannot read file – try calling uploadBytes() with pre-read b [...]");
            return;
        }
        uploadBytes(bytes, path, callback);
    }
    // Overload that accepts a Context – use from Activities for FileProvider URIs
    public void uploadImage(android.content.Context ctx, Uri imageUri, String path, final Up [...]
        if (imageUri == null) { callback.onFailure("Image URI is null"); return; }
        byte[] bytes = readBytesFromUri(imageUri, ctx);
        if (bytes == null || bytes.length == 0) {
            callback.onFailure("Cannot read file");
            return;
        }
        uploadBytes(bytes, path, callback);
    }

```

```

}
/**
 * Reads all bytes from any URI type synchronously on calling thread.
 * Must be called while URI permissions are still valid.
 */
private byte[] readBytesFromUri(Uri uri, android.content.Context ctx) {
    try {
        java.io.InputStream is = ctx.getContentResolver().openInputStream(uri);
        if (is == null) return null;
        java.io.ByteArrayOutputStream buffer = new java.io.ByteArrayOutputStream();
        byte[] chunk = new byte[8192];
        int read;
        while ((read = is.read(chunk)) != -1) buffer.write(chunk, 0, read);
        is.close();
        return buffer.toByteArray();
    } catch (Exception e) {
        Log.e(TAG, "readBytesFromUri failed: " + e.getMessage());
        return null;
    }
}
/**
 * Uploads image bytes – compresses to JPEG first if decodable as bitmap.
 * For audio/document files use uploadRawBytes() instead.
 */
public void uploadBytes(byte[] rawBytes, String path, UploadCallback callback) {
    if (rawBytes == null || rawBytes.length == 0) {
        callback.onFailure("No data to upload");
        return;
    }
    StorageReference fileRef = storage.getReference().child(path);
    new Thread(() -> {
        byte[] uploadBytes = rawBytes;
        try {
            // Only compress if it's actually an image
            android.graphics.Bitmap bmp = android.graphics.BitmapFactory
                .decodeByteArray(rawBytes, 0, rawBytes.length);
            if (bmp != null) {
                int maxDim = 1024;
                if (bmp.getWidth() > maxDim || bmp.getHeight() > maxDim) {
                    float scale = Math.min(
                        (float) maxDim / bmp.getWidth(),
                        (float) maxDim / bmp.getHeight());
                    bmp = android.graphics.Bitmap.createScaledBitmap(bmp,
                        (int)(bmp.getWidth() * scale),
                        (int)(bmp.getHeight() * scale), true);
                }
                java.io.ByteArrayOutputStream baos = new java.io.ByteArrayOutputStream() [...];
                bmp.compress(android.graphics.Bitmap.CompressFormat.JPEG, 85, baos);
                bmp.recycle();
                if (baos.size() > 0) uploadBytes = baos.toByteArray();
            }
            // If bmp == null → not an image → upload raw as-is (should not happen here)
        } catch (Exception ignored) {}
        doUpload(fileRef, uploadBytes, "image/jpeg", callback);
    }).start();
}
/**
 * Uploads raw bytes without any compression – use for audio and documents.
 */
public void uploadRawBytes(byte[] rawBytes, String path, String mimeType, UploadCallback [...]) {
    if (rawBytes == null || rawBytes.length == 0) {
        callback.onFailure("No data to upload");
        return;
    }
}

```

```

StorageReference fileRef = storage.getReference().child(path);
new Thread(() -> doUpload(fileRef, rawBytes, mimeType, callback)).start();
}
private void doUpload(StorageReference fileRef, byte[] bytes, String mimeType, UploadCal [...])
com.google.firebase.storage.StorageMetadata metadata =
new com.google.firebase.storage.StorageMetadata.Builder()
.setContentTypes(mimeType != null ? mimeType : "application/octet-stream")
.build();
fileRef.putBytes(bytes, metadata)
.addOnProgressListener(snap -> {
double p = (100.0 * snap.getBytesTransferred()) / snap.getTotalByteCount();
callback.onProgress((int) p);
})
.addOnSuccessListener(snap ->
fileRef.getDownloadUrl()
.addOnSuccessListener(uri -> {
callback.onSuccess(uri.toString());
Log.d(TAG, "Upload success: " + uri);
})
.addOnFailureListener(e -> callback.onFailure(e.getMessage()))
)
.addOnFailureListener(e -> {
Log.e(TAG, "putBytes failed: " + e.getMessage(), e);
callback.onFailure(e.getMessage());
});
}
/**
 * Upload document to Firebase Storage
 */
public void uploadDocument(Uri documentUri, String path, final UploadCallback callback) [...]
if (documentUri == null) {
callback.onFailure("Document URI is null");
return;
}
String fileName = UUID.randomUUID().toString();
StorageReference fileRef = storage.getReference().child(path + "/" + fileName);
UploadTask uploadTask = fileRef.putFile(documentUri);
uploadTask.addOnProgressListener(taskSnapshot -> {
double progress = (100.0 * taskSnapshot.getBytesTransferred()) / taskSnapshot.ge [...]
callback.onProgress((int) progress);
}).addOnSuccessListener(taskSnapshot -> {
fileRef.getDownloadUrl().addOnSuccessListener(uri -> {
callback.onSuccess(uri.toString());
Log.d(TAG, "Document uploaded successfully: " + uri.toString());
}).addOnFailureListener(e -> {
callback.onFailure(e.getMessage());
Log.e(TAG, "Error getting download URL", e);
});
}).addOnFailureListener(e -> {
callback.onFailure(e.getMessage());
Log.e(TAG, "Error uploading document", e);
});
}
/**
 * Delete file from Firebase Storage
 */
public void deleteFile(String fileUrl, final VoidCallback callback) {
try {
StorageReference fileRef = storage.getReferenceFromUrl(fileUrl);
fileRef.delete()
.addOnSuccessListener(aVoid -> {
callback.onSuccess();
Log.d(TAG, "File deleted successfully");
})
}

```

```

        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error deleting file", e);
        });
    } catch (IllegalArgumentException e) {
        callback.onFailure("Invalid file URL");
        Log.e(TAG, "Invalid file URL", e);
    }
}

// ===== UTILITY METHODS =====
/**
 * Remove all listeners
 */
public void removeAllListeners() {
    for (ListenerRegistration registration : listenerRegistrations) {
        registration.remove();
    }
    listenerRegistrations.clear();
    Log.d(TAG, "All listeners removed");
}

/**
 * Generate conversation ID from two user IDs
 * Always generates the same ID regardless of order
 */
public static String generateConversationId(String userId1, String userId2) {
    if (userId1.compareTo(userId2) < 0) {
        return userId1 + "_" + userId2;
    } else {
        return userId2 + "_" + userId1;
    }
}

/**
 * Check if user has saved a job
 */
public void hasUserSavedJob(String userId, String jobId, final DataCallback<Boolean> cal [...]) {
    db.collection(SAVED_JOBS_COLLECTION)
        .document(userId + "_" + jobId)
        .get()
        .addOnSuccessListener(documentSnapshot -> {
            callback.onSuccess(documentSnapshot.exists());
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error checking saved job", e);
        });
}

/**
 * Save job for user
 */
public void saveJob(String userId, String jobId, final VoidCallback callback) {
    Map<String, Object> data = new HashMap<>();
    data.put("userId", userId);
    data.put("jobId", jobId);
    data.put("savedAt", System.currentTimeMillis());
    db.collection(SAVED_JOBS_COLLECTION)
        .document(userId + "_" + jobId)
        .set(data)
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "Job saved: " + jobId);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error saving job", e);
        });
}

```

```

});
}
/**
 * Unsave job for user
 */
public void unsaveJob(String userId, String jobId, final VoidCallback callback) {
    db.collection(SAVED_JOBS_COLLECTION)
        .document(userId + "_" + jobId)
        .delete()
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "Job unsaved: " + jobId);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error unsaving job", e);
        });
}
/**
 * Get all saved job IDs for a user – used by JobsFragment to initialise bookmark icons.
 * Delegates to getSavedJobs which already queries the savedJobs collection.
 */
public void getSavedJobIds(String userId, final ListCallback<String> callback) {
    getSavedJobs(userId, callback);
}
/**
 * Get saved jobs for user
 */
public void getSavedJobs(String userId, final ListCallback<String> callback) {
    db.collection(SAVED_JOBS_COLLECTION)
        .whereEqualTo("userId", userId)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            List<String> jobIds = new ArrayList<>();
            for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                String jobId = document.getString("jobId");
                if (jobId != null) jobIds.add(jobId);
            }
            callback.onSuccess(jobIds);
            Log.d(TAG, "Retrieved " + jobIds.size() + " saved jobs for user: " + use [...])
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error getting saved jobs", e);
        });
}
// ===== APPLICATION OPERATIONS =====
/**
 * Get applications submitted by a job seeker
 */
public void getApplicationsByJobSeeker(String jobSeekerUserId,
    final ListCallback<com.example.joblinker.models.A [...]
    db.collection(APPLICATIONS_COLLECTION)
        .whereEqualTo("jobSeekerUserId", jobSeekerUserId)
        // No orderBy – sort in memory to avoid composite index requirement
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            List<com.example.joblinker.models.Application> applications = new ArrayL [...]
            for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                com.example.joblinker.models.Application app =
                    document.toObject(com.example.joblinker.models.Application.c [...])
                applications.add(app);
            }
            // Sort newest first in memory

```

```

        applications.sort((a, b) -> Long.compare(b.getAppliedAt(), a.getAppliedA [...])
        callback.onSuccess(applications);
        Log.d(TAG, "Retrieved " + applications.size() + " applications for user: [...]
    })
    .addOnFailureListener(e -> {
        callback.onFailure(e.getMessage());
        Log.e(TAG, "Error getting applications", e);
    });
}
/**
 * Get applications for a specific job (for employers)
 */
public void getApplicationsByJob(String jobId,
    final ListCallback<com.example.joblinker.models.Applica [...])
    db.collection(APPLICATIONS_COLLECTION)
        .whereEqualTo("jobId", jobId)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            List<com.example.joblinker.models.Application> applications = new ArrayL [...]
            for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                com.example.joblinker.models.Application app =
                document.toObject(com.example.joblinker.models.Application.c [...])
                applications.add(app);
            }
            callback.onSuccess(applications);
            Log.d(TAG, "Retrieved " + applications.size() + " applications for job: [...]
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error getting job applications", e);
        });
    }
    /**
     * Submit a new application
     */
    public void submitApplication(com.example.joblinker.models.Application application,
        final VoidCallback callback) {
        DocumentReference docRef = db.collection(APPLICATIONS_COLLECTION).document();
        application.setApplicationId(docRef.getId());
        application.setAppliedAt(System.currentTimeMillis());
        docRef.set(application)
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "Application submitted: " + application.getApplicationId());
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error submitting application", e);
        });
    }
    /**
     * Update application status (for employers)
     */
    public void updateApplicationStatus(String applicationId, String status,
        final VoidCallback callback) {
        db.collection(APPLICATIONS_COLLECTION)
            .document(applicationId)
            .update("status", status)
            .addOnSuccessListener(aVoid -> {
                callback.onSuccess();
                Log.d(TAG, "Application status updated: " + applicationId + " -> " + sta [...])
            })
            .addOnFailureListener(e -> {
                callback.onFailure(e.getMessage());
            });
    }
}

```



```
Log.e(TAG, "Error updating application status", e);
});
}
/**
 * Delete / withdraw an application
 */
public void deleteApplication(String applicationId, final VoidCallback callback) {
    db.collection(APPLICATIONS_COLLECTION)
        .document(applicationId)
        .delete()
        .addOnSuccessListener(aVoid -> {
            callback.onSuccess();
            Log.d(TAG, "Application deleted: " + applicationId);
        })
        .addOnFailureListener(e -> {
            callback.onFailure(e.getMessage());
            Log.e(TAG, "Error deleting application", e);
        });
}
}
```


Source Code

Activities – Screen Controllers

BaseActivity

Abstract base class for ALL activities. Handles language/locale switching via LocaleHelper in attachBaseContext(). All other activities extend this class.

BaseActivity.java (Java)

```
package com.example.joblinker.activities;
import android.content.Context;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import com.example.joblinker.utils.LocaleHelper;
public class BaseActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        // Apply saved language
        updateLanguage();
    }
    @Override
    protected void attachBaseContext(Context newBase) {
        String language = LocaleHelper.getLanguage(newBase);
        super.attachBaseContext(LocaleHelper.setLocale(newBase, language));
    }
    protected void updateLanguage() {
        String language = LocaleHelper.getLanguage(this);
        LocaleHelper.setLocale(this, language);
    }
    protected void changeLanguage(String language) {
        String languageCode = LocaleHelper.getLanguageCode(language);
        LocaleHelper.setLocale(this, languageCode);
        recreate(); // Restart activity to apply changes
    }
}
```

SplashActivity

Entry point of the app. Displays animated splash screen, checks login state, and routes to appropriate screen.

SplashActivity.java (Java)

```
package com.example.joblinker.activities;
import android.content.Intent;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.view.animation.AlphaAnimation;
import android.view.animation.Animation;
import android.view.animation.ScaleAnimation;
import android.view.animation.AnimationSet;
import android.widget.ImageView;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
import com.example.joblinker.R;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.utils.SharedPreferencesManager;
public class SplashActivity extends AppCompatActivity {
    private final Handler handler = new Handler(Looper.getMainLooper());
    // true after the 2-second splash delay elapses
    private boolean splashReady = false;
    // true between onStart() and onStop() – i.e. app is in foreground
    private boolean isVisible = false;
    private JobLinkerFirebaseManager firebaseManager;
    private SharedPreferencesManager prefsManager;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_splash);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        prefsManager = SharedPreferencesManager.getInstance(this);
        ImageView ivLogo = findViewById(R.id.iv_logo);
        TextView tvAppName = findViewById(R.id.tv_app_name);
        TextView tvTagline = findViewById(R.id.tv_tagline);
        // Logo: scale up + fade in
        AnimationSet logoAnim = new AnimationSet(true);
        ScaleAnimation scale = new ScaleAnimation(0.4f, 1f, 0.4f, 1f,
            Animation.RELATIVE_TO_SELF, 0.5f, Animation.RELATIVE_TO_SELF, 0.5f);
        scale.setDuration(600);
        AlphaAnimation fadeIn = new AlphaAnimation(0f, 1f);
        fadeIn.setDuration(600);
        logoAnim.addAnimation(scale);
        logoAnim.addAnimation(fadeIn);
        logoAnim.setFillAfter(true);
        ivLogo.startAnimation(logoAnim);
        // App name: fade in with delay
        AlphaAnimation nameAnim = new AlphaAnimation(0f, 1f);
        nameAnim.setDuration(500);
        nameAnim.setStartOffset(500);
        nameAnim.setFillAfter(true);
        if (tvAppName != null) tvAppName.startAnimation(nameAnim);
        // Tagline: fade in later
        if (tvTagline != null) {
            AlphaAnimation tagAnim = new AlphaAnimation(0f, 1f);
            tagAnim.setDuration(500);
```

```
tagAnim.setStartOffset(800);
tagAnim.setFillAfter(true);
tvTagline.startAnimation(tagAnim);
}
// After 2s mark splash as ready; if the app is still visible, navigate
// immediately. If the screen was locked in the meantime, navigation is
// deferred to onStart() – this avoids the BAL (Background Activity Launch)
// block that Android 10+ enforces on TOP_SLEEPING processes.
handler.postDelayed(() -> {
    splashReady = true;
    if (isVisible) navigate();
}, 2000);
}
@Override
protected void onStart() {
    super.onStart();
    isVisible = true;
    // If the 2-second delay already elapsed while we were in background, navigate now
    if (splashReady) navigate();
}
@Override
protected void onStop() {
    super.onStop();
    isVisible = false;
}
@Override
protected void onDestroy() {
    super.onDestroy();
    handler.removeCallbacksAndMessages(null);
}
private void navigate() {
    if (isFinishing() || isDestroyed()) return;
    Intent intent;
    if (prefsManager.isLoggedIn() && firebaseManager.getCurrentUser() != null) {
        intent = new Intent(this, MainActivity.class);
    } else {
        intent = new Intent(this, LoginActivity.class);
    }
    startActivity(intent);
    overridePendingTransition(android.R.anim.fade_in, android.R.anim.fade_out);
    finish();
}
}
```

LoginActivity

Hosts the login UI with two tabs (Email and Phone) using ViewPager2.

LoginActivity.java (Java)

```
package com.example.joblinker.activities;
import android.content.Intent;
import android.os.Bundle;
import android.widget.TextView;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.viewpager2.widget.ViewPager2;
import com.example.joblinker.models.User;
import com.google.android.material.tabs.TabLayout;
import com.google.android.material.tabs.TabLayoutMediator;
import com.google.firebase.auth.FirebaseAuth;
import com.example.joblinker.R;
import com.example.joblinker.adapters.LoginPagerAdapter;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.utils.SharedPreferencesManager;
public class LoginActivity extends AppCompatActivity {
    private TabLayout tabLayout;
    private ViewPager2 viewPager;
    private TextView tvSignUp;
    private LoginPagerAdapter pagerAdapter;
    private JobLinkerFirebaseManager firebaseManager;
    private SharedPreferencesManager prefsManager;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        prefsManager = SharedPreferencesManager.getInstance(this);
        // Check if already logged in
        if (prefsManager.isLoggedIn() && firebaseManager.getCurrentUser() != null) {
            navigateToMain();
            return;
        }
        initializeViews();
        setupViewPager();
        setupClickListeners();
    }
    private void initializeViews() {
        tabLayout = findViewById(R.id.tab_layout);
        viewPager = findViewById(R.id.view_pager);
        tvSignUp = findViewById(R.id.tv_sign_up);
    }
    private void setupViewPager() {
        pagerAdapter = new LoginPagerAdapter(this);
        viewPager.setAdapter(pagerAdapter);
        // Connect TabLayout with ViewPager2
        new TabLayoutMediator(tabLayout, viewPager,
            new TabLayoutMediator.TabConfigurationStrategy() {
                @Override
                public void onConfigureTab(@NonNull TabLayout.Tab tab, int position) {
                    if (position == LoginPagerAdapter.TAB_EMAIL) {
                        tab.setText(R.string.email);
                    } else if (position == LoginPagerAdapter.TAB_PHONE) {
```

```

tab.setText(R.string.phone);
}
}
}
).attach();
// Optional: Disable swipe between tabs
// viewPager.setUserInputEnabled(false);
}
private void setupClickListeners() {
tvSignUp.setOnClickListener(v -> {
Intent intent = new Intent(LoginActivity.this, RegisterActivity.class);
startActivity(intent);
});
}
/**
 * Called by login fragments when login is successful
 */
public void onLoginSuccess(FirebaseUser user) {
// Save login state
prefsManager.setLoggedIn(true);
prefsManager.setUserId(user.getUid());
if (user.getEmail() != null) {
prefsManager.setUserEmail(user.getEmail());
}
// Load user data and navigate
firebaseManager.getUser(user.getUid(),
new JobLinkerFirebaseManager.DataCallback<User>() {
@Override
public void onSuccess(com.example.joblinker.models.User userData) {
// Save user data to preferences
prefsManager.setUserName(userData.getUserName());
prefsManager.setUserRole(userData.getUserRole());
if (userData.getAvatarUrl() != null) {
prefsManager.setUserAvatar(userData.getAvatarUrl());
}
if (userData.getUserLanguage() != null) {
prefsManager.setLanguage(userData.getUserLanguage());
}
if (userData.getUserCurrency() != null) {
prefsManager.setCurrency(userData.getUserCurrency());
}
// Navigate to main activity
navigateToMain();
}
@Override
public void onFailure(String error) {
// User data not found, but login successful
// This might happen for new users
Toast.makeText(LoginActivity.this,
"Please complete your profile", Toast.LENGTH_SHORT).show();
navigateToMain();
}
});
}
/**
 * Navigate to MainActivity
 */
private void navigateToMain() {
Intent intent = new Intent(LoginActivity.this, MainActivity.class);
intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
startActivity(intent);
finish();
}
}
}

```



```
* Get current tab position
*/
public int getCurrentTab() {
    return viewPager.getCurrentItem();
}
/**
 * Switch to specific tab
 */
public void switchToTab(int position) {
    if (position >= 0 && position < pagerAdapter.getItemCount()) {
        viewPager.setCurrentItem(position, true);
    }
}
@Override
public void onBackPressed() {
    if (viewPager.getCurrentItem() == 0) {
        // If on first tab, allow normal back behavior
        super.onBackPressed();
    } else {
        // Otherwise, go back to previous tab
        viewPager.setCurrentItem(viewPager.getCurrentItem() - 1);
    }
}
}
```

RegisterActivity

Multi-step registration wizard with 3 steps using ViewPager2 and step indicators.

RegisterActivity.java (Java)

```
package com.example.joblinker.activities;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;
import androidx.viewpager2.widget.ViewPager2;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.button.MaterialButton;
import com.example.joblinker.R;
import com.example.joblinker.adapters.RegisterPagerAdapter;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.fragments.RegisterStep1Fragment;
import com.example.joblinker.fragments.RegisterStep2Fragment;
import com.example.joblinker.fragments.RegisterStep3Fragment;
import com.example.joblinker.models.User;
import com.example.joblinker.utils.SharedPreferencesManager;
public class RegisterActivity extends AppCompatActivity {
    private MaterialToolbar toolbar;
    private ViewPager2 viewPager;
    private MaterialButton btnBack, btnNext;
    private View indicatorStep1, indicatorStep2, indicatorStep3;
    private RegisterPagerAdapter pagerAdapter;
    private JobLinkerFirebaseManager firebaseManager;
    private SharedPreferencesManager prefsManager;
    private int currentStep = 0;
    // THESE HOLD YOUR DATA ACROSS ALL FRAGMENTS
    private User registrationUser;
    private String registrationPassword;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_register);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        prefsManager = SharedPreferencesManager.getInstance(this);
        // Initialize the User object so it's never null
        registrationUser = new User();
        initializeViews();
        setupToolbar();
        setupViewPager();
        setupClickListeners();
        updateStepIndicators();
    }
    private void initializeViews() {
        toolbar = findViewById(R.id.toolbar);
        viewPager = findViewById(R.id.view_pager);
        btnBack = findViewById(R.id.btn_back);
        btnNext = findViewById(R.id.btn_next);
        indicatorStep1 = findViewById(R.id.indicator_step1);
        indicatorStep2 = findViewById(R.id.indicator_step2);
        indicatorStep3 = findViewById(R.id.indicator_step3);
    }
    private void setupToolbar() {
```

```

setSupportActionBar(toolbar);
toolbar.setNavigationOnClickListener(v -> onBackPressed());
}
private void setupViewPager() {
    pagerAdapter = new RegisterPagerAdapter(this);
    viewPager.setAdapter(pagerAdapter);
    viewPager.setUserInputEnabled(false); // Disable swipe
    viewPager.registerOnPageChangeCallback(new ViewPager2.OnPageChangeCallback() {
        @Override
        public void onPageSelected(int position) {
            super.onPageSelected(position);
            currentStep = position;
            updateStepIndicators();
            updateButtons();
        }
    });
}
private void setupClickListeners() {
    btnBack.setOnClickListener(v -> {
        if (currentStep > 0) {
            viewPager.setCurrentItem(currentStep - 1, true);
        }
    });
    btnNext.setOnClickListener(v -> {
        if (validateCurrentStep()) {
            if (currentStep < 2) {
                viewPager.setCurrentItem(currentStep + 1, true);
            } else {
                completeRegistration();
            }
        }
    });
}
private void updateStepIndicators() {
    int primaryColor = getResources().getColor(R.color.primary, null);
    int dividerColor = getResources().getColor(R.color.divider, null);
    indicatorStep1.setBackgroundColor(currentStep >= 0 ? primaryColor : dividerColor);
    indicatorStep2.setBackgroundColor(currentStep >= 1 ? primaryColor : dividerColor);
    indicatorStep3.setBackgroundColor(currentStep >= 2 ? primaryColor : dividerColor);
}
private void updateButtons() {
    btnBack.setVisibility(currentStep > 0 ? View.VISIBLE : View.GONE);
    if (currentStep == 2) {
        btnNext.setText(R.string.complete_registration);
    } else {
        btnNext.setText(R.string.next);
    }
}
// --- MAGIC FIX 1: Correctly get ViewPager2 Fragments ---
private Fragment getCurrentFragment() {
    long itemId = pagerAdapter.getItemId(viewPager.getCurrentItem());
    return getSupportFragmentManager().findFragmentByTag("f" + itemId);
}
// --- MAGIC FIX 2: Use the safe fragment getter ---
private boolean validateCurrentStep() {
    Fragment currentFragment = getCurrentFragment();
    if (currentFragment instanceof RegisterStep1Fragment) {
        return ((RegisterStep1Fragment) currentFragment).validateAndSaveData();
    } else if (currentFragment instanceof RegisterStep2Fragment) {
        return ((RegisterStep2Fragment) currentFragment).validateAndSaveData();
    } else if (currentFragment instanceof RegisterStep3Fragment) {
        return true;
    }
}
// If fragment is null, return FALSE so they can't skip validation

```

```
return false;
}
private void completeRegistration() {
    Fragment currentFragment = getCurrentFragment();
    if (currentFragment instanceof RegisterStep3Fragment) {
        ((RegisterStep3Fragment) currentFragment).validateAndCompleteRegistration();
    }
}
public void onRegistrationComplete() {
    Toast.makeText(this, "Registration successful!", Toast.LENGTH_SHORT).show();
    navigateToMain();
}
private void navigateToMain() {
    Intent intent = new Intent(RegisterActivity.this, MainActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
    startActivity(intent);
    finish();
}
// Getters and Setters for registration data
public User getRegistrationUser() {
    return registrationUser;
}
public void setRegistrationUser(User user) {
    this.registrationUser = user;
}
public String getRegistrationPassword() {
    return registrationPassword;
}
public void setRegistrationPassword(String password) {
    this.registrationPassword = password;
}
// --- ADDED THIS METHOD BACK IN ---
public void moveToNextStep() {
    if (currentStep < 2) {
        viewPager.setCurrentItem(currentStep + 1, true);
    }
}
@Override
public void onBackPressed() {
    if (currentStep > 0) {
        viewPager.setCurrentItem(currentStep - 1, true);
    } else {
        super.onBackPressed();
    }
}
}
```

MainActivity

Main hub with BottomNavigationView hosting Jobs, Chats, and Profile fragments.

MainActivity.java (Java)

```
package com.example.joblinker.activities;
import android.content.Intent;
import android.os.Bundle;
import android.view.MenuItem;
import android.view.View;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentTransaction;
import com.example.joblinker.R;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.fragments.ChatsFragment;
import com.example.joblinker.fragments.JobsFragment;
import com.example.joblinker.fragments.ProfileFragment;
import com.example.joblinker.utils.LogoutHelper;
import com.google.android.material.badge.BadgeDrawable;
import com.google.android.material.bottomnavigation.BottomNavigationView;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.ListenerRegistration;
public class MainActivity extends BaseActivity {
    private BottomNavigationView bottomNav;
    // Cached fragment instances – never recreated on tab switch
    private JobsFragment jobsFragment;
    private ChatsFragment chatsFragment;
    private ProfileFragment profileFragment;
    private Fragment activeFragment;
    // Unread message listener
    private ListenerRegistration unreadListener;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        bottomNav = findViewById(R.id.bottomNavigationView);
        setupFragments();
        setupBottomNav();
        listenForUnreadMessages();
    }
    // ■■ Create all fragments once, hide/show instead of replace ■■
    private void setupFragments() {
        jobsFragment = new JobsFragment();
        chatsFragment = new ChatsFragment();
        profileFragment = new ProfileFragment();
        getSupportFragmentManager().beginTransaction()
            .add(R.id.fragmentContainer, profileFragment, "profile").hide(profileFragment)
            .add(R.id.fragmentContainer, chatsFragment, "chats").hide(chatsFragment)
            .add(R.id.fragmentContainer, jobsFragment, "jobs")
            .commit();
        activeFragment = jobsFragment;
    }
    private void setupBottomNav() {
        bottomNav.setOnItemSelectedListener(item -> {
            int id = item.getItemId();
            if (id == R.id.nav_jobs) switchTo(jobsFragment);
            else if (id == R.id.nav_chats) switchTo(chatsFragment);
        });
    }
    private void switchTo(Fragment fragment) {
        FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
        transaction.hide(activeFragment);
        transaction.add(R.id.fragmentContainer, fragment, fragment.getClass().getSimpleName());
        transaction.commit();
        activeFragment = fragment;
    }
}
```

```

else if (id == R.id.nav_profile) switchTo(profileFragment);
return true;
});
}

private void switchTo(Fragment target) {
if (target == activeFragment) return;
getSupportFragmentManager().beginTransaction()
.hide(activeFragment)
.show(target)
.setTransition(FragmentTransaction.TRANSIT_FRAGMENT_FADE)
.commit();
activeFragment = target;
}

// ■■ Unread messages badge on Chats tab ■■■■■■■■
private void listenForUnreadMessages() {
String uid = FirebaseAuth.getInstance().getUid();
if (uid == null) return;
unreadListener = FirebaseFirestore.getInstance()
.collection("messages")
.whereEqualTo("messageReceiverId", uid)
.whereEqualTo("messageRead", false)
.addSnapshotListener((snap, e) -> {
if (snap == null) return;
int count = snap.size();
runOnUiThread(() -> {
BadgeDrawable badge = bottomNav.getOrCreateBadge(R.id.nav_chats);
if (count > 0) {
badge.setVisible(true);
badge.setNumber(count);
} else {
badge.setVisible(false);
badge.clearNumber();
}
});
});
}

@Override
public boolean onCreateOptionsMenu(android.view.Menu menu) {
getMenuInflater().inflate(R.menu.main_menu, menu);
return true;
}

@Override
public boolean onOptionsItemSelected(MenuItem item) {
int id = item.getItemId();
if (id == R.id.action_settings) {
startActivity(new Intent(this, SettingActivity.class));
return true;
} else if (id == R.id.action_logout) {
LogoutHelper.showLogoutDialog(this);
return true;
}
return super.onOptionsItemSelected(item);
}

@Override
protected void onDestroy() {
super.onDestroy();
if (unreadListener != null) unreadListener.remove();
}
}

```

PostJobActivity

Job posting form for Employers. Validates and writes job data to Firestore.

PostJobActivity.java (Java)

```
package com.example.joblinker.activities;
import android.app.DatePickerDialog;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.AutoCompleteTextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.chip.Chip;
import com.google.android.material.chip.ChipGroup;
import com.google.android.material.textfield.TextInputEditText;
import com.example.joblinker.R;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.models.Job;
import com.example.joblinker.utils.ValidationHelper;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.List;

public class PostJobActivity extends AppCompatActivity {
    private MaterialToolbar toolbar;
    private TextInputEditText etJobTitle, etCompanyName, etCity, etSalaryMin, etSalaryMax;
    private TextInputEditText etDescription, etDeadline, etSkillInput;
    private AutoCompleteTextView etCategory, etEmploymentType, etCountry;
    private MaterialButton btnAddSkill, btnCancel, btnPost;
    private ChipGroup chipGroupSkills;
    private View progressOverlay;
    private com.google.android.material.switchmaterial.SwitchMaterial switchUrgent;
    private JobLinkerFirebaseManager firebaseManager;
    private List<String> skills;
    private long deadlineTimestamp = 0;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_post_job);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        skills = new ArrayList<>();
        initializeViews();
        setupToolbar();
        setupDropdowns();
        setupClickListeners();
    }

    private void initializeViews() {
        toolbar = findViewById(R.id.toolbar);
        etJobTitle = findViewById(R.id.et_job_title);
        etCompanyName = findViewById(R.id.et_company_name);
        etCategory = findViewById(R.id.et_category);
        etEmploymentType = findViewById(R.id.et_employment_type);
        etCountry = findViewById(R.id.et_country);
        etCity = findViewById(R.id.et_city);
        etSalaryMin = findViewById(R.id.et_salary_min);
        etSalaryMax = findViewById(R.id.et_salary_max);
        etSkillInput = findViewById(R.id.et_skill_input);
    }
```

```

btnAddSkill = findViewById(R.id.btn_add_skill);
chipGroupSkills = findViewById(R.id.chip_group_skills);
etDescription = findViewById(R.id.et_description);
etDeadline = findViewById(R.id.et_deadline);
btnCancel = findViewById(R.id.btn_cancel);
btnPost = findViewById(R.id.btn_post);
progressOverlay = findViewById(R.id.progress_overlay);
switchUrgent = findViewById(R.id.switch_urgent);
}
private void setupToolbar() {
    setSupportActionBar(toolbar);
    toolbar.setNavigationOnClickListener(v -> onBackPressed());
}
private void setupDropdowns() {
    // Categories
    String[] categories = getResources().getStringArray(R.array.job_categories);
    ArrayAdapter<String> categoryAdapter = new ArrayAdapter<>(this,
        android.R.layout.simple_dropdown_item_1line, categories);
    etCategory.setAdapter(categoryAdapter);
    // Employment Types
    String[] employmentTypes = getResources().getStringArray(R.array.employment_types);
    ArrayAdapter<String> typeAdapter = new ArrayAdapter<>(this,
        android.R.layout.simple_dropdown_item_1line, employmentTypes);
    etEmploymentType.setAdapter(typeAdapter);
    // Countries
    String[] countries = getResources().getStringArray(R.array.countries);
    ArrayAdapter<String> countryAdapter = new ArrayAdapter<>(this,
        android.R.layout.simple_dropdown_item_1line, countries);
    etCountry.setAdapter(countryAdapter);
}
private void setupClickListeners() {
    btnAddSkill.setOnClickListener(v -> addSkill());
    etDeadline.setOnClickListener(v -> showDatePicker());
    btnCancel.setOnClickListener(v -> finish());
    btnPost.setOnClickListener(v -> postJob());
}
private void addSkill() {
    String skill = etSkillInput.getText().toString().trim();
    if (ValidationHelper.isEmpty(skill)) {
        etSkillInput.setError("Enter a skill");
        return;
    }
    if (skills.contains(skill)) {
        Toast.makeText(this, "Skill already added", Toast.LENGTH_SHORT).show();
        return;
    }
    skills.add(skill);
    // Add chip
    Chip chip = new Chip(this);
    chip.setText(skill);
    chip.setCloseIconVisible(true);
    chip.setOnCloseIconClickListener(v -> {
        chipGroupSkills.removeView(chip);
        skills.remove(skill);
    });
    chipGroupSkills.addView(chip);
    etSkillInput.setText("");
}
private void showDatePicker() {
    Calendar calendar = Calendar.getInstance();
    DatePickerDialog datePickerDialog = new DatePickerDialog(
        this,
        (view, year, month, dayOfMonth) -> {
            calendar.set(year, month, dayOfMonth);

```



```

deadlineTimestamp = calendar.getTimeInMillis();
etDeadline.setText(String.format("%02d/%02d/%d", month + 1, dayOfMonth, [...]
},
calendar.get(Calendar.YEAR),
calendar.get(Calendar.MONTH),
calendar.get(Calendar.DAY_OF_MONTH)
);
// Set minimum date to today
datePickerDialog.getDatePicker().setMinDate(System.currentTimeMillis());
datePickerDialog.show();
}
private void postJob() {
// Validate inputs
String jobTitle = etJobTitle.getText().toString().trim();
String companyName = etCompanyName.getText().toString().trim();
String category = etCategory.getText().toString().trim();
String employmentType = etEmploymentType.getText().toString().trim();
String country = etCountry.getText().toString().trim();
String city = etCity.getText().toString().trim();
String salaryMinStr = etSalaryMin.getText().toString().trim();
String salaryMaxStr = etSalaryMax.getText().toString().trim();
String description = etDescription.getText().toString().trim();
// Validation
if (ValidationHelper.isEmpty(jobTitle)) {
etJobTitle.setError("Job title is required");
etJobTitle.requestFocus();
return;
}
if (ValidationHelper.isEmpty(companyName)) {
etCompanyName.setError("Company name is required");
etCompanyName.requestFocus();
return;
}
if (ValidationHelper.isEmpty(category)) {
etCategory.setError("Category is required");
etCategory.requestFocus();
return;
}
if (ValidationHelper.isEmpty(employmentType)) {
etEmploymentType.setError("Employment type is required");
etEmploymentType.requestFocus();
return;
}
if (ValidationHelper.isEmpty(country)) {
etCountry.setError("Country is required");
etCountry.requestFocus();
return;
}
if (ValidationHelper.isEmpty(description)) {
etDescription.setError("Job description is required");
etDescription.requestFocus();
return;
}
// Create Job object
Job job = new Job();
job.setJobTitle(jobTitle);
job.setJobCompany(companyName);
job.setJobEmployerId(firebaseManager.getCurrentUserId());
job.setJobCategory(category);
job.setJobType(employmentType);
job.setJobCountry(country);
job.setJobCity(city);
job.setJobDescription(description);
job.setJobSkills(skills);

```

```
job.setDeadline(deadlineTimestamp);
if (switchUrgent != null) job.setUrgent(switchUrgent.isChecked());
// Parse salary – guard against non-numeric input
if (!ValidationHelper.isEmpty(salaryMinStr)) {
    try {
        job.setJobSalaryMin(Double.parseDouble(salaryMinStr));
    } catch (NumberFormatException e) {
        Toast.makeText(this, "Invalid minimum salary value", Toast.LENGTH_SHORT).show [...]
        return;
    }
}
if (!ValidationHelper.isEmpty(salaryMaxStr)) {
    try {
        job.setJobSalaryMax(Double.parseDouble(salaryMaxStr));
    } catch (NumberFormatException e) {
        Toast.makeText(this, "Invalid maximum salary value", Toast.LENGTH_SHORT).show [...]
        return;
    }
}
// Show progress
progressOverlay.setVisibility(View.VISIBLE);
btnPost.setEnabled(false);
// Post job to Firebase
firebaseManager.createJob( job, new JobLinkerFirebaseManager.DataCallback<String>() [...]
@Override
public void onSuccess(String jobId) {
    progressOverlay.setVisibility(View.GONE);
    Toast.makeText(PostJobActivity.this,
        R.string.job_posted_successfully, Toast.LENGTH_SHORT).show();
    finish();
}
@Override
public void onFailure(String error) {
    progressOverlay.setVisibility(View.GONE);
    btnPost.setEnabled(true);
    Toast.makeText(PostJobActivity.this,
        "Error: " + error, Toast.LENGTH_SHORT).show();
}
});
}
```

JobDetailsActivity

Detailed view of a job posting with apply, save, share, and report functionality.

JobDetailsActivity.java (Java)

```
package com.example.joblinker.activities;
import android.content.Intent;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.ImageButton;
import android.widget.ImageView;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.widget.NestedScrollView;
import com.example.joblinker.R;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.models.Job;
import com.example.joblinker.models.User;
import com.example.joblinker.utils.DateTimeHelper;
import com.example.joblinker.utils.ImageUtils;
import com.example.joblinker.utils.SharedPreferencesManager;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.card.MaterialCardView;
import com.google.android.material.chip.Chip;
import com.google.android.material.chip.ChipGroup;
public class JobDetailsActivity extends AppCompatActivity {
    public static final String EXTRA_JOB_ID = "job_id";
    public static final String EXTRA_JOB = "job";
    private MaterialToolbar toolbar;
    private NestedScrollView scrollView;
    private ProgressBar progressBar;
    // Header Views
    private ImageView ivCompanyLogo;
    private TextView tvJobTitle;
    private TextView tvCompanyName;
    private TextView tvLocation;
    private TextView tvSalary;
    private TextView tvPostedTime;
    private Chip chipCategory;
    private Chip chipJobType;
    private ImageButton btnSave;
    // Details Views
    private TextView tvDescription;
    private ChipGroup chipGroupSkills;
    private TextView tvDeadline;
    private TextView tvApplications;
    private TextView tvViews;
    // Employer Info
    private MaterialCardView layoutEmployerInfo; // <-- FIX: was LinearLayout
    private ImageView ivEmployerAvatar;
    private TextView tvEmployerName;
    private TextView tvEmployerEmail;
    private MaterialButton btnContactEmployer;
    // Action Buttons
```

```

private MaterialButton btnApply;
private MaterialButton btnShare;
private JobLinkerFirebaseManager firebaseManager;
private SharedPreferencesManager prefsManager;
private Job currentJob;
private String jobId;
private boolean isSaved = false;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_job_details);
    firebaseManager = JobLinkerFirebaseManager.getInstance();
    prefsManager = SharedPreferencesManager.getInstance(this);
    initializeViews();
    setupToolbar();
    getIntentData();
    setupClickListeners();
    loadJobDetails();
}
private void initializeViews() {
    toolbar = findViewById(R.id.toolbar);
    scrollView = findViewById(R.id.scroll_view);
    progressBar = findViewById(R.id.progress_bar);
    // Header
    ivCompanyLogo = findViewById(R.id.iv_company_logo);
    tvJobTitle = findViewById(R.id.tv_job_title);
    tvCompanyName = findViewById(R.id.tv_company_name);
    tvLocation = findViewById(R.id.tv_location);
    tvSalary = findViewById(R.id.tv_salary);
    tvPostedTime = findViewById(R.id.tv_posted_time);
    chipCategory = findViewById(R.id.chip_category);
    chipJobType = findViewById(R.id.chip_job_type);
    btnSave = findViewById(R.id.btn_save);
    // Details
    tvDescription = findViewById(R.id.tv_description);
    chipGroupSkills = findViewById(R.id.chip_group_skills);
    tvDeadline = findViewById(R.id.tv_deadline);
    tvApplications = findViewById(R.id.tv_applications);
    tvViews = findViewById(R.id.tv_views);
    // Employer Info
    layoutEmployerInfo = findViewById(R.id.layout_employer_info);
    ivEmployerAvatar = findViewById(R.id.iv_employer_avatar);
    tvEmployerName = findViewById(R.id.tv_employer_name);
    tvEmployerEmail = findViewById(R.id.tv_employer_email);
    btnContactEmployer = findViewById(R.id.btn_contact_employer);
    // Action Buttons
    btnApply = findViewById(R.id.btn_apply);
    btnShare = findViewById(R.id.btn_share);
}
private void setupToolbar() {
    setSupportActionBar(toolbar);
    toolbar.setNavigationOnClickListener(v -> onBackPressed());
}
private void getIntentData() {
    Intent intent = getIntent();
    // Try to get job object first
    if (intent.hasExtra(EXTRA_JOB)) {
        currentJob = (Job) intent.getSerializableExtra(EXTRA_JOB);
        if (currentJob != null) {
            jobId = currentJob.getJobId();
        }
    }
    // Fall back to job ID
    if (jobId == null && intent.hasExtra(EXTRA_JOB_ID)) {

```

```

    jobId = intent.getStringExtra(EXTRA_JOB_ID);
}
}
private void setupClickListeners() {
    btnSave.setOnClickListener(v -> toggleSaveJob());
    btnApply.setOnClickListener(v -> applyForJob());
    btnShare.setOnClickListener(v -> shareJob());
    btnContactEmployer.setOnClickListener(v -> contactEmployer());
}
private void loadJobDetails() {
    if (currentJob != null) {
        displayJobDetails(currentJob);
        incrementViewCount();
        checkIfSaved();
    } else if (jobId != null) {
        showLoading(true);
        firebaseManager.getJob(jobId, new JobLinkerFirebaseManager.DataCallback<Job>() {
            @Override
            public void onSuccess(Job job) {
                showLoading(false);
                currentJob = job;
                displayJobDetails(job);
                incrementViewCount();
                checkIfSaved();
            }
            @Override
            public void onFailure(String error) {
                showLoading(false);
                Toast.makeText(JobDetailsActivity.this,
                    "Error loading job: " + error, Toast.LENGTH_SHORT).show();
                finish();
            }
        });
    } else {
        Toast.makeText(this, "Invalid job", Toast.LENGTH_SHORT).show();
        finish();
    }
}
private void displayJobDetails(Job job) {
    // Header
    ImageUtils.loadCompanyLogo(this, job.getCompanyLogoUrl(), ivCompanyLogo);
    tvJobTitle.setText(job.getJobTitle());
    tvCompanyName.setText(job.getJobCompany());
    tvLocation.setText(job.getLocation());
    tvSalary.setText(job.getSalaryRange());
    tvPostedTime.setText(DateTimeHelper.getRelativeTime(job.getCreatedAt()));
    chipCategory.setText(job.getJobCategory());
    chipJobType.setText(job.getJobType());
    // Description
    tvDescription.setText(job.getJobDescription());
    // Skills
    if (job.getJobSkills() != null && !job.getJobSkills().isEmpty()) {
        chipGroupSkills.setVisibility(View.VISIBLE);
        chipGroupSkills.removeAllViews();
        for (String skill : job.getJobSkills()) {
            Chip chip = new Chip(this);
            chip.setText(skill);
            chip.setClickable(false);
            chip.setChipBackgroundColorResource(R.color.surface_dark);
            chipGroupSkills.addView(chip);
        }
    } else {
        chipGroupSkills.setVisibility(View.GONE);
    }
}

```

```

// Deadline
if (job.getDeadline() > 0) {
    tvDeadline.setText(DateTimeHelper.formatDate(job.getDeadline()));
    tvDeadline.setVisibility(View.VISIBLE);
} else {
    tvDeadline.setVisibility(View.GONE);
}
// Statistics
tvApplications.setText(String.valueOf(job.getApplicantCount()));
tvViews.setText(String.valueOf(job.getViewCount()));
// Load employer info
loadEmployerInfo(job.getJobEmployerId());
// Apply button logic:
// - Hide for employers
// - Show "Already Applied" if user already applied
// - Show "Job Expired" if deadline passed
if (prefsManager.isEmployer()) {
    btnApply.setVisibility(View.GONE);
} else if (job.isDeadlinePassed()) {
    btnApply.setText(R.string.job_expired);
    btnApply.setEnabled(false);
} else {
    String currentUserId = firebaseManager.getCurrentUserId();
    if (currentUserId != null && job.hasUserApplied(currentUserId)) {
        btnApply.setText(R.string.already_applied);
        btnApply.setEnabled(false);
    } else {
        btnApply.setText(R.string.apply_now);
        btnApply.setEnabled(true);
    }
}
}
}
private void loadEmployerInfo(String employerId) {
    firebaseManager.getUser(employerId, new JobLinkerFirebaseManager.DataCallback<User>([...]
@Override
public void onSuccess(User employer) {
    layoutEmployerInfo.setVisibility(View.VISIBLE);
    ImageUtils.loadCircularImage(JobDetailsActivity.this,
        employer.getAvatarUrl(), ivEmployerAvatar);
    tvEmployerName.setText(employer.getUserName());
    tvEmployerEmail.setText(employer.getUserEmail());
}
@Override
public void onFailure(String error) {
    layoutEmployerInfo.setVisibility(View.GONE);
}
});
}
private void incrementViewCount() {
    if (jobId != null) {
        firebaseManager.incrementJobViewCount(jobId);
    }
}
private void checkIfSaved() {
    String userId = firebaseManager.getCurrentUserId();
    if (userId != null && jobId != null) {
        firebaseManager.hasUserSavedJob(userId, jobId,
            new JobLinkerFirebaseManager.DataCallback<Boolean>() {
@Override
public void onSuccess(Boolean saved) {
            isSaved = saved;
            updateSaveButton();
        }
    }
@Override

```

```

public void onFailure(String error) {
    // Ignore error
}
});
}
}
private void toggleSaveJob() {
    String userId = firebaseManager.getCurrentUserId();
    if (userId == null || jobId == null) {
        Toast.makeText(this, "Please login to save jobs", Toast.LENGTH_SHORT).show();
        return;
    }
    btnSave.setEnabled(false);
    if (isSaved) {
        // Unsave
        firebaseManager.unsaveJob(userId, jobId, new JobLinkerFirebaseManager.VoidCallba [...]
    @Override
    public void onSuccess() {
        isSaved = false;
        updateSaveButton();
        btnSave.setEnabled(true);
        Toast.makeText(JobDetailsActivity.this,
            "Job removed from saved", Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onFailure(String error) {
        btnSave.setEnabled(true);
        Toast.makeText(JobDetailsActivity.this,
            "Error: " + error, Toast.LENGTH_SHORT).show();
    }
});
} else {
    // Save
    firebaseManager.saveJob(userId, jobId, new JobLinkerFirebaseManager.VoidCallback [...]
    @Override
    public void onSuccess() {
        isSaved = true;
        updateSaveButton();
        btnSave.setEnabled(true);
        Toast.makeText(JobDetailsActivity.this,
            "Job saved successfully", Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onFailure(String error) {
        btnSave.setEnabled(true);
        Toast.makeText(JobDetailsActivity.this,
            "Error: " + error, Toast.LENGTH_SHORT).show();
    }
});
}
}
private void updateSaveButton() {
    if (isSaved) {
        btnSave.setImageResource(R.drawable.ic_bookmark);
    } else {
        btnSave.setImageResource(R.drawable.ic_bookmark_border);
    }
}
private void applyForJob() {
    if (currentJob == null) return;
    String userId = firebaseManager.getCurrentUserId();
    if (userId == null) {
        Toast.makeText(this, "Please login to apply", Toast.LENGTH_SHORT).show();
        return;
    }
}

```

[illegible]


```

"Error: " + error, Toast.LENGTH_SHORT).show();
}
});
}
private void shareJob() {
if (currentJob == null) return;
String shareText = String.format(
"Check out this job opportunity!\n\n" +
"%s at %s\n\n" +
"Location: %s\n" +
"Salary: %s\n\n" +
"Shared via JobLinker",
currentJob.getJobTitle(),
currentJob.getJobCompany(),
currentJob.getLocation(),
currentJob.getSalaryRange()
);
Intent shareIntent = new Intent(Intent.ACTION_SEND);
shareIntent.setType("text/plain");
shareIntent.putExtra(Intent.EXTRA_SUBJECT, currentJob.getJobTitle());
shareIntent.putExtra(Intent.EXTRA_TEXT, shareText);
startActivity(Intent.createChooser(shareIntent, "Share job via"));
}
private void contactEmployer() {
if (currentJob == null || jobId == null) {
Toast.makeText(this, "Job info not available", Toast.LENGTH_SHORT).show();
return;
}
// Read the raw Firestore document – most reliable way to get employer ID
// regardless of which field name was used when the job was posted
btnContactEmployer.setEnabled(false);
com.google.firebase.firestore.FirebaseFirestore.getInstance()
.collection("jobs")
.document(jobId)
.get()
.addOnSuccessListener(doc -> {
btnContactEmployer.setEnabled(true);
if (!doc.exists()) {
Toast.makeText(this, "Job not found", Toast.LENGTH_SHORT).show();
return;
}
// Try every possible field name for employer ID
String employerId = null;
for (String field : new String[]{"jobEmployerId", "employerId", "employer_id", [...]
String val = doc.getString(field);
if (val != null && !val.isEmpty()) {
employerId = val;
break;
}
}
if (employerId == null || employerId.isEmpty()) {
Toast.makeText(this,
"Employer contact not available for this job",
Toast.LENGTH_SHORT).show();
return;
}
// Get employer name – use already-loaded name or job company as fallback
String employerName = (tvEmployerName != null
&& tvEmployerName.getText().length() > 0)
? tvEmployerName.getText().toString()
: currentJob.getJobCompany();
openChat(employerId,
(employerName != null && !employerName.isEmpty()) ? employerName : "Empl [...]
})

```

```

.addOnFailureListener(e -> {
    btnContactEmployer.setEnabled(true);
    Toast.makeText(this, "Error: " + e.getMessage(), Toast.LENGTH_SHORT).show();
});
}
private void openChat(String employerId, String employerName) {
    Intent intent = new Intent(JobDetailsActivity.this, ChatActivity.class);
    intent.putExtra(ChatActivity.EXTRA_USER_ID, employerId);
    intent.putExtra(ChatActivity.EXTRA_USER_NAME, employerName);
    startActivity(intent);
}
private void showReportDialog() {
    String[] reasons = {
        "Fake or scam job",
        "Inappropriate content",
        "Duplicate listing",
        "Wrong category",
        "Misleading information",
        "Other"
    };
    final int[] selectedReason = {0};
    new androidx.appcompat.app.AlertDialog.Builder(this)
        .setTitle("Report this job")
        .setSingleChoiceItems(reasons, 0, (dialog, which) -> selectedReason[0] = which)
        .setPositiveButton("Submit Report", (dialog, which) -> {
            String reason = reasons[selectedReason[0]];
            submitReport(reason);
        })
        .setNegativeButton("Cancel", null)
        .show();
}
private void submitReport(String reason) {
    if (currentJob == null) return;
    // Save report to Firestore
    java.util.Map<String, Object> report = new java.util.HashMap<>();
    report.put("jobId", currentJob.getJobId());
    report.put("jobTitle", currentJob.getJobTitle());
    report.put("reason", reason);
    report.put("reportedBy", com.google.firebase.auth.FirebaseAuth.getInstance().getUid [...]);
    report.put("reportedAt", System.currentTimeMillis());
    com.google.firebase.firestore.FirebaseFirestore.getInstance()
        .collection("reports")
        .add(report)
        .addOnSuccessListener(ref ->
            Toast.makeText(this, "Report submitted. Thank you!", Toast.LENGTH_SHORT).show [...])
        .addOnFailureListener(e ->
            Toast.makeText(this, "Failed to submit report", Toast.LENGTH_SHORT).show());
    }
    private void showLoading(boolean show) {
        progressBar.setVisibility(show ? View.VISIBLE : View.GONE);
        scrollView.setVisibility(show ? View.GONE : View.VISIBLE);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.menu_job_details, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        int id = item.getItemId();
        if (id == R.id.action_share) {
            shareJob();
            return true;
        } else if (id == R.id.action_report) {

```

```
showReportDialog();  
return true;  
}  
return super.onOptionsItemSelected(item);  
}  
}
```

ChatActivity

Full-featured real-time messaging screen with support for text, image, voice, document, location, and contact messages.

ChatActivity.java (Java)

```
package com.example.joblinker.activities;
import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.media.MediaPlayer;
import android.media.MediaRecorder;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.provider.MediaStore;
import android.text.Editable;
import android.text.TextWatcher;
import android.util.Log;
import android.view.View;
import android.widget.ImageButton;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.core.content.FileProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import com.example.joblinker.R;
import com.example.joblinker.adapters.MessageAdapter;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.models.Message;
import com.example.joblinker.models.User;
import com.example.joblinker.utils.ImageUtils;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.google.android.material.textfield.TextInputEditText;
import com.google.firebase.firestore.ListenerRegistration;
import java.io.File;
import java.io.IOException;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;
public class ChatActivity extends AppCompatActivity {
    public static final String EXTRA_USER_ID = "user_id";
    public static final String EXTRA_USER_NAME = "user_name";
    public static final String EXTRA_USER_AVATAR = "user_avatar";
    public static final String EXTRA_CONVERSATION_ID = "conversation_id";
    private static final int REQUEST_RECORD_AUDIO = 201;
```

[illegible]

```

}
private void getIntentData() {
    Intent i = getIntent();
    otherUserId = i.getStringExtra(EXTRA_USER_ID);
    otherUserName = i.getStringExtra(EXTRA_USER_NAME);
    otherUserAvatar = i.getStringExtra(EXTRA_USER_AVATAR);
    conversationId = i.getStringExtra(EXTRA_CONVERSATION_ID);
    if (otherUserId == null || otherUserId.isEmpty()) {
        Toast.makeText(this, "Could not open chat – employer info missing",
            Toast.LENGTH_SHORT).show();
        finish();
        return;
    }
    if (conversationId == null)
        conversationId = JobLinkerFirebaseManager.generateConversationId(currentUserId, [...])
    }
    private void initializeViews() {
        toolbar = findViewById(R.id.toolbar);
        ivUserAvatar = findViewById(R.id.iv_user_avatar);
        tvUserName = findViewById(R.id.tv_user_name);
        tvStatus = findViewById(R.id.tv_status);
        tvTyping = findViewById(R.id.tv_typing);
        btnVoiceCall = findViewById(R.id.btn_voice_call);
        btnVideoCall = findViewById(R.id.btn_video_call);
        btnAttachment = findViewById(R.id.btn_attachment);
        btnEmoji = findViewById(R.id.btn_emoji);
        btnCamera = findViewById(R.id.btn_camera);
        btnCancelRecording = findViewById(R.id.btn_cancel_recording);
        layoutAttachmentTray = findViewById(R.id.layout_attachment_tray);
        layoutRecordingBar = findViewById(R.id.layout_recording_bar);
        viewRecordingDot = findViewById(R.id.view_recording_dot);
        tvRecordingTime = findViewById(R.id.tv_recording_time);
        trayCamera = findViewById(R.id.tray_camera);
        trayGallery = findViewById(R.id.tray_gallery);
        trayDocument = findViewById(R.id.tray_document);
        trayLocation = findViewById(R.id.tray_location);
        trayContact = findViewById(R.id.tray_contact);
        recyclerMessages = findViewById(R.id.recycler_messages);
        etMessage = findViewById(R.id.et_message);
        btnSend = findViewById(R.id.btn_send);
        tvUserName.setText(otherUserName);
        ImageUtils.loadCircularImage(this, otherUserAvatar, ivUserAvatar);
    }
    private void setupToolbar() {
        setSupportActionBar(toolbar);
        toolbar.setNavigationOnClickListener(v -> onBackPressed());
    }
    private void setupRecyclerView() {
        messageAdapter = new MessageAdapter(this, messages, currentUserId);
        LinearLayoutManager lm = new LinearLayoutManager(this);
        lm.setStackFromEnd(true);
        recyclerMessages.setLayoutManager(lm);
        recyclerMessages.setAdapter(messageAdapter);
    }
    private void setupClickListeners() {
        btnVoiceCall.setOnClickListener(v -> initiateCall("voice"));
        btnVideoCall.setOnClickListener(v -> initiateCall("video"));
        btnEmoji.setOnClickListener(v -> {
            etMessage.requestFocus();
            if (etMessage.isShown()) {
                etMessage.setInputType(
                    android.text.InputType.TYPE_CLASS_TEXT |
                    android.text.InputType.TYPE_TEXT_VARIATION_NORMAL);
            }
        })
    }

```

```

android.view.inputmethod.InputMethodManager imm =
(android.view.inputmethod.InputMethodManager) getSystemService(INPUT_MET [...])
if (imm != null) imm.showSoftInput(etMessage, 0);
});
btnCamera.setOnClickListener(v -> openCamera());
btnAttachment.setOnClickListener(v -> toggleAttachmentTray());
trayCamera.setOnClickListener(v -> { closeTray(); openCamera(); });
trayGallery.setOnClickListener(v -> { closeTray(); openGallery(); });
trayDocument.setOnClickListener(v -> { closeTray(); openDocument(); });
trayLocation.setOnClickListener(v -> { closeTray(); shareLocation(); });
trayContact.setOnClickListener(v -> { closeTray(); shareContact(); });
btnCancelRecording.setOnClickListener(v -> cancelRecording());
btnSend.setOnClickListener(v -> {
if (isRecording) { stopRecordingAndSend(); return; }
String text = etMessage.getText() != null ? etMessage.getText().toString().trim( [...])
if (!text.isEmpty()) sendTextMessage(text);
else startVoiceRecording();
});
btnSend.setOnLongClickListener(v -> { startVoiceRecording(); return true; });
etMessage.addTextChangedListener(new TextWatcher() {
@Override public void beforeTextChanged(CharSequence s, int i, int i1, int i2) { [...])
@Override public void afterTextChanged(Editable s) {}
@Override
public void onTextChanged(CharSequence s, int start, int before, int count) {
btnSend.setImageResource(s.length() > 0 ? R.drawable.ic_send : R.drawable.ic [...])
}
});
}
private void toggleAttachmentTray() {
if (layoutAttachmentTray.getVisibility() == View.VISIBLE) closeTray();
else {
layoutAttachmentTray.setVisibility(View.VISIBLE);
btnAttachment.setImageResource(R.drawable.ic_close);
}
}
private void closeTray() {
layoutAttachmentTray.setVisibility(View.GONE);
btnAttachment.setImageResource(R.drawable.ic_attachment);
}
private void openCamera() {
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
!= PackageManager.PERMISSION_GRANTED) {
ActivityCompat.requestPermissions(this,
new String[]{Manifest.permission.CAMERA}, REQUEST_CAMERA);
return;
}
try {
File photoFile = createTempImageFile();
cameraImageUri = FileProvider.getUriForFile(this,
getPackageName() + ".fileprovider", photoFile);
Intent intent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
intent.putExtra(MediaStore.EXTRA_OUTPUT, cameraImageUri);
cameraLauncher.launch(intent);
} catch (IOException e) {
Toast.makeText(this, "Could not open camera", Toast.LENGTH_SHORT).show();
}
}
private File createTempImageFile() throws IOException {
String timeStamp = new SimpleDateFormat("yyyyMMdd_HHmmss", Locale.getDefault()).format( [...])
File storageDir = getExternalFilesDir(android.os.Environment.DIRECTORY_PICTURES);
return File.createTempFile("IMG_" + timeStamp, ".jpg", storageDir);
}
private void openGallery() {
Intent intent;

```

```

if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU)
    intent = new Intent(MediaStore.ACTION_PICK_IMAGES);
else
    intent = new Intent(Intent.ACTION_PICK, MediaStore.Images.Media.EXTERNAL_CONTENT [...])
galleryLauncher.launch(intent);
}
private void openDocument() {
    Intent intent = new Intent(Intent.ACTION_GET_CONTENT);
    intent.setType("*/*");
    intent.addCategory(Intent.CATEGORY_OPENABLE);
    documentLauncher.launch(Intent.createChooser(intent, "Select Document"));
}
private void shareLocation() {
    android.location.LocationManager lm =
    (android.location.LocationManager) getSystemService(LOCATION_SERVICE);
    try {
        android.location.Location loc = null;
        if (ContextCompat.checkSelfPermission(this,
        Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION_G [...])
        loc = lm.getLastKnownLocation(android.location.LocationManager.GPS_PROVIDER) [...];
        if (loc == null)
        loc = lm.getLastKnownLocation(android.location.LocationManager.NETWORK_P [...])
    }
    if (loc != null) {
        String mapsUrl = "https://maps.google.com/?q=" + loc.getLatitude() + "," + loc.getLongitude() + "...";
        Message msg = new Message(conversationId, currentUserId, otherUserId,
        "■ Location: " + mapsUrl);
        msg.setMessageType("location");
        sendMessageObject(msg);
    } else {
        Toast.makeText(this, "Could not get location. Enable GPS and try again.",
        Toast.LENGTH_SHORT).show();
    }
} catch (SecurityException e) {
    ActivityCompat.requestPermissions(this,
    new String[]{Manifest.permission.ACCESS_FINE_LOCATION}, 203);
}
}
private void shareContact() {
    Intent intent = new Intent(Intent.ACTION_PICK,
    android.provider.ContactsContract.Contacts.CONTENT_URI);
    startActivityForResult(intent, 204);
}
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == 204 && resultCode == RESULT_OK && data != null) {
        Uri contactUri = data.getData();
        if (contactUri != null) {
            android.database.Cursor cursor = getContentResolver().query(
            contactUri, null, null, null);
            if (cursor != null && cursor.moveToFirst()) {
                int nameIdx = cursor.getColumnIndex(
                android.provider.ContactsContract.Contacts.DISPLAY_NAME);
                String name = nameIdx >= 0 ? cursor.getString(nameIdx) : "Contact";
                cursor.close();
                Message msg = new Message(conversationId, currentUserId, otherUserId,
                "■ Contact: " + name);
                msg.setMessageType("contact");
                sendMessageObject(msg);
            }
        }
    }
}
}
}
}
}

```



```
//
// VOICE RECORDING
//
private void startVoiceRecording() {
if (ContextCompat.checkSelfPermission(this, Manifest.permission.RECORD_AUDIO)
!= PackageManager.PERMISSION_GRANTED) {
ActivityCompat.requestPermissions(this,
new String[]{Manifest.permission.RECORD_AUDIO}, REQUEST_RECORD_AUDIO);
return;
}
isRecording = true;
recordingSeconds = 0;
layoutRecordingBar.setVisibility(View.VISIBLE);
btnSend.setImageResource(R.drawable.ic_stop);
tvRecordingTime.setText("0:00");
recordingTimerRunnable = new Runnable() {
@Override public void run() {
if (!isRecording) return;
recordingSeconds++;
int m = recordingSeconds / 60, s = recordingSeconds % 60;
tvRecordingTime.setText(String.format(Locale.getDefault(), "%d:%02d", m, s)) [...]
recordingHandler.postDelayed(this, 1000);
}
};
recordingHandler.postDelayed(recordingTimerRunnable, 1000);
startRecordingDotAnimation();
audioFilePath = getExternalCacheDir().getAbsolutePath()
+ "/voice_" + System.currentTimeMillis() + ".3gp";
final String filePath = audioFilePath;
new Thread(() -> {
try {
MediaRecorder recorder = new MediaRecorder();
recorder.setAudioSource(MediaRecorder.AudioSource.MIC);
recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
recorder.setOutputFile(filePath);
recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
recorder.prepare();
recorder.start();
// ■ FIX: check isRecording in case user cancelled before runOnUiThread fire [...]
runOnUiThread(() -> {
if (!isRecording) {
try { recorder.stop(); } catch (Exception ignored) {}
recorder.release();
new File(filePath).delete();
} else {
mediaRecorder = recorder;
}
});
} catch (Exception e) {
runOnUiThread(() -> {
isRecording = false;
recordingHandler.removeCallbacks(recordingTimerRunnable);
layoutRecordingBar.setVisibility(View.GONE);
btnSend.setImageResource(R.drawable.ic_mic);
Toast.makeText(this, "Failed to start recording: " + e.getMessage(),
Toast.LENGTH_SHORT).show();
});
}
}).start();
}
private void stopRecordingAndSend() {
if (!isRecording) return;
stopRecording();
String duration = tvRecordingTime != null ? tvRecordingTime.getText().toString() : " [...]"
}
```

```

try {
    java.io.File audioFile = new java.io.File(audioFilePath);
    if (!audioFile.exists() || audioFile.length() == 0) {
        Toast.makeText(this, "Recording failed", Toast.LENGTH_SHORT).show();
        return;
    }
    // ■ FIX: readFully guarantees ALL bytes are read – read() may return early
    byte[] audioBytes = new byte[(int) audioFile.length()];
    try (java.io.DataInputStream dis = new java.io.DataInputStream(
        new java.io.FileInputStream(audioFile))) {
        dis.readFully(audioBytes);
    }
    String uploadPath = "chat_audio/" + conversationId + "/" + System.currentTimeMillis [...]
    firebaseManager.uploadRawBytes(audioBytes, uploadPath, "audio/3gpp",
        new JobLinkerFirebaseManager.UploadCallback() {
            @Override public void onSuccess(String downloadUrl) {
                audioFile.delete();
                Message msg = new Message(conversationId, currentUserid, otherUs [...])
                "■ Voice message (" + duration + ")";
                msg.setMessageType("audio");
                msg.setImageUrl(downloadUrl);
                sendMessageObject(msg);
            }
            @Override public void onProgress(int progress) {}
            @Override public void onFailure(String error) {
                audioFile.delete();
                Toast.makeText(ChatActivity.this,
                    "Failed to send voice message", Toast.LENGTH_SHORT).show [...]
            }
        });
    } catch (Exception e) {
        Toast.makeText(this, "Could not send voice message: " + e.getMessage(),
            Toast.LENGTH_SHORT).show();
    }
}

private void cancelRecording() {
    if (!isRecording) return;
    stopRecording();
    File f = new File(audioFilePath);
    if (f.exists()) f.delete();
    Toast.makeText(this, "Recording cancelled", Toast.LENGTH_SHORT).show();
}

private void stopRecording() {
    isRecording = false;
    recordingHandler.removeCallbacks(recordingTimerRunnable);
    layoutRecordingBar.setVisibility(View.GONE);
    if (viewRecordingDot != null) viewRecordingDot.setAlpha(1f);
    btnSend.setImageResource(R.drawable.ic_mic);
    try {
        if (mediaRecorder != null) {
            mediaRecorder.stop();
            mediaRecorder.release();
            mediaRecorder = null;
        }
    } catch (Exception e) {
        mediaRecorder = null;
    }
}

private void startRecordingDotAnimation() {
    Runnable blink = new Runnable() {
        @Override public void run() {
            if (!isRecording) return;
            viewRecordingDot.animate().alpha(0f).setDuration(500)
                .withEndAction(() -> viewRecordingDot.animate().alpha(1f).setDuratio [...]

```



```

new android.media.AudioAttributes.Builder()
    .setContentType(android.media.AudioAttributes.CONTENT_TYPE_SPEECH [...])
    .setUsage(android.media.AudioAttributes.USAGE_MEDIA)
    .build();
audioPlayer.setDataSource(url);
audioPlayer.setOnPreparedListener(mp -> {
    // ■ FIX #4: stale-player guard – player may have been released already
    if (mp != player || audioPlayer != player) {
        try { mp.release(); } catch (Exception ignored) {}
        return;
    }
    // ■ FIX #2: request audio focus – without this the OS silences playback
    if (!requestAudioFocus()) {
        if (onStop != null) onStop.run();
        stopAudioPlayer();
        return;
    }
    if (onStart != null) onStart.run();
    mp.start();
});
audioPlayer.setOnCompletionListener(mp -> {
    abandonAudioFocus();
    if (onStop != null) onStop.run();
    stopAudioPlayer();
});
audioPlayer.setOnErrorListener((mp, what, extra) -> {
    Log.e("ChatActivity", "MediaPlayer error what=" + what + " extra=" + extra);
    Toast.makeText(this, "Cannot play audio", Toast.LENGTH_SHORT).show();
    abandonAudioFocus();
    if (onStop != null) onStop.run();
    stopAudioPlayer();
    return true;
});
audioPlayer.prepareAsync();
} catch (Exception e) {
    Log.e("ChatActivity", "playVoiceMessage failed: " + e.getMessage(), e);
    Toast.makeText(this, "Cannot play audio: " + e.getMessage(), Toast.LENGTH_SHORT) [...];
    if (onStop != null) onStop.run();
    stopAudioPlayer();
}
}

private boolean requestAudioFocus() {
    if (audioManager == null)
        audioManager = (android.media.AudioManager) getSystemService(AUDIO_SERVICE);
    audioFocusRequest = new android.media.AudioFocusRequest.Builder(
        android.media.AudioManager.AUDIOFOCUS_GAIN_TRANSIENT)
        .setAudioAttributes(new android.media.AudioAttributes.Builder()
            .setUsage(android.media.AudioAttributes.USAGE_MEDIA)
            .setContentType(android.media.AudioAttributes.CONTENT_TYPE_SPEECH)
            .build())
        .setOnAudioFocusChangeListener(focusChange -> {
            if (focusChange == android.media.AudioManager.AUDIOFOCUS_LOSS ||
                focusChange == android.media.AudioManager.AUDIOFOCUS_LOSS_TRANSI [...])
                runOnUiThread(this::stopAudioPlayer);
        })
        .build();
    return audioManager.requestAudioFocus(audioFocusRequest)
        == android.media.AudioManager.AUDIOFOCUS_REQUEST_GRANTED;
}

private void abandonAudioFocus() {
    if (audioManager != null && audioFocusRequest != null) {
        audioManager.abandonAudioFocusRequest(audioFocusRequest);
        audioFocusRequest = null;
    }
}

```

```

}
}

public void stopAudioPlayer() {
// ■ FIX #4: capture then null – prevents double-release on re-entrant calls
MediaPlayer p = audioPlayer;
audioPlayer = null;
playingUrl = null;
if (p != null) {
try { p.stop(); } catch (Exception ignored) {}
try { p.release(); } catch (Exception ignored) {}
}
abandonAudioFocus();
}

// ████████████████████████████████████████████████████████████████████████
// LIFECYCLE
// ████████████████████████████████████████████████████████████████████████

@Override
protected void onDestroy() {
super.onDestroy();
stopAudioPlayer();
if (messageListener != null) messageListener.remove();
if (isRecording) stopRecording();
recordingHandler.removeCallbacksAndMessages(null);
}
}

```

CallActivity

Voice and video call interface using Agora RTC SDK.

CallActivity.java (Java)

```
package com.example.joblinker.activities;
import android.content.Intent;
import android.media.AudioManager;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.view.View;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.example.joblinker.R;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.models.Call;
import com.example.joblinker.utils.ImageUtils;
/**
 * Call screen – UI and Firebase call record management.
 * Voice/video transmission requires a real-time SDK (Agora/WebRTC).
 * To enable calls: add your Agora App ID in strings.xml as "agora_app_id"
 * and integrate the Agora SDK dependency.
 */
public class CallActivity extends AppCompatActivity {
    public static final String EXTRA_RECEIVER_ID = "receiver_id";
    public static final String EXTRA_RECEIVER_NAME = "receiver_name";
    public static final String EXTRA_RECEIVER_AVATAR = "receiver_avatar";
    public static final String EXTRA_CALL_TYPE = "call_type";
    private ImageView ivUserAvatar;
    private TextView tvUserName, tvCallStatus, tvCallTimer;
    private View viewPulse, remoteVideoContainer, localVideoContainer;
    private FloatingActionButton btnMute, btnSpeaker, btnHangUp, btnVideoToggle;
    private JobLinkerFirebaseManager firebaseManager;
    private String receiverId, receiverName, receiverAvatar, callType, callId;
    private boolean isMuted = false;
    private boolean isSpeakerOn = false;
    private boolean isVideoOn = true;
    private long callStartTime = 0;
    private final Handler timerHandler = new Handler(Looper.getMainLooper());
    private Runnable timerRunnable;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_call);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        getIntentData();
        initializeViews();
        setupUI();
        setupClickListeners();
        startCall();
    }
    private void getIntentData() {
        receiverId = getIntent().getStringExtra(EXTRA_RECEIVER_ID);
        receiverName = getIntent().getStringExtra(EXTRA_RECEIVER_NAME);
        receiverAvatar = getIntent().getStringExtra(EXTRA_RECEIVER_AVATAR);
    }
}
```


[illegible]

[illegible]

```
}  
finish();  
}  
@Override  
protected void onDestroy() {  
    super.onDestroy();  
    timerHandler.removeCallbacksAndMessages(null);  
}  
@Override  
public void onBackPressed() {  
    endCall();  
}  
}
```

EditProfileActivity

Profile editing form with avatar upload to Firebase Storage.

EditProfileActivity.java (Java)

```
package com.example.joblinker.activities;
import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.os.Environment;
import android.provider.MediaStore;
import android.util.Log;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.AutoCompleteTextView;
import android.widget.ImageView;
import android.widget.ProgressBar;
import android.widget.RadioGroup;
import android.widget.TextView;
import android.widget.Toast;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.core.content.FileProvider;
import com.bumptech.glide.Glide;
import com.example.joblinker.R;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.card.MaterialCardView;
import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.google.android.material.radiobutton.MaterialRadioButton;
import com.google.android.material.textfield.TextInputEditText;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;
import java.io.File;
import java.io.IOException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.HashMap;
import java.util.Locale;
import java.util.Map;
public class EditProfileActivity extends AppCompatActivity {
    private static final String TAG = "EditProfileActivity";
    private static final int REQUEST_CAMERA_PERMISSION = 100;
    // Views
    private MaterialToolbar toolbar;
    private FloatingActionButton fabChangePhoto;
    private ImageView ivAvatar;
    private TextInputEditText etFullName, etEmail, etPhone, etBio, etCity;
    private AutoCompleteTextView actvCountry, actvLanguage, actvCurrency;
    private RadioGroup rgUserRole;
```

[illegible]

```

etPhone = findViewById(R.id.etPhone);
etBio = findViewById(R.id.etBio);
etCity = findViewById(R.id.etCity);
actvCountry = findViewById(R.id.actvCountry);
actvLanguage = findViewById(R.id.actvLanguage);
actvCurrency = findViewById(R.id.actvCurrency);
rgUserRole = findViewById(R.id.rgUserRole);
rbJobSeeker = findViewById(R.id.rbJobSeeker);
rbEmployer = findViewById(R.id.rbEmployer);
cardRoleInfo = findViewById(R.id.cardRoleInfo);
tvRoleInfo = findViewById(R.id.tvRoleInfo);
btnSaveChanges = findViewById(R.id.btnSaveChanges);
progressBar = findViewById(R.id.progressBar);
setSupportActionBar(toolbar);
}

private void setupListeners() {
toolbar.setNavigationOnClickListener(v -> finish());
fabChangePhoto.setOnClickListener(v -> showPhotoOptions());
rgUserRole.setOnCheckedChangeListener((group, checkedId) ->
updateRoleInfo(checkedId == R.id.rbJobSeeker));
btnSaveChanges.setOnClickListener(v -> saveChanges());
}

// PHOTO HANDLING

private void showPhotoOptions() {
String[] options = {"Take Photo", "Choose from Gallery", "Remove Photo"};
new AlertDialog.Builder(this)
.setTitle("Change Profile Picture")
.setItems(options, (dialog, which) -> {
switch (which) {
case 0: checkCameraPermissionAndOpen(); break;
case 1: openGallery(); break;
case 2: removePhoto(); break;
}
})
.show();
}

private void checkCameraPermissionAndOpen() {
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
== PackageManager.PERMISSION_GRANTED) {
openCamera();
} else {
ActivityCompat.requestPermissions(this,
new String[]{Manifest.permission.CAMERA}, REQUEST_CAMERA_PERMISSION);
}
}

@Override
public void onRequestPermissionsResult(int requestCode, String[] permissions, int[] grantResults) {
super.onRequestPermissionsResult(requestCode, permissions, grantResults);
if (requestCode == REQUEST_CAMERA_PERMISSION) {
if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
openCamera();
} else {
Toast.makeText(this, "Camera permission denied", Toast.LENGTH_SHORT).show();
}
}
}

private void openCamera() {
try {
File photoFile = createImageFile();
cameraImageUri = FileProvider.getUriForFile(this,
getPackageName() + ".fileprovider", photoFile);
Intent cameraIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);

```

```

cameraIntent.putExtra(MediaStore.EXTRA_OUTPUT, cameraImageUri);
cameraLauncher.launch(cameraIntent);
} catch (IOException e) {
Log.e(TAG, "Error creating image file", e);
Toast.makeText(this, "Could not open camera", Toast.LENGTH_SHORT).show();
}
}
private void openGallery() {
Intent galleryIntent;
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
galleryIntent = new Intent(MediaStore.ACTION_PICK_IMAGES);
} else {
galleryIntent = new Intent(Intent.ACTION_PICK,
MediaStore.Images.Media.EXTERNAL_CONTENT_URI);
}
galleryLauncher.launch(galleryIntent);
}
private void removePhoto() {
currentAvatarUrl = null;
if (ivAvatar != null) {
ivAvatar.setImageResource(R.drawable.ic_person_placeholder);
}
// Clear avatar in Firestore
if (userId != null) {
firestore.collection("users").document(userId)
.update("avatarUrl", null)
.addOnSuccessListener(v ->
Toast.makeText(this, "Photo removed", Toast.LENGTH_SHORT).show())
.addOnFailureListener(e ->
Toast.makeText(this, "Failed to remove photo", Toast.LENGTH_SHORT).show( [...])
)
}
}
private void showAvatarPreview(Uri uri) {
if (ivAvatar != null) {
Glide.with(this).load(uri).circleCrop().into(ivAvatar);
}
}
private byte[] readImageBytes(Uri uri) {
try {
java.io.InputStream is = getContentResolver().openInputStream(uri);
if (is == null) return null;
java.io.ByteArrayOutputStream buffer = new java.io.ByteArrayOutputStream();
byte[] chunk = new byte[8192]; int read;
while ((read = is.read(chunk)) != -1) buffer.write(chunk, 0, read);
is.close();
return buffer.toByteArray();
} catch (Exception e) {
Log.e(TAG, "readImageBytes failed", e);
return null;
}
}
private void uploadAvatar(Uri imageUri) {
if (userId == null) return;
isUploadingPhoto = true;
progressBar.setVisibility(View.VISIBLE);
btnSaveChanges.setEnabled(false);
btnSaveChanges.setText("Uploading photo...");
// Read bytes NOW on main thread while URI permissions are valid
byte[] imageBytes = readImageBytes(imageUri);
if (imageBytes == null || imageBytes.length == 0) {
isUploadingPhoto = false;
progressBar.setVisibility(View.GONE);
btnSaveChanges.setEnabled(true);
btnSaveChanges.setText("Save Changes");
}
}

```

```

Toast.makeText(this, "Could not read photo", Toast.LENGTH_SHORT).show();
return;
}
// Fixed path per user – overwrites previous avatar automatically
String path = "avatars/" + userId + ".jpg";
// Use context-aware overload so FileProvider URIs work correctly
firebaseManager.uploadBytes(imageBytes, path,
    new JobLinkerFirebaseManager.UploadCallback() {
        @Override public void onSuccess(String downloadUrl) {
            currentAvatarUrl = downloadUrl;
            isUploadingPhoto = false;
            progressBar.setVisibility(View.GONE);
            btnSaveChanges.setEnabled(true);
            btnSaveChanges.setText("Save Changes");
            // Save to Firestore immediately so it persists even without tapping Sav [...]
            firestore.collection("users").document(userId)
                .update("avatarUrl", downloadUrl);
            // Refresh the preview with no-cache
            if (ivAvatar != null) {
                com.bumptech.glide.Glide.with(EditProfileActivity.this)
                    .load(downloadUrl)
                    .circleCrop()
                    .skipMemoryCache(true)
                    .diskCacheStrategy(com.bumptech.glide.load.engine.DiskCacheStrat [...])
                    .into(ivAvatar);
            }
            Toast.makeText(EditProfileActivity.this,
                "■ Photo updated!", Toast.LENGTH_SHORT).show();
        }
        @Override public void onProgress(int progress) {
            btnSaveChanges.setText("Uploading " + progress + "%...");
        }
        @Override public void onFailure(String error) {
            isUploadingPhoto = false;
            progressBar.setVisibility(View.GONE);
            btnSaveChanges.setEnabled(true);
            btnSaveChanges.setText("Save Changes");
            Toast.makeText(EditProfileActivity.this,
                "Upload failed: " + error, Toast.LENGTH_LONG).show();
            Log.e(TAG, "Avatar upload failed: " + error);
        }
    });
}
/**
 * Copies any URI (content://, file://, FileProvider) to app's private cache.
 * Returns the File, or null if copy failed.
 */
private java.io.File copyUriToCache(Uri uri) {
    try {
        java.io.InputStream in = getContentResolver().openInputStream(uri);
        if (in == null) return null;
        java.io.File out = new java.io.File(getCacheDir(),
            "avatar_upload_" + System.currentTimeMillis() + ".jpg");
        java.io.FileOutputStream fos = new java.io.FileOutputStream(out);
        byte[] buf = new byte[8192];
        int len;
        while ((len = in.read(buf)) != -1) fos.write(buf, 0, len);
        fos.close();
        in.close();
        return out;
    } catch (Exception e) {
        Log.e(TAG, "copyUriToCache failed", e);
        return null;
    }
}

```



```

}
private File createImageFile() throws IOException {
    String timeStamp = new SimpleDateFormat("yyyyMMdd_HHmmss", Locale.US).format(new Date [...])
    String imageFileName = "JPEG_" + timeStamp + "_";
    File storageDir = getExternalFilesDir(Environment.DIRECTORY_PICTURES);
    return File.createTempFile(imageFileName, ".jpg", storageDir);
}
// =====
// LOAD / SAVE PROFILE
// =====
private void loadUserData() {
    progressBar.setVisibility(View.VISIBLE);
    FirebaseUser currentUser = firebaseAuth.getCurrentUser();
    if (currentUser == null) { loadFromSharedPreferences(); return; }
    etEmail.setText(currentUser.getEmail());
    firestore.collection("users").document(userId).get()
        .addOnSuccessListener(doc -> {
            if (doc.exists()) {
                setText(etFullName, doc.getString("userName"));
                setText(etPhone, doc.getString("userPhone"));
                setText(etBio, doc.getString("userBio"));
                setText(etCity, doc.getString("userCity"));
                setDropDown(actvCountry, doc.getString("userCountry"));
                setDropDown(actvLanguage, doc.getString("userLanguage"));
                setDropDown(actvCurrency, doc.getString("userCurrency"));
                currentRole = doc.getString("userRole");
                currentAvatarUrl = doc.getString("avatarUrl");
                if (currentAvatarUrl != null && ivAvatar != null) {
                    Glide.with(this)
                        .load(currentAvatarUrl)
                        .circleCrop()
                        .skipMemoryCache(true)
                        .diskCacheStrategy(com.bumptech.glide.load.engine.DiskCacheStrat [...])
                        .placeholder(R.drawable.ic_person_placeholder)
                        .into(ivAvatar);
                }
                boolean isEmployer = "employer".equalsIgnoreCase(currentRole);
                if (isEmployer) rbEmployer.setChecked(true);
                else rbJobSeeker.setChecked(true);
                updateRoleInfo(!isEmployer);
            } else {
                loadFromSharedPreferences();
            }
            progressBar.setVisibility(View.GONE);
        })
        .addOnFailureListener(e -> {
            loadFromSharedPreferences();
            progressBar.setVisibility(View.GONE);
        });
}
private void saveChanges() {
    // Block save if photo is still uploading
    if (isUploadingPhoto) {
        Toast.makeText(this, "Please wait – photo is still uploading...",
            Toast.LENGTH_SHORT).show();
        return;
    }
    String name = etFullName.getText().toString().trim();
    String phone = etPhone.getText().toString().trim();
    String bio = etBio.getText().toString().trim();
    String city = etCity.getText().toString().trim();
    String country = actvCountry.getText().toString().trim();
    String language = actvLanguage.getText().toString().trim();
    String currency = actvCurrency.getText().toString().trim();
}

```

[illegible]

[illegible]

```
private void setupDropdowns() {  
    actvCountry.setAdapter(new ArrayAdapter<>(this,  
        android.R.layout.simple_dropdown_item_1line, countries));  
    actvLanguage.setAdapter(new ArrayAdapter<>(this,  
        android.R.layout.simple_dropdown_item_1line, languages));  
    actvCurrency.setAdapter(new ArrayAdapter<>(this,  
        android.R.layout.simple_dropdown_item_1line, currencies));  
}  
}
```

FilterActivity

Advanced job filter screen. Returns FilterState to JobsFragment via ActivityResultLauncher.

FilterActivity.java (Java)

```
package com.example.joblinker.activities;
import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.widget.AdapterView;
import android.widget.AutoCompleteTextView;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AppCompatActivity;
import com.example.joblinker.R;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.textfield.TextInputEditText;
public class FilterActivity extends AppCompatActivity {
    public static final String EXTRA_FILTER_JOB_TYPE = "filter_job_type";
    public static final String EXTRA_FILTER_CATEGORY = "filter_category";
    public static final String EXTRA_FILTER_LOCATION = "filter_location";
    public static final String EXTRA_FILTER_SALARY_MIN = "filter_salary_min";
    public static final String EXTRA_FILTER_SALARY_MAX = "filter_salary_max";
    public static final String EXTRA_FILTER_SORT = "filter_sort";
    // sort values
    public static final String SORT_NEWEST = "newest";
    public static final String SORT_OLDEST = "oldest";
    private MaterialToolbar toolbar;
    private AutoCompleteTextView dropdownJobType;
    private AutoCompleteTextView dropdownCategory;
    private TextInputEditText etLocation;
    private TextInputEditText etSalaryMin;
    private TextInputEditText etSalaryMax;
    private AutoCompleteTextView dropdownSort;
    private MaterialButton btnApply;
    private MaterialButton btnClear;
    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_filter);
        initializeViews();
        setupToolbar();
        setupDropdowns();
        fillFromIntent();
        setupButtons();
    }
    private void initializeViews() {
        toolbar = findViewById(R.id.toolbar);
        dropdownJobType = findViewById(R.id.dropdown_job_type);
        dropdownCategory = findViewById(R.id.dropdown_category);
        etLocation = findViewById(R.id.et_location);
        etSalaryMin = findViewById(R.id.et_salary_min);
        etSalaryMax = findViewById(R.id.et_salary_max);
        dropdownSort = findViewById(R.id.dropdown_sort);
        btnApply = findViewById(R.id.btn_apply_filters);
        btnClear = findViewById(R.id.btn_clear_filters);
    }
    private void setupToolbar() {
        setSupportActionBar(toolbar);
```

```

toolbar.setNavigationOnClickListener(v -> finish());
}
private void setupDropdowns() {
// You can replace these with values from resources if you want
String[] jobTypes = new String[]{"All", "Full-time", "Part-time", "Remote", "Contract"};
String[] categories = new String[]{"All", "Software", "Design", "Marketing", "Sales"};
String[] sorts = new String[]{"Newest", "Oldest"};
dropdownJobType.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, jobTypes));
dropdownCategory.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, categories));
dropdownSort.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, sorts));
// defaults
if (TextUtils.isEmpty(dropdownJobType.getText())) dropdownJobType.setText("All", false);
if (TextUtils.isEmpty(dropdownCategory.getText())) dropdownCategory.setText("All", false);
if (TextUtils.isEmpty(dropdownSort.getText())) dropdownSort.setText("Newest", false);
}
private void fillFromIntent() {
Intent intent = getIntent();
if (intent == null) return;
setIfPresent(dropdownJobType, intent.getStringExtra(EXTRA_FILTER_JOB_TYPE), "All");
setIfPresent(dropdownCategory, intent.getStringExtra(EXTRA_FILTER_CATEGORY), "All");
String location = intent.getStringExtra(EXTRA_FILTER_LOCATION);
if (location != null) etLocation.setText(location);
long min = intent.getLongExtra(EXTRA_FILTER_SALARY_MIN, -1);
long max = intent.getLongExtra(EXTRA_FILTER_SALARY_MAX, -1);
if (min >= 0) etSalaryMin.setText(String.valueOf(min));
if (max >= 0) etSalaryMax.setText(String.valueOf(max));
String sort = intent.getStringExtra(EXTRA_FILTER_SORT);
if (SORT_OLDEST.equals(sort)) dropdownSort.setText("Oldest", false);
else dropdownSort.setText("Newest", false);
}
private void setIfPresent(AutoCompleteTextView view, String value, String fallback) {
if (!TextUtils.isEmpty(value)) view.setText(value, false);
else view.setText(fallback, false);
}
private void setupButtons() {
btnApply.setOnClickListener(v -> {
Intent data = new Intent();
data.putExtra(EXTRA_FILTER_JOB_TYPE, dropdownJobType.getText().toString());
data.putExtra(EXTRA_FILTER_CATEGORY, dropdownCategory.getText().toString());
data.putExtra(EXTRA_FILTER_LOCATION, safeText(etLocation));
data.putExtra(EXTRA_FILTER_SALARY_MIN, parseLongOrMinusOne(safeText(etSalaryMin)));
data.putExtra(EXTRA_FILTER_SALARY_MAX, parseLongOrMinusOne(safeText(etSalaryMax)));
String sortUi = dropdownSort.getText().toString();
data.putExtra(EXTRA_FILTER_SORT, "Oldest".equalsIgnoreCase(sortUi) ? SORT_OLDEST : SORT_NEWEST);
setResult(RESULT_OK, data);
finish();
});
btnClear.setOnClickListener(v -> {
Intent data = new Intent();
data.putExtra(EXTRA_FILTER_JOB_TYPE, "All");
data.putExtra(EXTRA_FILTER_CATEGORY, "All");
data.putExtra(EXTRA_FILTER_LOCATION, "");
data.putExtra(EXTRA_FILTER_SALARY_MIN, -1L);
data.putExtra(EXTRA_FILTER_SALARY_MAX, -1L);
data.putExtra(EXTRA_FILTER_SORT, SORT_NEWEST);
setResult(RESULT_OK, data);
finish();
});
}
private String safeText(TextInputEditText editText) {
return editText.getText() == null ? "" : editText.getText().toString().trim();
}
private long parseLongOrMinusOne(String s) {
if (TextUtils.isEmpty(s)) return -1;
}

```

```
try {  
    return Long.parseLong(s);  
} catch (Exception e) {  
    return -1;  
}  
}  
}
```


[illegible]

[illegible]

[illegible]

```
if (btnPrivacyPolicy != null) {
    btnPrivacyPolicy.setOnClickListener(v -> showTextDialog("Privacy Policy",
        "Your privacy matters to us.\n\n" +
        "\u2022 We collect only the information needed to operate the app.\n" +
        "\u2022 Your data is stored securely in Google Firebase.\n" +
        "\u2022 We do not sell your personal information to third parties.\n" +
        "\u2022 You can delete your account and all data at any time.\n" +
        "\u2022 We use anonymous analytics to improve the app experience.\n\n" +
        "For the full privacy policy visit our website."));
}

if (btnAbout != null) {
    btnAbout.setOnClickListener(v -> showTextDialog("About JobLinker",
        "JobLinker v1.0\n\n" +
        "JobLinker connects job seekers with employers through a modern, " +
        "easy-to-use mobile platform.\n\n" +
        "Features:\n" +
        "\u2022 Post and browse job listings\n" +
        "\u2022 Real-time chat between seekers and employers\n" +
        "\u2022 Voice & video calls powered by Agora\n" +
        "\u2022 Smart job matching and filters\n" +
        "\u2022 Push notifications for new jobs and messages\n\n" +
        "Built with \ud83d\ude07 using Firebase & Android."));
}

// DANGER ZONE
if (btnDeleteAccount != null) {
    btnDeleteAccount.setOnClickListener(v -> showDeleteAccountDialog());
}

if (btnLogout != null) {
    btnLogout.setOnClickListener(v -> LogoutHelper.showLogoutDialog(this));
}

// 
// DIALOG IMPLEMENTATIONS
// 
// Change Password 
private void showChangePasswordDialog() {
    View dialogView = LayoutInflater.from(this)
        .inflate(R.layout.dialog_change_password, null);
    EditText etCurrentPassword = dialogView.findViewById(R.id.et_current_password);
    EditText etNewPassword = dialogView.findViewById(R.id.et_new_password);
    EditText etConfirmPassword = dialogView.findViewById(R.id.et_confirm_password);
    // Fallback: if layout doesn't exist yet, use a simple 3-field approach
    if (etCurrentPassword == null) {
        // Build a simple dialog manually
        showSimpleChangePasswordDialog();
        return;
    }

    new AlertDialog.Builder(this)
        .setTitle("Change Password")
        .setView(dialogView)
        .setPositiveButton("Change", (dialog, which) -> {
            String current = etCurrentPassword.getText().toString().trim();
            String newPass = etNewPassword.getText().toString().trim();
            String confirm = etConfirmPassword.getText().toString().trim();
            performChangePassword(current, newPass, confirm);
        })
        .setNegativeButton("Cancel", null)
        .show();
}

private void showSimpleChangePasswordDialog() {
    // Current password field
    final EditText etCurrent = new EditText(this);
    etCurrent.setHint("Current password");
    etCurrent.setInputType(
```

```

        android.text.InputType.TYPE_CLASS_TEXT |
        android.text.InputType.TYPE_TEXT_VARIATION_PASSWORD));
        final EditText etNew = new EditText(this);
        etNew.setHint("New password (min 6 chars)");
        etNew.setInputType(
        android.text.InputType.TYPE_CLASS_TEXT |
        android.text.InputType.TYPE_TEXT_VARIATION_PASSWORD));
        final EditText etConfirm = new EditText(this);
        etConfirm.setHint("Confirm new password");
        etConfirm.setInputType(
        android.text.InputType.TYPE_CLASS_TEXT |
        android.text.InputType.TYPE_TEXT_VARIATION_PASSWORD));
        LinearLayout layout = new LinearLayout(this);
        layout.setOrientation(LinearLayout.VERTICAL);
        int pad = (int)(16 * getResources().getDisplayMetrics().density);
        layout.setPadding(pad, pad, pad, 0);
        layout.addView(etCurrent);
        layout.addView(etNew);
        layout.addView(etConfirm);
        new AlertDialog.Builder(this)
            .setTitle("Change Password")
            .setView(layout)
            .setPositiveButton("Change", (dialog, which) ->
                performChangePassword(
                    etCurrent.getText().toString().trim(),
                    etNew.getText().toString().trim(),
                    etConfirm.getText().toString().trim()))
            .setNegativeButton("Cancel", null)
            .show();
    }

    private void performChangePassword(String current, String newPass, String confirm) {
        if (TextUtils.isEmpty(current)) {
            Toast.makeText(this, "Enter your current password", Toast.LENGTH_SHORT).show();
            return;
        }
        if (newPass.length() < 6) {
            Toast.makeText(this, "New password must be at least 6 characters", Toast.LENGTH_ [...]
            return;
        }
        if (!newPass.equals(confirm)) {
            Toast.makeText(this, "Passwords do not match", Toast.LENGTH_SHORT).show();
            return;
        }
        FirebaseUser user = firebaseAuth.getCurrentUser();
        if (user == null || user.getEmail() == null) return;
        ProgressDialogHelper pd = new ProgressDialogHelper(this, "Changing password...");
        pd.show();
        // Re-authenticate first
        AuthCredential credential = EmailAuthProvider.getCredential(user.getEmail(), current [...])
        user.reauthenticate(credential).addOnCompleteListener(reAuthTask -> {
            if (!reAuthTask.isSuccessful()) {
                pd.dismiss();
                Toast.makeText(this, "Current password is incorrect", Toast.LENGTH_SHORT).sh [...]
                return;
            }
            // Now change the password
            firebaseManager.changePassword(newPass, new JobLinkerFirebaseManager.VoidCallbac [...])
            @Override
            public void onSuccess() {
                pd.dismiss();
                Toast.makeText(SettingActivity.this,
                    "Password changed successfully!", Toast.LENGTH_SHORT).show();
            }
        }
        @Override

```

[illegible]

[illegible]

```

Toast.makeText(this, "Password required", Toast.LENGTH_SHORT).show() [...]
return;
}
performDeleteAccount(password);
})
.setNegativeButton("Cancel", null)
.show();
}
private void performDeleteAccount(String password) {
    FirebaseUser user = firebaseAuth.getCurrentUser();
    if (user == null || user.getEmail() == null) return;
    ProgressDialogHelper pd = new ProgressDialogHelper(this, "Deleting account...");
    pd.show();
    AuthCredential credential = EmailAuthProvider.getCredential(user.getEmail(), passwor [...]
    user.reauthenticate(credential).addOnCompleteListener(reAuthTask -> {
        if (!reAuthTask.isSuccessful()) {
            pd.dismiss();
            Toast.makeText(this, "Incorrect password", Toast.LENGTH_SHORT).show();
            return;
        }
        firebaseManager.deleteAccount(new JobLinkerFirebaseManager.VoidCallback() {
            @Override
            public void onSuccess() {
                pd.dismiss();
                prefsManager.clearAll();
                Toast.makeText(SettingActivity.this,
                    "Account deleted.", Toast.LENGTH_SHORT).show();
                Intent intent = new Intent(SettingActivity.this, LoginActivity.class);
                intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLE [...]
                startActivity(intent);
            }
            @Override
            public void onFailure(String error) {
                pd.dismiss();
                Toast.makeText(SettingActivity.this,
                    "Failed to delete account: " + error, Toast.LENGTH_LONG).show();
            }
        });
    });
}
}
}
}
}

```


SavedJobsActivity

Lists all jobs bookmarked by the user. Fetches from Firestore 'savedJobs' collection and renders with JobAdapter. Shows empty state when list is empty.

SavedJobsActivity.java (Java)

```
package com.example.joblinker.activities;
import android.os.Bundle;
import android.view.View;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import com.google.android.material.appbar.MaterialToolbar;
import com.example.joblinker.R;
import com.example.joblinker.adapters.JobAdapter;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.models.Job;
import java.util.ArrayList;
import java.util.List;
public class SavedJobsActivity extends AppCompatActivity {
    private MaterialToolbar toolbar;
    private RecyclerView recyclerView;
    private ProgressBar progressBar;
    private TextView tvEmpty;
    private JobLinkerFirebaseManager firebaseManager;
    private JobAdapter jobAdapter;
    private final List<Job> savedJobs = new ArrayList<>();
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_saved_jobs);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        toolbar = findViewById(R.id.toolbar);
        recyclerView = findViewById(R.id.recycler_saved_jobs);
        progressBar = findViewById(R.id.progress_bar);
        tvEmpty = findViewById(R.id.tv_empty);
        setSupportActionBar(toolbar);
        toolbar.setNavigationOnClickListener(v -> onBackPressed());
        jobAdapter = new JobAdapter(this, savedJobs);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        recyclerView.setAdapter(jobAdapter);
        loadSavedJobs();
    }
    private void loadSavedJobs() {
        String userId = firebaseManager.getCurrentUserId();
        if (userId == null) return;
        progressBar.setVisibility(View.VISIBLE);
        tvEmpty.setVisibility(View.GONE);
        firebaseManager.getSavedJobs(userId,
            new JobLinkerFirebaseManager.ListCallback<String>() {
                @Override
                public void onSuccess(List<String> jobIds) {
                    if (jobIds.isEmpty()) {
                        progressBar.setVisibility(View.GONE);
                        tvEmpty.setVisibility(View.VISIBLE);
                    }
                    return;
                }
            }
        );
    }
}
```

```
}
// Fetch each job by ID
savedJobs.clear();
final int[] count = {0};
for (String jobId : jobIds) {
    firebaseManager.getJob(jobId,
        new JobLinkerFirebaseManager.DataCallback<Job>() {
            @Override
            public void onSuccess(Job job) {
                savedJobs.add(job);
                count[0]++;
                if (count[0] == jobIds.size()) {
                    progressBar.setVisibility(View.GONE);
                    jobAdapter.notifyDataSetChanged();
                    if (savedJobs.isEmpty())
                        tvEmpty.setVisibility(View.VISIBLE);
                }
            }
            @Override
            public void onFailure(String error) {
                count[0]++;
                if (count[0] == jobIds.size()) {
                    progressBar.setVisibility(View.GONE);
                    jobAdapter.notifyDataSetChanged();
                }
            }
        });
}
@Override
public void onFailure(String error) {
    progressBar.setVisibility(View.GONE);
    Toast.makeText(SavedJobsActivity.this,
        "Error loading saved jobs: " + error, Toast.LENGTH_SHORT).show();
}
});
}
```

EmailVerificationActivity

Email verification flow post-registration.

EmailVerificationActivity.java (Java)

```
package com.example.joblinker.activities.Email;
import android.os.Bundle;
import android.os.CountDownTimer;
import android.view.View;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.textfield.TextInputEditText;
import com.example.joblinker.R;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
public class EmailVerificationActivity extends AppCompatActivity {
    private MaterialToolbar toolbar;
    private TextView tvVerificationMessage;
    private TextInputEditText etVerificationCode;
    private TextView tvResendCode;
    private TextView tvResendTimer;
    private MaterialButton btnVerify;
    private MaterialButton btnSkipVerification;
    private JobLinkerFirebaseManager firebaseManager;
    private CountDownTimer resendTimer;
    private boolean canResend = false;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_email_verification);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        initializeViews();
        setupToolbar();
        setupClickListeners();
        startResendTimer();
    }
    private void initializeViews() {
        toolbar = findViewById(R.id.toolbar);
        tvVerificationMessage = findViewById(R.id.tv_verification_message);
        etVerificationCode = findViewById(R.id.et_verification_code);
        tvResendCode = findViewById(R.id.tv_resend_code);
        tvResendTimer = findViewById(R.id.tv_resend_timer);
        btnVerify = findViewById(R.id.btn_verify);
        btnSkipVerification = findViewById(R.id.btn_skip_verification);
    }
    private void setupToolbar() {
        setSupportActionBar(toolbar);
        toolbar.setNavigationOnClickListener(v -> onBackPressed());
    }
    private void setupClickListeners() {
        btnVerify.setOnClickListener(v -> verifyCode());
        tvResendCode.setOnClickListener(v -> {
            if (canResend) {
                resendCode();
            }
        });
        btnSkipVerification.setOnClickListener(v -> finish());
    }
}
```

```

}
private void verifyCode() {
String code = etVerificationCode.getText().toString().trim();
if (code.isEmpty() || code.length() != 6) {
etVerificationCode.setError("Enter 6-digit code");
return;
}
// TODO: Implement verification logic
Toast.makeText(this, "Verification successful!", Toast.LENGTH_SHORT).show();
finish();
}
private void resendCode() {
firebaseManager.sendEmailVerification(new JobLinkerFirebaseManager.VoidCallback() {
@Override
public void onSuccess() {
Toast.makeText(EmailVerificationActivity.this,
"Verification code sent", Toast.LENGTH_SHORT).show();
startResendTimer();
}
@Override
public void onFailure(String error) {
Toast.makeText(EmailVerificationActivity.this,
"Error: " + error, Toast.LENGTH_SHORT).show();
}
});
}
private void startResendTimer() {
canResend = false;
tvResendCode.setEnabled(false);
tvResendCode.setAlpha(0.5f);
tvResendTimer.setVisibility(View.VISIBLE);
resendTimer = new CountDownTimer(60000, 1000) {
@Override
public void onTick(long millisUntilFinished) {
int seconds = (int) (millisUntilFinished / 1000);
tvResendTimer.setText(String.format("Resend code in %ds", seconds));
}
@Override
public void onFinish() {
canResend = true;
tvResendCode.setEnabled(true);
tvResendCode.setAlpha(1.0f);
tvResendTimer.setVisibility(View.GONE);
}
};
resendTimer.start();
}
@Override
protected void onDestroy() {
super.onDestroy();
if (resendTimer != null) {
resendTimer.cancel();
}
}
}
}

```


Source Code

Fragments – UI Panels

JobsFragment

Main job listing fragment with search, quick-filter chips, and real-time Firestore listener.

JobsFragment.java (Java)

```
package com.example.joblinker.fragments;
import android.content.Intent;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.text.Editable;
import android.text.TextUtils;
import android.text.TextWatcher;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.LinearLayout;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import com.example.joblinker.R;
import com.example.joblinker.activities.FilterActivity;
import com.example.joblinker.activities.JobDetailsActivity;
import com.example.joblinker.activities.PostJobActivity;
import com.example.joblinker.adapters.JobAdapter;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.models.Job;
import com.example.joblinker.utils.SharedPreferencesManager;
import com.google.android.material.chip.ChipGroup;
import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.google.firebase.firestore.ListenerRegistration;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class JobsFragment extends Fragment {
    // Search debounce: wait 300ms after last keystroke before filtering
    private static final long SEARCH_DEBOUNCE_MS = 300;
    private EditText etSearch;
    private ImageButton btnFilter;
    private ChipGroup chipGroup;
    private RecyclerView recyclerJobs;
    private LinearLayout layoutEmpty;
    private ProgressBar progressBar;
    private FloatingActionButton fabPostJob;
    private TextView tvResultCount; // NEW: shows "X jobs found"
    private JobAdapter jobAdapter;
    private final List<Job> allJobs = new ArrayList<>();
    private final List<Job> filteredJobs = new ArrayList<>();
```

```

private JobLinkerFirebaseManager firebaseManager;
private SharedPreferencesManager prefsManager;
private ListenerRegistration jobsListener;
// Debounce handler
private final Handler searchHandler = new Handler(Looper.getMainLooper());
private Runnable searchRunnable;
// Quick chip filter
private String currentFilter = "All";
// Advanced filter state
private String filterJobType = "All";
private String filterCategory = "All";
private String filterLocation = "";
private long filterSalaryMin = -1;
private long filterSalaryMax = -1;
private String filterSort = FilterActivity.SORT_NEWEST;
private ActivityResultLauncher<Intent> filterLauncher;
@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container [...],
@Nullable Bundle savedInstanceState) {
    View view = inflater.inflate(R.layout.fragment_jobs, container, false);
    firebaseManager = JobLinkerFirebaseManager.getInstance();
    prefsManager = SharedPreferencesManager.getInstance(requireContext());
    setupFilterLauncher();
    initializeViews(view);
    setupRecyclerView();
    setupSearchListener();
    setupChipListener();
    setupClickListeners();
    loadJobs();
    return view;
}

private void setupFilterLauncher() {
    filterLauncher = registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        result -> {
            if (result.getResultCode() != android.app.Activity.RESULT_OK
            || result.getData() == null) return;
            Intent data = result.getData();
            filterJobType = nvl(data.getStringExtra(FilterActivity.EXTRA_FILTER_JOB_ [...])
            filterCategory = nvl(data.getStringExtra(FilterActivity.EXTRA_FILTER_CAT [...])
            filterLocation = nvl(data.getStringExtra(FilterActivity.EXTRA_FILTER_LOC [...])
            filterSalaryMin = data.getLongExtra(FilterActivity.EXTRA_FILTER_SALARY_M [...])
            filterSalaryMax = data.getLongExtra(FilterActivity.EXTRA_FILTER_SALARY_M [...])
            filterSort = nvl(data.getStringExtra(FilterActivity.EXTRA_FILTER_SORT),
            FilterActivity.SORT_NEWEST);
            applyAllFilters();
        });
}

private void initializeViews(View view) {
    etSearch = view.findViewById(R.id.et_search);
    btnFilter = view.findViewById(R.id.btn_filter);
    chipGroup = view.findViewById(R.id.chip_group);
    recyclerJobs = view.findViewById(R.id.recycler_jobs);
    layoutEmpty = view.findViewById(R.id.layout_empty);
    progressBar = view.findViewById(R.id.progress_bar);
    fabPostJob = view.findViewById(R.id.fab_post_job);
    tvResultCount = view.findViewById(R.id.tv_result_count);
    fabPostJob.setVisibility(prefsManager.isEmployer() ? View.VISIBLE : View.GONE);
}

private void setupRecyclerView() {
    jobAdapter = new JobAdapter(requireContext(), filteredJobs);
    recyclerJobs.setLayoutManager(new LinearLayoutManager(requireContext()));
    recyclerJobs.setAdapter(jobAdapter);
}

```



```

jobAdapter.setOnJobClickListener(new JobAdapter.OnJobClickListener() {
    @Override
    public void onJobClick(Job job) {
        Intent intent = new Intent(requireContext(), JobDetailsActivity.class);
        intent.putExtra(JobDetailsActivity.EXTRA_JOB_ID, job.getId());
        startActivity(intent);
    }
    @Override
    public void onSaveClick(Job job, boolean isSaved) {
        // Persist save/unsave to Firebase
        String userId = firebaseManager.getCurrentUserId();
        if (userId == null) {
            Toast.makeText(requireContext(),
                "Please login to save jobs", Toast.LENGTH_SHORT).show();
            jobAdapter.markJobSaved(job.getId(), !isSaved); // revert optimistic
            return;
        }
        JobLinkerFirebaseManager.VoidCallback cb = new JobLinkerFirebaseManager.Void [...]
        @Override public void onSuccess() { /* UI already updated optimistically [...]
        @Override public void onFailure(String error) {
            // Revert optimistic update on failure
            jobAdapter.markJobSaved(job.getId(), !isSaved);
            Toast.makeText(requireContext(), "Error: " + error, Toast.LENGTH_SHO [...]
        }
    };
    if (isSaved) {
        firebaseManager.saveJob(userId, job.getId(), cb);
    } else {
        firebaseManager.unsaveJob(userId, job.getId(), cb);
    }
}
});
// Load saved job IDs once so bookmark icons show correctly from the start
loadSavedJobIds();
}
private void loadSavedJobIds() {
    String userId = firebaseManager.getCurrentUserId();
    if (userId == null) return;
    firebaseManager.getSavedJobIds(userId, new JobLinkerFirebaseManager.ListCallback<Str [...]
    @Override
    public void onSuccess(List<String> ids) {
        if (ids == null || !isAdded()) return;
        Set<String> idSet = new HashSet<>(ids);
        jobAdapter.setSavedJobIds(idSet);
    }
    @Override
    public void onFailure(String error) { /* non-critical, ignore */ }
});
}
/**
 * Debounced search: waits 300 ms after the last keystroke before filtering.
 * This avoids running the filter loop on every character typed.
 */
private void setupSearchListener() {
    etSearch.addTextChangedListener(new TextWatcher() {
        @Override public void beforeTextChanged(CharSequence s, int st, int c, int a) {}
        @Override public void afterTextChanged(Editable s) {}
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int count) {
            if (searchRunnable != null) searchHandler.removeCallbacks(searchRunnable);
            searchRunnable = () -> applyAllFilters();
            searchHandler.postDelayed(searchRunnable, SEARCH_DEBOUNCE_MS);
        }
    });
}
});

```

```

}
private void setupChipListener() {
    chipGroup.setOnCheckedChangeListener((group, checkedIds) -> {
        if (checkedIds == null || checkedIds.isEmpty()) return;
        int id = checkedIds.get(0);
        if (id == R.id.chip_all) currentFilter = "All";
        else if (id == R.id.chip_full_time) currentFilter = "Full-time";
        else if (id == R.id.chip_part_time) currentFilter = "Part-time";
        else if (id == R.id.chip_remote) currentFilter = "Remote";
        else if (id == R.id.chip_contract) currentFilter = "Contract";
        else if (id == R.id.chip_internship) currentFilter = "Internship";
        else currentFilter = "All";
        applyAllFilters();
    });
}

private void setupClickListeners() {
    btnFilter.setOnClickListener(v -> {
        Intent intent = new Intent(requireContext(), FilterActivity.class);
        intent.putExtra(FilterActivity.EXTRA_FILTER_JOB_TYPE, filterJobType);
        intent.putExtra(FilterActivity.EXTRA_FILTER_CATEGORY, filterCategory);
        intent.putExtra(FilterActivity.EXTRA_FILTER_LOCATION, filterLocation);
        intent.putExtra(FilterActivity.EXTRA_FILTER_SALARY_MIN, filterSalaryMin);
        intent.putExtra(FilterActivity.EXTRA_FILTER_SALARY_MAX, filterSalaryMax);
        intent.putExtra(FilterActivity.EXTRA_FILTER_SORT, filterSort);
        filterLauncher.launch(intent);
    });
    fabPostJob.setOnClickListener(v ->
        startActivity(new Intent(requireContext(), PostJobActivity.class)));
}

private void loadJobs() {
    progressBar.setVisibility(View.VISIBLE);
    layoutEmpty.setVisibility(View.GONE);
    recyclerJobs.setVisibility(View.GONE);
    updateResultCount(0, true);
    jobsListener = firebaseManager.listenToActiveJobs(
        new JobLinkerFirebaseManager.ListCallback<Job>() {
            @Override
            public void onSuccess(List<Job> jobs) {
                progressBar.setVisibility(View.GONE);
                allJobs.clear();
                if (jobs != null) allJobs.addAll(jobs);
                applyAllFilters();
            }
            @Override
            public void onFailure(String error) {
                progressBar.setVisibility(View.GONE);
                layoutEmpty.setVisibility(View.VISIBLE);
                recyclerJobs.setVisibility(View.GONE);
                updateResultCount(0, false);
                Toast.makeText(requireContext(),
                    "Error loading jobs: " + error, Toast.LENGTH_SHORT).show();
            }
        });
}

private void applyAllFilters() {
    String query = etSearch.getText() == null ? "" : etSearch.getText().toString().trim();
    List<Job> result = new ArrayList<>();
    for (Job job : allJobs) {
        if (job == null) continue;
        // 1) Text search (title, company, description)
        if (!TextUtils.isEmpty(query)) {
            boolean matchesSearch = containsCI(job.getJobTitle(), query)
                || containsCI(job.getJobCompany(), query)
                || containsCI(job.getJobDescription(), query);

```

```

if (!matchesSearch) continue;
}
// 2) Quick chip (job type)
if (!"All".equals(currentFilter)
&& !currentFilter.equals(job.getJobType())) continue;
// 3) Advanced: job type
if (!"All".equals(filterJobType)
&& !filterJobType.equals(job.getJobType())) continue;
// 4) Advanced: category
if (!"All".equals(filterCategory)
&& !filterCategory.equals(job.getJobCategory())) continue;
// 5) Advanced: location substring
if (!TextUtils.isEmpty(filterLocation)
&& !containsCI(job.getLocation(), filterLocation)) continue;
// 6) Advanced: salary range (numeric fields are available on Job model)
if (filterSalaryMin > 0 && job.getJobSalaryMax() > 0
&& job.getJobSalaryMax() < filterSalaryMin) continue;
if (filterSalaryMax > 0 && job.getJobSalaryMin() > filterSalaryMax) continue;
result.add(job);
}
// 7) Sort
if (FilterActivity.SORT_OLDEST.equals(filterSort)) {
Collections.sort(result, (a, b) -> Long.compare(safeCreatedAt(a), safeCreatedAt( [...])
} else {
Collections.sort(result, (a, b) -> Long.compare(safeCreatedAt(b), safeCreatedAt( [...])
}
// Use DiffUtil-based submitList instead of notifyDataSetChanged
jobAdapter.submitList(result);
filteredJobs.clear();
filteredJobs.addAll(result);
updateResultCount(result.size(), false);
if (result.isEmpty()) {
layoutEmpty.setVisibility(View.VISIBLE);
recyclerJobs.setVisibility(View.GONE);
} else {
layoutEmpty.setVisibility(View.GONE);
recyclerJobs.setVisibility(View.VISIBLE);
}
}
}
/** Show or hide the result counter label */
private void updateResultCount(int count, boolean loading) {
if (tvResultCount == null) return;
if (loading) {
tvResultCount.setVisibility(View.GONE);
return;
}
tvResultCount.setVisibility(View.VISIBLE);
if (count == 0) {
tvResultCount.setText("");
tvResultCount.setVisibility(View.GONE);
} else {
tvResultCount.setText(count + (count == 1 ? " job found" : " jobs found"));
}
}
private boolean containsCI(String src, String q) {
if (src == null || q == null) return false;
return src.toLowerCase().contains(q.toLowerCase());
}
private long safeCreatedAt(Job job) {
try { return job.getCreatedAt(); } catch (Exception e) { return 0; }
}
private static String nvl(String s, String def) { return s != null ? s : def; }
@Override
public void onDestroyView() {

```

```
super.onDestroyView();  
// Cancel any pending debounce callback  
if (searchRunnable != null) searchHandler.removeCallbacks(searchRunnable);  
if (jobsListener != null) { jobsListener.remove(); jobsListener = null; }  
}  
}
```

ChatsFragment

Conversation list fragment with search and unread badge logic.

ChatsFragment.java (Java)

```
package com.example.joblinker.fragments;
import android.content.Intent;
import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.EditText;
import android.widget.LinearLayout;
import android.widget.ProgressBar;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import com.google.firebase.firestore.ListenerRegistration;
import com.example.joblinker.R;
import com.example.joblinker.activities.ChatActivity;
import com.example.joblinker.adapters.ConversationAdapter;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.models.Conversation;
import com.example.joblinker.models.User;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;
public class ChatsFragment extends Fragment
implements ConversationAdapter.OnConversationClickListener {
    private RecyclerView recyclerConversations;
    private LinearLayout layoutEmpty;
    private ProgressBar progressBar;
    private EditText etSearch;
    private ConversationAdapter conversationAdapter;
    private final List<Conversation> allConversations = new ArrayList<>();
    private final List<Conversation> filtered = new ArrayList<>();
    private JobLinkerFirebaseManager firebaseManager;
    private ListenerRegistration conversationsListener;
    @Nullable @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
    @Nullable ViewGroup container,
    @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_chats, container, false);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        recyclerConversations = view.findViewById(R.id.recycler_conversations);
        layoutEmpty = view.findViewById(R.id.layout_empty);
        progressBar = view.findViewById(R.id.progress_bar);
        etSearch = view.findViewById(R.id.et_search_chats);
        setupRecyclerView();
        setupSearch();
        loadConversations();
        return view;
    }
    private void setupRecyclerView() {
```

```

conversationAdapter = new ConversationAdapter(requireContext(), filtered, this);
recyclerConversations.setLayoutManager(new LinearLayoutManager(requireContext()));
recyclerConversations.setAdapter(conversationAdapter);
}

private void setupSearch() {
    if (etSearch == null) return;
    etSearch.addTextChangedListener(new TextWatcher() {
        @Override public void beforeTextChanged(CharSequence s, int i, int i1, int i2) { [...]}
        @Override public void afterTextChanged(Editable s) {}
        @Override
        public void onTextChanged(CharSequence s, int start, int b, int count) {
            filterConversations(s.toString().trim());
        }
    });
}

private void filterConversations(String query) {
    filtered.clear();
    if (query.isEmpty()) {
        filtered.addAll(allConversations);
    } else {
        String lower = query.toLowerCase();
        for (Conversation c : allConversations) {
            String name = c.getOtherUserName();
            String last = c.getLastMessage();
            if ((name != null && name.toLowerCase().contains(lower))
                || (last != null && last.toLowerCase().contains(lower))) {
                filtered.add(c);
            }
        }
    }
    conversationAdapter.notifyDataSetChanged();
    updateEmptyState();
}

private void loadConversations() {
    String userId = firebaseManager.getCurrentUserId();
    if (userId == null) return;
    progressBar.setVisibility(View.VISIBLE);
    // Real-time listener – updates the list whenever Firestore changes
    conversationsListener = firebaseManager.listenToConversations(userId,
        new JobLinkerFirebaseManager.ListCallback<Conversation>() {
            @Override
            public void onSuccess(List<Conversation> list) {
                if (!isAdded()) return;
                progressBar.setVisibility(View.GONE);
                allConversations.clear();
                for (Conversation c : list) loadOtherUserDetails(c);
            }
            @Override
            public void onFailure(String error) {
                if (!isAdded()) return;
                progressBar.setVisibility(View.GONE);
                layoutEmpty.setVisibility(View.VISIBLE);
                Toast.makeText(requireContext(),
                    "Error loading conversations: " + error, Toast.LENGTH_SHORT).show();
            }
        });
}

private void loadOtherUserDetails(Conversation conversation) {
    String currentUserId = firebaseManager.getCurrentUserId();
    String otherUserId = null;
    for (String id : conversation.getParticipants()) {
        if (!id.equals(currentUserId)) { otherUserId = id; break; }
    }
    if (otherUserId == null) return;
}

```

```

firebaseManager.getUser(otherUserId, new JobLinkerFirebaseManager.DataCallback<User> [...])
@Override
public void onSuccess(User user) {
    if (!isAdded()) return;
    conversation.setOtherUserName(user.getUserName());
    conversation.setOtherUserAvatarUrl(user.getAvatarUrl());
    conversation.setOtherUserOnline(user.isOnline());
    if (!allConversations.contains(conversation)) {
        allConversations.add(conversation);
    }
    // Refresh filtered list
    String query = etSearch != null
        ? etSearch.getText().toString().trim() : "";
    filterConversations(query);
}
@Override public void onFailure(String error) {}
});
}
private void updateEmptyState() {
    if (!isAdded()) return;
    boolean empty = filtered.isEmpty();
    layoutEmpty.setVisibility(empty ? View.VISIBLE : View.GONE);
    recyclerConversations.setVisibility(empty ? View.GONE : View.VISIBLE);
}
@Override
public void onConversationClick(Conversation conversation) {
    String currentUserId = firebaseManager.getCurrentUserId();
    String otherUserId = null;
    for (String id : conversation.getParticipants()) {
        if (!id.equals(currentUserId)) { otherUserId = id; break; }
    }
    if (otherUserId == null) return;
    Intent intent = new Intent(requireContext(), ChatActivity.class);
    intent.putExtra(ChatActivity.EXTRA_USER_ID, otherUserId);
    intent.putExtra(ChatActivity.EXTRA_USER_NAME, conversation.getOtherUserName()) [...]
    intent.putExtra(ChatActivity.EXTRA_USER_AVATAR, conversation.getOtherUserAvatarU [...]
    intent.putExtra(ChatActivity.EXTRA_CONVERSATION_ID, conversation.getConversationId() [...]
    startActivity(intent);
}
@Override
public void onDestroyView() {
    super.onDestroyView();
    if (conversationsListener != null) conversationsListener.remove();
}
}
}

```


[illegible]

```
}  
private void setupRecyclerView() {  
if (recyclerMyJobs == null) return;  
jobAdapter = new JobAdapter(requireContext(), myJobs);  
recyclerMyJobs.setLayoutManager(new LinearLayoutManager(requireContext()));  
recyclerMyJobs.setAdapter(jobAdapter);  
recyclerMyJobs.setNestedScrollingEnabled(false);  
}  
  
// ████████████████████████████████████████████████████████████████  
// DATA LOADING  
// ████████████████████████████████████████████████████████████████  
  
public void loadUserProfile() {  
String userId = firebaseManager.getCurrentUserId();  
if (userId == null) return;  
firebaseManager.getUser(userId, new JobLinkerFirebaseManager.DataCallback<User>() {  
@Override  
public void onSuccess(User user) {  
if (!isAdded()) return;  
currentUser = user;  
displayUserProfile(user);  
loadStats(user);  
}  
@Override  
public void onFailure(String error) {  
if (!isAdded()) return;  
Toast.makeText(requireContext(),  
"Error loading profile: " + error, Toast.LENGTH_SHORT).show();  
}  
});  
}  
  
// ■■ Display profile data ████████████████████████████████████████  
private void displayUserProfile(User user) {  
// Avatar  
ImageUtils.loadCircularImage(requireContext(), user.getAvatarUrl(), ivAvatar);  
// Online indicator  
if (viewOnline != null)  
viewOnline.setVisibility(user.isOnline() ? View.VISIBLE : View.GONE);  
// Name & role chip  
if (tvUserName != null)  
tvUserName.setText(user.getUserName() != null ? user.getUserName() : "");  
if (chipRole != null) {  
boolean isEmployer = "employer".equalsIgnoreCase(user.getUserRole());  
chipRole.setText(isEmployer ? "Employer" : "Job Seeker");  
}  
// Location  
String location = buildLocation(user);  
if (layoutLocation != null) {  
if (!location.isEmpty()) {  
if (tvLocation != null) tvLocation.setText(location);  
layoutLocation.setVisibility(View.VISIBLE);  
} else {  
layoutLocation.setVisibility(View.GONE);  
}  
}  
// Email  
if (layoutEmail != null) {  
if (user.getEmail() != null && !user.getEmail().isEmpty()) {  
if (tvEmail != null) tvEmail.setText(user.getEmail());  
layoutEmail.setVisibility(View.VISIBLE);  
} else {  
layoutEmail.setVisibility(View.GONE);  
}  
}  
// Phone
```

```

if (layoutPhone != null) {
    if (user.getUserPhone() != null && !user.getUserPhone().isEmpty()) {
        if (tvPhone != null) tvPhone.setText(user.getUserPhone());
        layoutPhone.setVisibility(View.VISIBLE);
    } else {
        layoutPhone.setVisibility(View.GONE);
    }
}
// Bio
if (tvBio != null) {
    if (user.getUserBio() != null && !user.getUserBio().isEmpty()) {
        tvBio.setText(user.getUserBio());
        tvBio.setVisibility(View.VISIBLE);
    } else {
        tvBio.setVisibility(View.GONE);
    }
}
// Skills (job seekers only)
boolean isJobSeeker = !"employer".equalsIgnoreCase(user.getUserRole());
if (layoutSkills != null) {
    if (isJobSeeker && user.getUserSkills() != null && !user.getUserSkills().isEmpty()) {
        layoutSkills.setVisibility(View.VISIBLE);
        if (chipGroupSkills != null) {
            chipGroupSkills.removeAllViews();
            for (String skill : user.getUserSkills()) {
                Chip chip = new Chip(requireContext());
                chip.setText(skill);
                chip.setClickable(false);
                chipGroupSkills.addView(chip);
            }
        }
    } else {
        layoutSkills.setVisibility(View.GONE);
    }
}
// My Jobs section (employers only)
boolean isEmployer = "employer".equalsIgnoreCase(user.getUserRole());
if (layoutMyJobs != null) {
    if (isEmployer) {
        layoutMyJobs.setVisibility(View.VISIBLE);
        loadMyJobs(user.getId());
    } else {
        layoutMyJobs.setVisibility(View.GONE);
    }
}
// Saved jobs section (job seekers only)
if (layoutSavedJobs != null) {
    layoutSavedJobs.setVisibility(isJobSeeker ? View.VISIBLE : View.GONE);
}
}

// Load stats (counts) ████████████████████████████████
private void loadStats(User user) {
    String userId = user.getId();
    boolean isEmployer = "employer".equalsIgnoreCase(user.getUserRole());
    // Jobs count
    if (isEmployer) {
        firebaseManager.getJobsByEmployer(userId,
            new JobLinkerFirebaseManager.ListCallback<Job>() {
                @Override public void onSuccess(List<Job> jobs) {
                    if (!isAdded()) return;
                    if (tvJobsCount != null)
                        tvJobsCount.setText(String.valueOf(jobs.size()));
                }
            }
        );
    }
    @Override public void onFailure(String error) {

```

[illegible]

[illegible]

LoginEmailFragment

Email/password login with Google Sign-In option.

LoginEmailFragment.java (Java)

```
package com.example.joblinker.fragments;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.textfield.TextInputEditText;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.GoogleAuthProvider;
import com.google.android.libraries.identity.googleid.GetGoogleIdOption;
import com.google.android.libraries.identity.googleid.GoogleIdTokenCredential;
import androidx.credentials.CredentialManager;
import androidx.credentials.CredentialManagerCallback;
import androidx.credentials.GetCredentialRequest;
import androidx.credentials.GetCredentialResponse;
import androidx.credentials.exceptions.GetCredentialException;
import com.example.joblinker.R;
import com.example.joblinker.activities.LoginActivity;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.utils.ValidationHelper;
import java.util.concurrent.Executors;
public class LoginEmailFragment extends Fragment {
    // ■■■ Replace with your Web Client ID from Firebase Console → Authentication → Google → [...]
    private static final String WEB_CLIENT_ID = "943441972422-p7rs8rbnq9k28tl7rpre25t327mroa [...]";
    private TextInputEditText etEmail, etPassword;
    private TextView tvForgotPassword;
    private MaterialButton btnLogin, btnGoogleLogin;
    private ProgressBar progressBar;
    private JobLinkerFirebaseManager firebaseManager;
    @Nullable @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
        @Nullable ViewGroup container,
        @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.login_email, container, false);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        initView(view);
        setupListeners();
        return view;
    }
    private void initView(View v) {
        etEmail = v.findViewById(R.id.et_email);
        etPassword = v.findViewById(R.id.et_password);
        tvForgotPassword = v.findViewById(R.id.tv_forgot_password);
        btnLogin = v.findViewById(R.id.btn_login);
        btnGoogleLogin = v.findViewById(R.id.btn_google_login);
        progressBar = v.findViewById(R.id.progress_bar);
    }
    private void setupListeners() {
```

[illegible]

```

GetCredentialRequest request = new GetCredentialRequest.Builder()
    .addCredentialOption(googleIdOption)
    .build();
CredentialManager credentialManager =
    CredentialManager.create(requireContext());
credentialManager.getCredentialAsync(
    requireContext(),
    request,
    null,
    Executors.newSingleThreadExecutor(),
    new CredentialManagerCallback<GetCredentialResponse, GetCredentialException> [...]
@Override
public void onResult(GetCredentialResponse result) {
    requireActivity().runOnUiThread(() -> {
        handleGoogleCredential(result));
    }
@Override
public void onError(@NonNull GetCredentialException e) {
    requireActivity().runOnUiThread(() -> {
        progressBar.setVisibility(View.GONE);
        if (btnGoogleLogin != null) btnGoogleLogin.setEnabled(true);
        Toast.makeText(requireContext(),
            "Google sign-in failed: " + e.getMessage(),
            Toast.LENGTH_SHORT).show();
    });
}
}
);
} catch (Exception e) {
    progressBar.setVisibility(View.GONE);
    if (btnGoogleLogin != null) btnGoogleLogin.setEnabled(true);
    Toast.makeText(requireContext(),
        "Google sign-in unavailable", Toast.LENGTH_SHORT).show();
    }
}
private void handleGoogleCredential(GetCredentialResponse result) {
    try {
        GoogleIdTokenCredential googleCred =
            GoogleIdTokenCredential.createFrom(result.getCredential().getData());
        String idToken = googleCred.getIdToken();
        com.google.firebase.auth.AuthCredential firebaseCred =
            GoogleAuthProvider.getCredential(idToken, null);
        com.google.firebase.auth.FirebaseAuth.getInstance()
            .signInWithCredential(firebaseCred)
            .addOnCompleteListener(task -> {
                progressBar.setVisibility(View.GONE);
                if (btnGoogleLogin != null) btnGoogleLogin.setEnabled(true);
                if (task.isSuccessful() && task.getResult().getUser() != null) {
                    FirebaseUser user = task.getResult().getUser();
                    // Create user profile in Firestore if first login
                    createUserIfNew(user);
                } else {
                    Toast.makeText(requireContext(),
                        "Google sign-in failed", Toast.LENGTH_SHORT).show();
                }
            });
    } catch (Exception e) {
        progressBar.setVisibility(View.GONE);
        if (btnGoogleLogin != null) btnGoogleLogin.setEnabled(true);
        Toast.makeText(requireContext(),
            "Error: " + e.getMessage(), Toast.LENGTH_SHORT).show();
    }
}
private void createUserIfNew(FirebaseUser firebaseUser) {

```


[illegible]

LoginPhoneFragment

Phone number + OTP verification login.

LoginPhoneFragment.java (Java)

```
package com.example.joblinker.fragments;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ProgressBar;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.textfield.TextInputEditText;
import com.google.firebase.FirebaseException;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.auth.PhoneAuthCredential;
import com.google.firebase.auth.PhoneAuthOptions;
import com.google.firebase.auth.PhoneAuthProvider;
import com.example.joblinker.R;
import com.example.joblinker.activities.LoginActivity;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.utils.ValidationHelper;
import java.util.concurrent.TimeUnit;

public class LoginPhoneFragment extends Fragment {
    private TextInputEditText etCountryCode, etPhone, etVerificationCode;
    private MaterialButton btnSendCode, btnVerify;
    private ProgressBar progressBar;
    private JobLinkerFirebaseManager firebaseManager;
    private FirebaseAuth firebaseAuth;
    private String verificationId;
    private PhoneAuthProvider.ForceResendingToken resendToken;
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container [...]
    @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.login_phone, container, false);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        firebaseAuth = FirebaseAuth.getInstance();
        initializeViews(view);
        setupClickListeners();
        return view;
    }
    private void initializeViews(View view) {
        etCountryCode = view.findViewById(R.id.et_country_code);
        etPhone = view.findViewById(R.id.et_phone);
        etVerificationCode = view.findViewById(R.id.et_verification_code);
        btnSendCode = view.findViewById(R.id.btn_send_code);
        btnVerify = view.findViewById(R.id.btn_verify);
        progressBar = view.findViewById(R.id.progress_bar);
    }
    private void setupClickListeners() {
        btnSendCode.setOnClickListener(v -> sendVerificationCode());
        btnVerify.setOnClickListener(v -> verifyCode());
    }
}
```

```
// Step 1: send OTP
private void sendVerificationCode() {
    String countryCode = etCountryCode.getText().toString().trim();
    String phone = etPhone.getText().toString().trim();
    if (ValidationHelper.isEmpty(countryCode)) {
        etCountryCode.setError(getString(R.string.error_empty_field));
        etCountryCode.requestFocus();
        return;
    }
    if (ValidationHelper.isEmpty(phone)) {
        etPhone.setError(getString(R.string.error_empty_field));
        etPhone.requestFocus();
        return;
    }
    if (!ValidationHelper.isValidPhone(phone)) {
        etPhone.setError(getString(R.string.error_invalid_phone));
        etPhone.requestFocus();
        return;
    }
    // Ensure country code starts with +
    if (!countryCode.startsWith("+")) countryCode = "+" + countryCode;
    String fullPhone = countryCode + phone;
    progressBar.setVisibility(View.VISIBLE);
    btnSendCode.setEnabled(false);
    PhoneAuthOptions options = PhoneAuthOptions.newBuilder(firebaseAuth)
        .setPhoneNumber(fullPhone)
        .setTimeout(60L, TimeUnit.SECONDS)
        .setActivity(requireActivity())
        .setCallbacks(new PhoneAuthProvider.OnVerificationStateChangedCallbacks() {
            // Auto-verified (instant on same device / test numbers)
            @Override
            public void onVerificationCompleted(@NonNull PhoneAuthCredential credent [...]) {
                if (!isAdded()) return;
                progressBar.setVisibility(View.GONE);
                signInWithCredential(credential);
            }
            @Override
            public void onVerificationFailed(@NonNull FirebaseException e) {
                if (!isAdded()) return;
                progressBar.setVisibility(View.GONE);
                btnSendCode.setEnabled(true);
                Toast.makeText(requireContext(),
                    "Verification failed: " + e.getMessage(),
                    Toast.LENGTH_LONG).show();
            }
        })
        // OTP sent – show the code input
        @Override
        public void onCodeSent(@NonNull String vId,
            @NonNull PhoneAuthProvider.ForceResendingToken tok [...]) {
            if (!isAdded()) return;
            verificationId = vId;
            resendToken = token;
            progressBar.setVisibility(View.GONE);
            btnSendCode.setEnabled(true);
            etVerificationCode.setVisibility(View.VISIBLE);
            btnVerify.setVisibility(View.VISIBLE);
            Toast.makeText(requireContext(),
                getString(R.string.verification_code_sent),
                Toast.LENGTH_SHORT).show();
        }
    }.build();
    PhoneAuthProvider.verifyPhoneNumber(options);
}
```

```
// ■■■ Step 2: verify OTP ■■■■
private void verifyCode() {
String code = etVerificationCode.getText().toString().trim();
if (ValidationHelper.isEmpty(code)) {
etVerificationCode.setError(getString(R.string.error_empty_field));
etVerificationCode.requestFocus();
return;
}
if (code.length() != 6) {
etVerificationCode.setError("Code must be 6 digits");
etVerificationCode.requestFocus();
return;
}
if (verificationId == null) {
Toast.makeText(requireContext(),
"Pleace request a code first", Toast.LENGTH_SHORT).show();
return;
}
progressBar.setVisibility(View.VISIBLE);
btnVerify.setEnabled(false);
PhoneAuthCredential credential =
PhoneAuthProvider.getCredential(verificationId, code);
signInWithCredential(credential);
}

// ■■■ Sign in ■■■■
private void signInWithCredential(PhoneAuthCredential credential) {
firebaseManager.signInWithPhoneCredential(credential,
new JobLinkerFirebaseManager.AuthCallback() {
@Override
public void onSuccess(FirebaseUser user) {
if (!isAdded()) return;
progressBar.setVisibility(View.GONE);
// Navigate to main
if (getActivity() instanceof LoginActivity) {
((LoginActivity) getActivity()).onLoginSuccess(user);
}
}
}
@Override
public void onFailure(String error) {
if (!isAdded()) return;
progressBar.setVisibility(View.GONE);
btnVerify.setEnabled(true);
Toast.makeText(requireContext(),
"Sign in failed: " + error, Toast.LENGTH_LONG).show();
}
});
}
```

RegisterStep1Fragment

Step 1: basic user info and role selection.

RegisterStep1Fragment.java (Java)

```
package com.example.joblinker.fragments;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import com.google.android.material.button.MaterialButtonToggleGroup;
import com.google.android.material.textfield.TextInputEditText;
import com.example.joblinker.R;
import com.example.joblinker.activities.RegisterActivity;
import com.example.joblinker.models.User;
import com.example.joblinker.utils.ValidationHelper;
public class RegisterStep1Fragment extends Fragment {
    private TextInputEditText etFullName, etEmail, etPhone, etPassword;
    private MaterialButtonToggleGroup toggleRole;
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container [...]
    @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_register_step1, container, false);
        initializeViews(view);
        return view;
    }
    private void initializeViews(View view) {
        etFullName = view.findViewById(R.id.et_full_name);
        etEmail = view.findViewById(R.id.et_email);
        etPhone = view.findViewById(R.id.et_phone);
        etPassword = view.findViewById(R.id.et_password);
        toggleRole = view.findViewById(R.id.toggle_role);
    }
    public boolean validateAndSaveData() {
        String fullName = etFullName.getText().toString().trim();
        String email = etEmail.getText().toString().trim();
        String phone = etPhone.getText().toString().trim();
        String password = etPassword.getText().toString().trim();
        // Validation
        if (ValidationHelper.isEmpty(fullName)) {
            etFullName.setError(getString(R.string.error_empty_field));
            etFullName.requestFocus();
            return false;
        }
        if (!ValidationHelper.isValidName(fullName)) {
            etFullName.setError("Please enter a valid name");
            etFullName.requestFocus();
            return false;
        }
        if (ValidationHelper.isEmpty(email)) {
            etEmail.setError(getString(R.string.error_empty_field));
            etEmail.requestFocus();
            return false;
        }
        if (!ValidationHelper.isValidEmail(email)) {
```

```
etEmail.setError(getString(R.string.error_invalid_email));
etEmail.requestFocus();
return false;
}
if (ValidationHelper.isEmpty(phone)) {
etPhone.setError(getString(R.string.error_empty_field));
etPhone.requestFocus();
return false;
}
if (!ValidationHelper.isValidPhone(phone)) {
etPhone.setError(getString(R.string.error_invalid_phone));
etPhone.requestFocus();
return false;
}
if (ValidationHelper.isEmpty(password)) {
etPassword.setError(getString(R.string.error_empty_field));
etPassword.requestFocus();
return false;
}
if (!ValidationHelper.isValidPassword(password)) {
etPassword.setError(getString(R.string.error_short_password));
etPassword.requestFocus();
return false;
}
// Determine role
String role = "JobSeeker";
if (toggleRole.getCheckedButtonId() == R.id.btn_employer) {
role = "Employer";
}
// Save to RegisterActivity
if (getActivity() instanceof RegisterActivity) {
RegisterActivity activity = (RegisterActivity) getActivity();
// 1. Get the existing User object and update it
User user = activity.getRegistrationUser();
if (user == null) {
user = new User(); // Fallback just in case it's null
}
user.setUserName(fullName);
user.setUserEmail(email);
user.setUserPhone(phone);
user.setUserRole(role);
// 2. Save the User object back to Activity
activity.setRegistrationUser(user);
// 3. THIS WAS MISSING: Save the password to the Activity!
activity.setRegistrationPassword(password);
}
return true; // Proceed to next step
}
}
```

RegisterStep2Fragment

Step 2: professional details and company info.

RegisterStep2Fragment.java (Java)

```
package com.example.joblinker.fragments;
import android.os.Bundle;
import android.os.CountDownTimer;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.textfield.TextInputEditText;
import com.google.android.material.textfield.TextInputLayout;
import com.example.joblinker.R;
import com.example.joblinker.activities.RegisterActivity;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.models.User;
import com.example.joblinker.utils.ValidationHelper;
public class RegisterStep2Fragment extends Fragment {
    private TextView tvVerificationMessage;
    private TextView tvEmailDisplay;
    private TextInputLayout tilVerificationCode;
    private TextInputEditText etVerificationCode;
    private TextView tvResendCode;
    private TextView tvResendTimer;
    private MaterialButton btnVerify;
    private MaterialButton btnSkipVerification;
    private JobLinkerFirebaseManager firebaseManager;
    private CountDownTimer resendTimer;
    private boolean canResend = false;
    private static final int RESEND_TIMEOUT = 60; // seconds
    private boolean emailVerified = false;
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container [...]
    @Nullable Bundle savedInstanceState) {
        // Ensure this layout name matches your actual XML file name!
        View view = inflater.inflate(R.layout.register_step2, container, false);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        initializeViews(view);
        setupClickListeners();
        displayEmailInfo();
        // Try to send verification, or simulate if user isn't created yet
        sendEmailVerification();
        return view;
    }
    private void initializeViews(View view) {
        tvVerificationMessage = view.findViewById(R.id.tv_verification_message);
        tvEmailDisplay = view.findViewById(R.id.tv_email_display);
        tilVerificationCode = view.findViewById(R.id.til_verification_code);
        etVerificationCode = view.findViewById(R.id.et_verification_code);
        tvResendCode = view.findViewById(R.id.tv_resend_code);
        tvResendTimer = view.findViewById(R.id.tv_resend_timer);
    }
```

```

btnVerify = view.findViewById(R.id.btn_verify);
btnSkipVerification = view.findViewById(R.id.btn_skip_verification);
}
private void setupClickListeners() {
btnVerify.setOnClickListener(v -> verifyEmailCode());
tvResendCode.setOnClickListener(v -> {
if (canResend) {
sendEmailVerification();
} else {
Toast.makeText(requireContext(),
"Please wait before requesting a new code",
Toast.LENGTH_SHORT).show();
}
});
btnSkipVerification.setOnClickListener(v -> skipEmailVerification());
}
private void displayEmailInfo() {
if (getActivity() instanceof RegisterActivity) {
RegisterActivity activity = (RegisterActivity) getActivity();
User user = activity.getRegistrationUser();
if (user != null && user.getUserEmail() != null) {
tvEmailDisplay.setText(user.getUserEmail());
String message = String.format(
"We've sent a 6-digit verification code to %. Please enter it below [...]
user.getUserEmail()
);
tvVerificationMessage.setText(message);
}
}
}
private void sendEmailVerification() {
try {
// This will attempt to use your custom Firebase Manager
firebaseManager.sendEmailVerification(new JobLinkerFirebaseManager.VoidCallback( [...]
@Override
public void onSuccess() {
Toast.makeText(requireContext(), "Verification code sent to your email", [...]
startResendTimer();
}
@Override
public void onFailure(String error) {
// Fallback to simulation if Firebase fails (because account isn't creat [...]
simulateEmailVerification();
}
});
} catch (Exception e) {
// Safety catch
simulateEmailVerification();
}
}
private void simulateEmailVerification() {
Toast.makeText(requireContext(), "(Simulation) Code sent to your email", Toast.LENGTH [...]
startResendTimer();
}
private void verifyEmailCode() {
String code = etVerificationCode.getText().toString().trim();
if (ValidationHelper.isEmpty(code)) {
tilVerificationCode.setError("Enter verification code");
etVerificationCode.requestFocus();
return;
}
if (code.length() != 6) {
tilVerificationCode.setError("Code must be 6 digits");
etVerificationCode.requestFocus();
}
}

```



```

return;
}
tilVerificationCode.setError(null);
// Disable button during verification
btnVerify.setEnabled(false);
btnVerify.setText("Verifying...");
// Simulate successful verification since Firebase natively uses links, not 6-digit [...]
new android.os.Handler(android.os.Looper.getMainLooper()).postDelayed(() -> {
    emailVerified = true;
    Toast.makeText(requireContext(), "Email verified successfully!", Toast.LENGTH_SH [...])
// Auto-move to next step
moveToNextStep();
}, 1000);
}
private void skipEmailVerification() {
    Toast.makeText(requireContext(), "You can verify your email later in settings", Toas [...]
    moveToNextStep();
}
private void moveToNextStep() {
    if (getActivity() instanceof RegisterActivity) {
        ((RegisterActivity) getActivity()).moveToNextStep();
    }
}
private void startResendTimer() {
    canResend = false;
    tvResendCode.setEnabled(false);
    tvResendCode.setAlpha(0.5f);
    tvResendTimer.setVisibility(View.VISIBLE);
    if (resendTimer != null) {
        resendTimer.cancel();
    }
    resendTimer = new CountDownTimer(RESEND_TIMEOUT * 1000, 1000) {
        @Override
        public void onTick(long millisUntilFinished) {
            int secondsRemaining = (int) (millisUntilFinished / 1000);
            if (tvResendTimer != null) {
                tvResendTimer.setText(String.format("Resend code in %ds", secondsRemaini [...])
            }
        }
    }
    @Override
    public void onFinish() {
        canResend = true;
        if (tvResendCode != null) {
            tvResendCode.setEnabled(true);
            tvResendCode.setAlpha(1.0f);
        }
        if (tvResendTimer != null) {
            tvResendTimer.setVisibility(View.GONE);
        }
    }
};
resendTimer.start();
}
/**
 * This is called by the global "Next" button inside RegisterActivity
 */
public boolean validateAndSaveData() {
    // Email verification is optional. If the user clicks the main Activity "Next" butto [...]
    // we just allow them to pass to Step 3.
    return true;
}
@Override
public void onDestroyView() {
    super.onDestroyView();

```

```
if (resendTimer != null) {  
    resendTimer.cancel();  
    resendTimer = null;  
}  
}  
}
```

RegisterStep3Fragment

Step 3: creates Firebase user and saves profile to Firestore.

RegisterStep3Fragment.java (Java)

```
package com.example.joblinker.fragments;
import android.os.Bundle;
import android.os.CountDownTimer;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.textfield.TextInputEditText;
import com.google.android.material.textfield.TextInputLayout;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.PhoneAuthCredential;
import com.google.firebase.auth.PhoneAuthProvider;
import com.example.joblinker.R;
import com.example.joblinker.activities.RegisterActivity;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.models.User;
import com.example.joblinker.utils.SharedPreferencesManager;
import com.example.joblinker.utils.ValidationHelper;
public class RegisterStep3Fragment extends Fragment {
    private TextView tvSmsVerificationMessage;
    private TextView tvPhoneDisplay;
    private MaterialButton btnSendSmsCode;
    private TextInputLayout tilSmsVerificationCode;
    private TextInputEditText etSmsVerificationCode;
    private LinearLayout layoutResendSms;
    private TextView tvResendSmsCode;
    private TextView tvSmsResendTimer;
    private MaterialButton btnVerifySms;
    private MaterialButton btnSkipSmsVerification;
    private MaterialButton btnCompleteRegistration;
    private JobLinkerFirebaseManager firebaseManager;
    private SharedPreferencesManager prefsManager;
    private CountDownTimer smsResendTimer;
    private boolean canResendSms = false;
    private boolean smsVerified = false;
    private static final int RESEND_TIMEOUT = 60; // seconds
    private String verificationId;
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container [...]
    @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_register_step3, container, false);
        firebaseManager = JobLinkerFirebaseManager.getInstance();
        prefsManager = SharedPreferencesManager.getInstance(requireContext());
        initializeViews(view);
        setupClickListeners();
        displayPhoneInfo();
        return view;
    }
}
```

```

}
private void initializeViews(View view) {
    tvSmsVerificationMessage = view.findViewById(R.id.tv_sms_verification_message);
    tvPhoneDisplay = view.findViewById(R.id.tv_phone_display);
    btnSendSmsCode = view.findViewById(R.id.btn_send_sms_code);
    tilSmsVerificationCode = view.findViewById(R.id.til_sms_verification_code);
    etSmsVerificationCode = view.findViewById(R.id.et_sms_verification_code);
    layoutResendSms = view.findViewById(R.id.layout_resend_sms);
    tvResendSmsCode = view.findViewById(R.id.tv_resend_sms_code);
    tvSmsResendTimer = view.findViewById(R.id.tv_sms_resend_timer);
    btnVerifySms = view.findViewById(R.id.btn_verify_sms);
    btnSkipSmsVerification = view.findViewById(R.id.btn_skip_sms_verification);
    btnCompleteRegistration = view.findViewById(R.id.btn_complete_registration);
}
private void setupClickListeners() {
    btnSendSmsCode.setOnClickListener(v -> sendSmsCode());
    btnVerifySms.setOnClickListener(v -> verifySmsCode());
    tvResendSmsCode.setOnClickListener(v -> {
        if (canResendSms) {
            sendSmsCode();
        } else {
            Toast.makeText(requireContext(),
                "Please wait before requesting a new code",
                Toast.LENGTH_SHORT).show();
        }
    });
    btnSkipSmsVerification.setOnClickListener(v -> completeRegistration());
    btnCompleteRegistration.setOnClickListener(v -> completeRegistration());
}
private void displayPhoneInfo() {
    if (getActivity() instanceof RegisterActivity) {
        RegisterActivity activity = (RegisterActivity) getActivity();
        User user = activity.getRegistrationUser();
        if (user != null && user.getUserPhone() != null) {
            tvPhoneDisplay.setText(user.getUserPhone());
        }
    }
}
private void sendSmsCode() {
    btnSendSmsCode.setEnabled(false);
    btnSendSmsCode.setText("Sending...");
    // TODO: Implement Firebase Phone Authentication
    // For demonstration, we'll simulate sending SMS
    simulateSendSms();
}
private void simulateSendSms() {
    // Simulate delay
    new android.os.Handler(android.os.Looper.getMainLooper()).postDelayed(() -> {
        // Show verification input
        btnSendSmsCode.setVisibility(View.GONE);
        tilSmsVerificationCode.setVisibility(View.VISIBLE);
        layoutResendSms.setVisibility(View.VISIBLE);
        btnVerifySms.setVisibility(View.VISIBLE);
        btnCompleteRegistration.setVisibility(View.VISIBLE);
        Toast.makeText(requireContext(),
            "Verification code sent via SMS",
            Toast.LENGTH_SHORT).show();
        startSmsResendTimer();
    }, 1500);
}
private void verifySmsCode() {
    String code = etSmsVerificationCode.getText().toString().trim();
    if (ValidationHelper.isEmpty(code)) {
        tilSmsVerificationCode.setError("Enter verification code");
    }
}

```

```

return;
}
if (code.length() != 6) {
    tilSmsVerificationCode.setError("Code must be 6 digits");
    return;
}
tilSmsVerificationCode.setError(null);
// TODO: Verify with Firebase Phone Auth
// For now, just mark as verified
btnVerifySms.setEnabled(false);
btnVerifySms.setText("Verifying...");
// Simulate verification
new android.os.Handler(android.os.Looper.getMainLooper()).postDelayed(() -> {
    smsVerified = true;
    Toast.makeText(requireContext(),
        "Phone number verified successfully!",
        Toast.LENGTH_SHORT).show();
    // Auto-complete registration
    completeRegistration();
}, 1000);
}

private void startSmsResendTimer() {
    canResendSms = false;
    tvResendSmsCode.setEnabled(false);
    tvResendSmsCode.setAlpha(0.5f);
    tvSmsResendTimer.setVisibility(View.VISIBLE);
    smsResendTimer = new CountDownTimer(RESEND_TIMEOUT * 1000, 1000) {
        @Override
        public void onTick(long millisUntilFinished) {
            int secondsRemaining = (int) (millisUntilFinished / 1000);
            tvSmsResendTimer.setText(
                String.format("Resend code in %ds", secondsRemaining)
            );
        }
        @Override
        public void onFinish() {
            canResendSms = true;
            tvResendSmsCode.setEnabled(true);
            tvResendSmsCode.setAlpha(1.0f);
            tvSmsResendTimer.setVisibility(View.GONE);
        }
    };
    smsResendTimer.start();
}

public void validateAndCompleteRegistration() {
    completeRegistration();
}

private void completeRegistration() {
    if (getActivity() instanceof RegisterActivity) {
        RegisterActivity activity = (RegisterActivity) getActivity();
        User user = activity.getRegistrationUser();
        String password = activity.getRegistrationPassword();
        if (user == null || password == null) {
            Toast.makeText(requireContext(),
                "Registration data missing",
                Toast.LENGTH_SHORT).show();
            return;
        }
        // Disable buttons
        btnCompleteRegistration.setEnabled(false);
        btnCompleteRegistration.setText("Creating account...");
        // Register user with Firebase
        firebaseManager.registerWithEmail(user.getUserEmail(), password,
            new JobLinkerFirebaseManager.AuthCallback() {

```

```

@Override
public void onSuccess(FirebaseUser firebaseUser) {
    // Set user ID
    user.setUserId(firebaseUser.getUid());
    user.setEmailVerified(firebaseManager.isEmailVerified());
    user.setPhoneVerified(smsVerified);
    // Create user in Firestore
    firebaseManager.createUser(user, new JobLinkerFirebaseManager.Vo [...])
@Override
public void onSuccess() {
    // Save to preferences
    prefsManager.setLoggedIn(true);
    prefsManager.setUserId(user.getUserId());
    prefsManager.setUserName(user.getUserName());
    prefsManager.setUserEmail(user.getUserEmail());
    prefsManager.setUserRole(user.getUserRole());
    Toast.makeText(requireContext(),
        "Registration successful!",
        Toast.LENGTH_SHORT).show();
    // Navigate to main activity
    activity.onRegistrationComplete();
}
@Override
public void onFailure(String error) {
    btnCompleteRegistration.setEnabled(true);
    btnCompleteRegistration.setText(R.string.complete_regist [...])
    Toast.makeText(requireContext(),
        "Error creating profile: " + error,
        Toast.LENGTH_SHORT).show();
}
});
}
@Override
public void onFailure(String error) {
    btnCompleteRegistration.setEnabled(true);
    btnCompleteRegistration.setText(R.string.complete_registration);
    Toast.makeText(requireContext(),
        "Registration failed: " + error,
        Toast.LENGTH_SHORT).show();
}
});
}
@Override
public void onDestroyView() {
    super.onDestroyView();
    if (smsResendTimer != null) {
        smsResendTimer.cancel();
    }
}
}
}

```


Source Code

Adapters – RecyclerView & Pager

JobAdapter

RecyclerView adapter for job cards. Handles save/unsave toggle and click listeners.

JobAdapter.java (Java)

```
package com.example.joblinker.adapters;
import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageButton;
import android.widget.ImageView;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.recyclerview.widget.DiffUtil;
import androidx.recyclerview.widget.RecyclerView;
import com.google.android.material.chip.Chip;
import com.example.joblinker.R;
import com.example.joblinker.models.Job;
import com.example.joblinker.utils.DateTimeHelper;
import com.example.joblinker.utils.ImageUtils;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class JobAdapter extends RecyclerView.Adapter<JobAdapter.JobViewHolder> {
    private final Context context;
    private final List<Job> jobs;
    private final Set<String> savedJobIds = new HashSet<>();
    private OnJobClickListener listener;
    public interface OnJobClickListener {
        void onJobClick(Job job);
        void onSaveClick(Job job, boolean isSaved);
    }
    public JobAdapter(Context context, List<Job> jobs) {
        this.context = context;
        this.jobs = jobs;
    }
    public void setOnJobClickListener(OnJobClickListener listener) {
        this.listener = listener;
    }
    public void submitList(List<Job> newJobs) {
        final List<Job> old = new ArrayList<>(jobs);
        DiffUtil.DiffResult diff = DiffUtil.calculateDiff(new DiffUtil.Callback() {
            @Override public int getOldListSize() { return old.size(); }
            @Override public int getNewListSize() { return newJobs.size(); }
            @Override public boolean areItemsTheSame(int o, int n) {
                String oid = old.get(o).getJobId(), nid = newJobs.get(n).getJobId();
                return oid != null && oid.equals(nid);
            }
            @Override public boolean areContentsTheSame(int o, int n) {
                Job a = old.get(o), b = newJobs.get(n);
                return safeEq(a.getJobTitle(), b.getJobTitle())
                    && safeEq(a.getJobCompany(), b.getJobCompany())
                    && a.getJobSalaryMin() == b.getJobSalaryMin();
            }
        });
    }
    public void clear() {
        jobs.clear();
    }
    public void addAll(List<Job> newJobs) {
        jobs.addAll(newJobs);
    }
}
```

```

diff.dispatchUpdatesTo(this);
}
public void setSavedJobIds(Set<String> ids) {
    savedJobIds.clear();
    if (ids != null) savedJobIds.addAll(ids);
    notifyDataSetChanged();
}
public void markJobSaved(String jobId, boolean saved) {
    if (jobId == null) return;
    if (saved) savedJobIds.add(jobId); else savedJobIds.remove(jobId);
    for (int i = 0; i < jobs.size(); i++) {
        if (jobId.equals(jobs.get(i).getJobId())) {
            notifyItemChanged(i, "save_changed");
            break;
        }
    }
}
@NonNull @Override
public JobViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
    return new JobViewHolder(LayoutInflater.from(context)
        .inflate(R.layout.item_job, parent, false));
}
@Override
public void onBindViewHolder(@NonNull JobViewHolder h, int pos) {
    bind(h, jobs.get(pos));
}
@Override
public void onBindViewHolder(@NonNull JobViewHolder h, int pos, @NonNull List<Object> pa [...])
if (!payloads.isEmpty() && "save_changed".equals(payloads.get(0))) {
    updateSaveIcon(h, jobs.get(pos)); return;
}
super.onBindViewHolder(h, pos, payloads);
}
private void bind(@NonNull JobViewHolder h, Job job) {
    ImageUtils.loadCompanyLogo(context, job.getCompanyLogoUrl(), h.ivCompanyLogo);
    h.tvJobTitle.setText(safe(job.getJobTitle()));
    h.tvCompanyName.setText(safe(job.getJobCompany()));
    h.tvLocation.setText(safe(job.getLocation()));
    h.tvSalary.setText(safe(job.getSalaryRange()));
    h.chipCategory.setText(safe(job.getJobCategory()));
    h.chipJobType.setText(safe(job.getJobType()));
    // Posted time + applicant count
    String time = DateTimeHelper.getRelativeTime(job.getCreatedAt());
    int apps = job.getApplicantCount();
    h.tvPostedTime.setText(apps > 0 ? time + " · " + apps + " applied" : time);
    // Remote chip
    if (h.chipRemote != null) {
        boolean remote = "Remote".equalsIgnoreCase(job.getJobType())
        || (job.getLocation() != null && job.getLocation().toLowerCase().contains("r [...])
    h.chipRemote.setVisibility(remote ? View.VISIBLE : View.GONE);
    }
    // Urgent badge
    if (h.tvUrgent != null) {
        h.tvUrgent.setVisibility(job.isUrgent() ? View.VISIBLE : View.GONE);
    }
    updateSaveIcon(h, job);
    h.itemView.setOnClickListener(v -> { if (listener != null) listener.onJobClick(job); [...])
    h.btnSave.setOnClickListener(v -> {
        if (listener == null || job.getJobId() == null) return;
        boolean cur = savedJobIds.contains(job.getJobId());
        markJobSaved(job.getJobId(), !cur);
        listener.onSaveClick(job, !cur);
    });
}
}

```

```
private void updateSaveIcon(@NonNull JobViewHolder h, Job job) {
    boolean saved = job.getJobId() != null && savedJobIds.contains(job.getJobId());
    h.btnSave.setImageResource(saved ? R.drawable.ic_bookmark : R.drawable.ic_bookmark_b [...]
}
private static String safe(String s) { return s != null ? s : ""; }
private static boolean safeEq(String a, String b) { return a == null ? b == null : a.equ [...]
@Override public int getItemCount() { return jobs.size(); }
static class JobViewHolder extends RecyclerView.ViewHolder {
    ImageView ivCompanyLogo;
    TextView tvJobTitle, tvCompanyName, tvLocation, tvSalary, tvPostedTime, tvUrgent;
    Chip chipCategory, chipJobType, chipRemote;
    ImageButton btnSave;
    JobViewHolder(@NonNull View v) {
        super(v);
        ivCompanyLogo = v.findViewById(R.id.iv_company_logo);
        tvJobTitle = v.findViewById(R.id.tv_job_title);
        tvCompanyName = v.findViewById(R.id.tv_company_name);
        tvLocation = v.findViewById(R.id.tv_location);
        tvSalary = v.findViewById(R.id.tv_salary);
        chipCategory = v.findViewById(R.id.chip_category);
        chipJobType = v.findViewById(R.id.chip_job_type);
        chipRemote = v.findViewById(R.id.chip_remote);
        tvPostedTime = v.findViewById(R.id.tv_posted_time);
        tvUrgent = v.findViewById(R.id.tv_urgent);
        btnSave = v.findViewById(R.id.btn_save);
    }
}
}
```

ConversationAdapter

RecyclerView adapter for conversation list items. Shows avatar, name, last message, timestamp, and unread badge.

ConversationAdapter.java (Java)

```
package com.example.joblinker.adapters;
import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;
import com.example.joblinker.R;
import com.example.joblinker.models.Conversation;
import com.example.joblinker.utils.DateTimeHelper;
import com.example.joblinker.utils.ImageUtils;
import java.util.List;
public class ConversationAdapter extends RecyclerView.Adapter<ConversationAdapter.ConversationViewHolder> {
    private Context context;
    private List<Conversation> conversations;
    private OnConversationClickListener listener;
    public interface OnConversationClickListener {
        void onConversationClick(Conversation conversation);
    }
    public ConversationAdapter(Context context, List<Conversation> conversations,
        OnConversationClickListener listener) {
        this.context = context;
        this.conversations = conversations;
        this.listener = listener;
    }
    @NonNull
    @Override
    public ConversationViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(context).inflate(R.layout.item_conversation, parent, false);
        return new ConversationViewHolder(view);
    }
    @Override
    public void onBindViewHolder(@NonNull ConversationViewHolder holder, int position) {
        Conversation conversation = conversations.get(position);
        // Avatar
        ImageUtils.loadCircularImage(context, conversation.getOtherUserAvatarUrl(), holder.imageView);
        // User info
        holder.tvName.setText(conversation.getOtherUserName());
        holder.tvLastMessage.setText(conversation.getLastMessage());
        holder.tvTimestamp.setText(DateTimeHelper.getMessageTimestamp(conversation.getLastMessageTimestamp()));
        // Online status
        holder.viewOnline.setVisibility(conversation.isOtherUserOnline() ? View.VISIBLE : View.GONE);
        // Unread count
        if (conversation.hasUnreadMessages()) {
            holder.tvUnreadCount.setVisibility(View.VISIBLE);
            holder.tvUnreadCount.setText(String.valueOf(conversation.getUnreadCount()));
        } else {
            holder.tvUnreadCount.setVisibility(View.GONE);
        }
        // Click listener
        holder.itemView.setOnClickListener(v -> {
```

```
if (listener != null) {
    listener.onConversationClick(conversation);
}
});
}
@Override
public int getItemCount() {
    return conversations.size();
}
static class ConversationViewHolder extends RecyclerView.ViewHolder {
    ImageView ivAvatar;
    View viewOnline;
    TextView tvName, tvLastMessage, tvTimestamp, tvUnreadCount;
    public ConversationViewHolder(@NonNull View itemView) {
        super(itemView);
        ivAvatar = itemView.findViewById(R.id.iv_avatar);
        viewOnline = itemView.findViewById(R.id.view_online);
        tvName = itemView.findViewById(R.id.tv_name);
        tvLastMessage = itemView.findViewById(R.id.tv_last_message);
        tvTimestamp = itemView.findViewById(R.id.tv_timestamp);
        tvUnreadCount = itemView.findViewById(R.id.tv_unread_count);
    }
}
}
```

MessageAdapter

RecyclerView adapter for chat messages. Handles 6 message types with sent/received variants.

MessageAdapter.java (Java)

```
package com.example.joblinker.adapters;
import android.content.Context;
import android.content.Intent;
import android.net.Uri;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;
import com.bumptech.glide.Glide;
import com.example.joblinker.R;
import com.example.joblinker.activities.ChatActivity;
import com.example.joblinker.models.Message;
import com.example.joblinker.utils.DateTimeHelper;
import com.example.joblinker.utils.ImageUtils;
import java.util.List;
public class MessageAdapter extends RecyclerView.Adapter<RecyclerView.ViewHolder> {
    private static final int VIEW_SENT = 1;
    private static final int VIEW_RECEIVED = 2;
    private final Context context;
    private final List<Message> messages;
    private final String currentUserId;
    // Track which URL is currently "playing" to show correct icon
    private String currentlyPlayingUrl = null;
    public MessageAdapter(Context context, List<Message> messages, String currentUserId) {
        this.context = context;
        this.messages = messages;
        this.currentUserId = currentUserId;
    }
    @Override public int getItemViewType(int position) {
        String senderId = messages.get(position).getMessageSenderId();
        return senderId != null && senderId.equals(currentUserId)
            ? VIEW_SENT : VIEW_RECEIVED;
    }
    @NonNull @Override
    public RecyclerView.ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType [...]) {
        LayoutInflater inf = LayoutInflater.from(context);
        if (viewType == VIEW_SENT)
            return new SentHolder(inf.inflate(R.layout.item_message_sent, parent, false));
        return new ReceivedHolder(inf.inflate(R.layout.item_message_received, parent, false) [...])
    }
    @Override
    public void onBindViewHolder(@NonNull RecyclerView.ViewHolder holder, int position) {
        Message msg = messages.get(position);
        if (holder instanceof SentHolder) bindSent ((SentHolder) holder, msg);
        else bindReceived((ReceivedHolder) holder, msg);
    }
    private void bindSent(SentHolder h, Message msg) {
        bindContent(h.tvMessage, h.ivImage, h.layoutMedia, h.tvMediaLabel, msg);
        h.tvTimestamp.setText(DateTimeHelper.formatTime(msg.getMessageTimestamp()));
        if (h.ivStatus != null)
            h.ivStatus.setImageResource(msg.isMessageRead())
```

```

? R.drawable.ic_check_double : R.drawable.ic_check);
}
private void bindReceived(ReceivedHolder h, Message msg) {
    ImageUtils.loadCircularImageCached(context, msg.getSenderAvatarUrl(), h.ivAvatar);
    bindContent(h.tvMessage, h.ivImage, h.layoutMedia, h.tvMediaLabel, msg);
    h.tvTimestamp.setText(DateTimeHelper.formatTime(msg.getMessageTimestamp()));
}
private void bindContent(Textview tvMessage, ImageView ivImage,
    View layoutMedia, Textview tvMediaLabel, Message msg) {
    String type = msg.getMessageType();
    if (type == null) type = "text";
    switch (type) {
        case "image":
            tvMessage.setVisibility(View.GONE);
            if (layoutMedia != null) layoutMedia.setVisibility(View.GONE);
            if (ivImage != null) {
                ivImage.setVisibility(View.VISIBLE);
                Glide.with(context).load(msg.getImageUrl())
                    .placeholder(R.drawable.ic_photo).centerCrop().into(ivImage);
                ivImage.setOnClickListener(v -> openUrl(msg.getImageUrl()));
            }
            break;
        case "audio":
            tvMessage.setVisibility(View.GONE);
            if (ivImage != null) ivImage.setVisibility(View.GONE);
            if (layoutMedia != null) {
                layoutMedia.setVisibility(View.VISIBLE);
                String baseLabel = msg.getMessageText() != null
                    ? msg.getMessageText() : "■ Voice message";
                // Show play/stop state
                if (tvMediaLabel != null) {
                    boolean isPlaying = baseLabel.equals(currentlyPlayingUrl)
                        || (msg.getImageUrl() != null
                            && msg.getImageUrl().equals(currentlyPlayingUrl));
                    tvMediaLabel.setText(isPlaying ? "■ " + baseLabel : "■ " + baseLabel [...])
                }
                layoutMedia.setOnClickListener(v -> {
                    if (!(context instanceof ChatActivity)) return;
                    ChatActivity activity = (ChatActivity) context;
                    final String audioUrl = msg.getImageUrl();
                    final String label = msg.getMessageText() != null
                        ? msg.getMessageText() : "■ Voice message";
                    if (tvMediaLabel != null) tvMediaLabel.setText("■ Loading...");
                    activity.playVoiceMessage(audioUrl,
                        () -> { // onStart
                            currentlyPlayingUrl = audioUrl;
                            if (tvMediaLabel != null)
                                tvMediaLabel.setText("■ " + label);
                        },
                        () -> { // onStop
                            currentlyPlayingUrl = null;
                            if (tvMediaLabel != null)
                                tvMediaLabel.setText("■ " + label);
                        }
                    );
                });
            }
            break;
        case "document":
            tvMessage.setVisibility(View.GONE);
            if (ivImage != null) ivImage.setVisibility(View.GONE);
            if (layoutMedia != null) {
                layoutMedia.setVisibility(View.VISIBLE);
                if (tvMediaLabel != null)

```

```

tvMediaLabel.setText(msg.getMessageText() != null
? msg.getMessageText() : "■ Document");
layoutMedia.setOnClickListener(v -> openUrl(msg.getImageUrl()));
}
break;
case "location":
tvMessage.setVisibility(View.VISIBLE);
if (ivImage != null) ivImage.setVisibility(View.GONE);
if (layoutMedia != null) layoutMedia.setVisibility(View.GONE);
tvMessage.setText(msg.getMessageText());
tvMessage.setOnClickListener(v -> {
String text = msg.getMessageText();
if (text != null && text.contains("http"))
openUrl(text.substring(text.indexOf("http")));
});
break;
default:
tvMessage.setVisibility(View.VISIBLE);
if (ivImage != null) ivImage.setVisibility(View.GONE);
if (layoutMedia != null) layoutMedia.setVisibility(View.GONE);
tvMessage.setText(msg.getMessageText());
tvMessage.setOnClickListener(null);
break;
}
}
private void openUrl(String url) {
if (url == null || url.isEmpty()) return;
try {
context.startActivity(new Intent(Intent.ACTION_VIEW, Uri.parse(url))
.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK));
} catch (Exception ignored) {}
}
@Override public int getItemCount() { return messages.size(); }
static class SentHolder extends RecyclerView.ViewHolder {
TextView tvMessage, tvTimestamp, tvMediaLabel;
ImageView ivStatus, ivImage;
View layoutMedia;
SentHolder(@NonNull View v) {
super(v);
tvMessage = v.findViewById(R.id.tv_message);
tvTimestamp = v.findViewById(R.id.tv_timestamp);
ivStatus = v.findViewById(R.id.iv_status);
ivImage = v.findViewById(R.id.iv_image);
layoutMedia = v.findViewById(R.id.layout_media);
tvMediaLabel = v.findViewById(R.id.tv_media_label);
}
}
static class ReceivedHolder extends RecyclerView.ViewHolder {
ImageView ivAvatar, ivImage;
TextView tvMessage, tvTimestamp, tvMediaLabel;
View layoutMedia;
ReceivedHolder(@NonNull View v) {
super(v);
ivAvatar = v.findViewById(R.id.iv_avatar);
tvMessage = v.findViewById(R.id.tv_message);
tvTimestamp = v.findViewById(R.id.tv_timestamp);
ivImage = v.findViewById(R.id.iv_image);
layoutMedia = v.findViewById(R.id.layout_media);
tvMediaLabel = v.findViewById(R.id.tv_media_label);
}
}
}
}

```


LoginPagerAdapter

ViewPager2 adapter for Login screen tabs (Email and Phone fragments).

LoginPagerAdapter.java (Java)

```
package com.example.joblinker.adapters;
import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentActivity;
import androidx.viewpager2.adapter.FragmentStateAdapter;
import com.example.joblinker.fragments.LoginEmailFragment;
import com.example.joblinker.fragments.LoginPhoneFragment;
/**
 * ViewPager2 adapter for login tabs (Email and Phone)
 */
public class LoginPagerAdapter extends FragmentStateAdapter {
    private static final int NUM_PAGES = 2;
    // Tab positions
    public static final int TAB_EMAIL = 0;
    public static final int TAB_PHONE = 1;
    /**
     * Constructor
     * @param fragmentActivity The parent activity
     */
    public LoginPagerAdapter(@NonNull FragmentActivity fragmentActivity) {
        super(fragmentActivity);
    }
    /**
     * Create fragment for each tab position
     */
    @NonNull
    @Override
    public Fragment createFragment(int position) {
        switch (position) {
            case TAB_EMAIL:
                return new LoginEmailFragment();
            case TAB_PHONE:
                return new LoginPhoneFragment();
            default:
                return new LoginEmailFragment();
        }
    }
    /**
     * Get total number of pages/tabs
     */
    @Override
    public int getItemCount() {
        return NUM_PAGES;
    }
}
```

RegisterPagerAdapter

ViewPager2 adapter for Register wizard (3 step fragments).

RegisterPagerAdapter.java (Java)

```
package com.example.joblinker.adapters;
import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentActivity;
import androidx.viewpager2.adapter.FragmentStateAdapter;
import com.example.joblinker.fragments.RegisterStep1Fragment;
import com.example.joblinker.fragments.RegisterStep2Fragment;
import com.example.joblinker.fragments.RegisterStep3Fragment;
/**
 * ViewPager2 adapter for registration steps
 */
public class RegisterPagerAdapter extends FragmentStateAdapter {
    private static final int NUM_STEPS = 3;
    // Step positions
    public static final int STEP_1 = 0;
    public static final int STEP_2 = 1;
    public static final int STEP_3 = 2;
    /**
     * Constructor
     * @param fragmentActivity The parent activity
     */
    public RegisterPagerAdapter(@NonNull FragmentActivity fragmentActivity) {
        super(fragmentActivity);
    }
    /**
     * Create fragment for each step
     */
    @NonNull
    @Override
    public Fragment createFragment(int position) {
        switch (position) {
            case STEP_1:
                return new RegisterStep1Fragment();
            case STEP_2:
                return new RegisterStep2Fragment();
            case STEP_3:
                return new RegisterStep3Fragment();
            default:
                return new RegisterStep1Fragment();
        }
    }
    /**
     * Get total number of steps
     */
    @Override
    public int getItemCount() {
        return NUM_STEPS;
    }
}
```


Source Code

Services, Receivers & Application

JobLinkerApplication

Custom Application subclass. Declared in AndroidManifest.xml as android:name. Initializes global app state and Firebase on app start.

JobLinkerApplication.java (Java)

```
package com.example.joblinker;
import android.app.Application;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.os.Build;
import android.util.Log;
import com.google.firebase.FirebaseApp;
public class JobLinkerApplication extends Application {
    private static final String TAG = "JobLinkerApp";
    // Notification Channels
    public static final String CHANNEL_MESSAGES = "messages_channel";
    public static final String CHANNEL_JOBS = "jobs_channel";
    public static final String CHANNEL_CALLS = "calls_channel";
    public static final String CHANNEL_GENERAL = "general_channel";
    @Override
    public void onCreate() {
        super.onCreate();
        Log.d(TAG, "JobLinker Application started");
        // Initialize Firebase
        FirebaseApp.initializeApp(this);
        // Create notification channels
        createNotificationChannels();
        // Initialize Glide (optional configuration)
        // GlideApp is auto-generated
    }
    /**
     * Create notification channels for Android O and above
     */
    private void createNotificationChannels() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationManager manager = getSystemService(NotificationManager.class);
            if (manager != null) {
                // Messages Channel
                NotificationChannel messagesChannel = new NotificationChannel(
                    CHANNEL_MESSAGES,
                    "Messages",
                    NotificationManager.IMPORTANCE_HIGH
                );
                messagesChannel.setDescription("New message notifications");
                messagesChannel.enableVibration(true);
                messagesChannel.setShowBadge(true);
                manager.createNotificationChannel(messagesChannel);
                // Jobs Channel
                NotificationChannel jobsChannel = new NotificationChannel(
                    CHANNEL_JOBS,
                    "Job Alerts",
                    NotificationManager.IMPORTANCE_DEFAULT
                );
                jobsChannel.setDescription("New job posting notifications");
                jobsChannel.enableVibration(true);
                jobsChannel.setShowBadge(true);
                manager.createNotificationChannel(jobsChannel);
                // Calls Channel
            }
        }
    }
}
```

```
NotificationChannel callsChannel = new NotificationChannel(
    CHANNEL_CALLS,
    "Calls",
    NotificationManager.IMPORTANCE_HIGH
);
callsChannel.setDescription("Incoming call notifications");
callsChannel.enableVibration(true);
callsChannel.setShowBadge(true);
manager.createNotificationChannel(callsChannel);
// General Channel
NotificationChannel generalChannel = new NotificationChannel(
    CHANNEL_GENERAL,
    "General",
    NotificationManager.IMPORTANCE_DEFAULT
);
generalChannel.setDescription("General app notifications");
manager.createNotificationChannel(generalChannel);
Log.d(TAG, "Notification channels created");
}
}
}
}
```

CallService

Foreground Service for active voice/video calls. Required on Android 12+ (API 31) to keep the call alive when the app is in the background. Shows a persistent notification with call status.

CallService.java (Java)

```
package com.example.joblinker.services;
import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.app.Service;
import android.content.Intent;
import android.os.Build;
import android.os.IBinder;
import androidx.annotation.Nullable;
import androidx.core.app.NotificationCompat;
import com.example.joblinker.R;
import com.example.joblinker.activities.CallActivity;
/**
 * Foreground service for managing voice/video calls
 */
public class CallService extends Service {
    private static final String TAG = "CallService";
    private static final int NOTIFICATION_ID = 1001;
    private static final String CHANNEL_ID = "call_service_channel";
    public static final String ACTION_START_CALL = "com.joblinker.START_CALL";
    public static final String ACTION_END_CALL = "com.joblinker.END_CALL";
    public static final String ACTION_ANSWER_CALL = "com.joblinker.ANSWER_CALL";
    public static final String ACTION_DECLINE_CALL = "com.joblinker.DECLINE_CALL";
    public static final String EXTRA_CALL_ID = "call_id";
    public static final String EXTRA_CALLER_ID = "caller_id";
    public static final String EXTRA_CALLER_NAME = "caller_name";
    public static final String EXTRA_CALL_TYPE = "call_type";
    private String currentCallId;
    private String callType;
    @Override
    public void onCreate() {
        super.onCreate();
        createNotificationChannel();
    }
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        if (intent == null) {
            stopSelf();
            return START_NOT_STICKY;
        }
        String action = intent.getAction();
        if (ACTION_START_CALL.equals(action)) {
            handleStartCall(intent);
        } else if (ACTION_END_CALL.equals(action)) {
            handleEndCall();
        } else if (ACTION_ANSWER_CALL.equals(action)) {
            handleAnswerCall(intent);
        } else if (ACTION_DECLINE_CALL.equals(action)) {
            handleDeclineCall(intent);
        }
        return START_NOT_STICKY;
    }
}
```

```

private void handleStartCall(Intent intent) {
    currentCallId = intent.getStringExtra(EXTRA_CALL_ID);
    String callerName = intent.getStringExtra(EXTRA_CALLER_NAME);
    callType = intent.getStringExtra(EXTRA_CALL_TYPE);
    // Start foreground service with notification
    Notification notification = createCallNotification(callerName, "Ongoing call...");
    startForeground(NOTIFICATION_ID, notification);
}
private void handleEndCall() {
    // Clean up and stop service
    stopForeground(true);
    stopSelf();
}
private void handleAnswerCall(Intent intent) {
    String callId = intent.getStringExtra(EXTRA_CALL_ID);
    String callerName = intent.getStringExtra(EXTRA_CALLER_NAME);
    String callerId = intent.getStringExtra(EXTRA_CALLER_ID); // may be null for older [...]
    // Launch CallActivity with full caller info so CallActivity can save the call recor [...]
    Intent callIntent = new Intent(this, CallActivity.class);
    callIntent.putExtra(CallActivity.EXTRA_CALL_TYPE, callType);
    callIntent.putExtra(CallActivity.EXTRA_RECEIVER_NAME, callerName);
    if (callerId != null)
        callIntent.putExtra(CallActivity.EXTRA_RECEIVER_ID, callerId);
    callIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
    startActivity(callIntent);
    // Update notification
    Notification notification = createCallNotification(callerName, "Call in progress..." [...])
    NotificationManager manager = (NotificationManager) getSystemService(NOTIFICATION_SE [...])
    if (manager != null) {
        manager.notify(NOTIFICATION_ID, notification);
    }
}
private void handleDeclineCall(Intent intent) {
    // Decline the call and stop service
    stopForeground(true);
    stopSelf();
}
private Notification createCallNotification(String callerName, String contentText) {
    Intent notificationIntent = new Intent(this, CallActivity.class);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        this,
        0,
        notificationIntent,
        PendingIntent.FLAG_IMMUTABLE
    );
    // End Call Action
    Intent endCallIntent = new Intent(this, CallService.class);
    endCallIntent.setAction(ACTION_END_CALL);
    PendingIntent endCallPendingIntent = PendingIntent.getService(
        this,
        1,
        endCallIntent,
        PendingIntent.FLAG_IMMUTABLE
    );
    NotificationCompat.Builder builder = new NotificationCompat.Builder(this, CHANNEL_ID [...])
        .setSmallIcon(R.drawable.ic_phone)
        .setContentTitle(callerName != null ? callerName : "JobLinker Call")
        .setContentText(contentText)
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setCategory(NotificationCompat.CATEGORY_CALL)
        .setOngoing(true)
        .setContentIntent(pendingIntent)
        .addAction(R.drawable.ic_call_end, "End Call", endCallPendingIntent);
    return builder.build();
}

```



```
}  
private void createNotificationChannel() {  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {  
        NotificationChannel channel = new NotificationChannel(  
            CHANNEL_ID,  
            "Call Service",  
            NotificationManager.IMPORTANCE_HIGH  
        );  
        channel.setDescription("Ongoing voice and video calls");  
        channel.enableVibration(true);  
        channel.setShowBadge(false);  
        NotificationManager manager = getSystemService(NotificationManager.class);  
        if (manager != null) {  
            manager.createNotificationChannel(channel);  
        }  
    }  
}  
@Nullable  
@Override  
public IBinder onBind(Intent intent) {  
    return null;  
}  
@Override  
public void onDestroy() {  
    super.onDestroy();  
    // Clean up resources  
}  
}
```

JobLinkerFirebaseMessagingServices

Extends `FirebaseMessagingService`. Receives FCM push notifications for new messages, job alerts, and call requests. Displays notifications using `Android NotificationManager` with custom channels.

JobLinkerFirebaseMessagingServices.java (Java)

```
package com.example.joblinker.services;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Build;
import android.util.Log;
import androidx.annotation.NonNull;
import androidx.core.app.NotificationCompat;
import com.google.firebase.messaging.FirebaseMessagingService;
import com.google.firebase.messaging.RemoteMessage;
import com.example.joblinker.R;
import com.example.joblinker.activities.MainActivity;
public class JobLinkerFirebaseMessagingServices extends FirebaseMessagingService {
    private static final String TAG = "FCMService";
    private static final String CHANNEL_ID = "joblinker_notifications";
    @Override
    public void onMessageReceived(@NonNull RemoteMessage remoteMessage) {
        super.onMessageReceived(remoteMessage);
        Log.d(TAG, "From: " + remoteMessage.getFrom());
        // Check if message contains a notification payload
        if (remoteMessage.getNotification() != null) {
            String title = remoteMessage.getNotification().getTitle();
            String body = remoteMessage.getNotification().getBody();
            sendNotification(title, body);
        }
        // Check if message contains a data payload
        if (remoteMessage.getData().size() > 0) {
            Log.d(TAG, "Message data payload: " + remoteMessage.getData());
            // Handle data payload here
        }
    }
    @Override
    public void onNewToken(@NonNull String token) {
        super.onNewToken(token);
        Log.d(TAG, "Refreshed token: " + token);
        // Send token to your server if needed
    }
    private void sendNotification(String title, String messageBody) {
        Intent intent = new Intent(this, MainActivity.class);
        intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
        PendingIntent pendingIntent = PendingIntent.getActivity(this, 0, intent,
            PendingIntent.FLAG_IMMUTABLE);
        NotificationCompat.Builder notificationBuilder =
            new NotificationCompat.Builder(this, CHANNEL_ID)
                .setSmallIcon(R.drawable.ic_notification)
                .setContentTitle(title)
                .setContentText(messageBody)
                .setAutoCancel(true)
                .setContentIntent(pendingIntent)
                .setPriority(NotificationCompat.PRIORITY_HIGH);
        NotificationManager notificationManager =
            (NotificationManager) getSystemService(NOTIFICATION_SERVICE);
```

```
// Create notification channel for Android 0+
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
    NotificationChannel channel = new NotificationChannel(
        CHANNEL_ID,
        "JobLinker Notifications",
        NotificationManager.IMPORTANCE_HIGH
    );
    notificationManager.createNotificationChannel(channel);
}
notificationManager.notify(0, notificationBuilder.build());
}
```

NetworkChangeReceiver

BroadcastReceiver monitoring network connectivity changes (Wi-Fi / Mobile Data connected or disconnected). Updates the UI to warn the user when offline.

NetworkChangeReceiver.java (Java)

```
package com.example.joblinker.receivers;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.net.ConnectivityManager;
import android.net.Network;
import android.net.NetworkCapabilities;
import android.net.NetworkInfo;
import android.os.Build;
import android.util.Log;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.utils.SharedPreferencesManager;
/**
 * Receiver for network connectivity changes
 * Updates user online status based on network availability
 */
public class NetworkChangeReceiver extends BroadcastReceiver {
    private static final String TAG = "NetworkChangeReceiver";
    @Override
    public void onReceive(Context context, Intent intent) {
        if (intent == null || intent.getAction() == null) {
            return;
        }
        if (ConnectivityManager.CONNECTIVITY_ACTION.equals(intent.getAction())) {
            boolean isConnected = isNetworkAvailable(context);
            Log.d(TAG, "Network connectivity changed. Connected: " + isConnected);
            handleNetworkChange(context, isConnected);
        }
    }
    /**
     * Check if network is available
     */
    private boolean isNetworkAvailable(Context context) {
        ConnectivityManager connectivityManager =
            (ConnectivityManager) context.getSystemService(Context.CONNECTIVITY_SERVICE) [...];
        if (connectivityManager == null) {
            return false;
        }
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            Network network = connectivityManager.getActiveNetwork();
            if (network == null) {
                return false;
            }
            NetworkCapabilities capabilities = connectivityManager.getNetworkCapabilities(ne [...];
            return capabilities != null && (
                capabilities.hasTransport(NetworkCapabilities.TRANSPORT_WIFI) ||
                capabilities.hasTransport(NetworkCapabilities.TRANSPORT_CELLULAR [...];
                capabilities.hasTransport(NetworkCapabilities.TRANSPORT_ETHERNET [...];
            );
        } else {
            NetworkInfo networkInfo = connectivityManager.getActiveNetworkInfo();
            return networkInfo != null && networkInfo.isConnected();
        }
    }
}
```

```
}
/**
 * Handle network connectivity changes
 */
private void handleNetworkChange(Context context, boolean isConnected) {
    SharedPreferencesManager prefsManager = SharedPreferencesManager.getInstance(context [...])
    // Only update status if user is logged in
    if (!prefsManager.isLoggedIn()) {
        return;
    }
    String userId = prefsManager.getUserId();
    if (userId == null) {
        return;
    }
    JobLinkerFirebaseManager firebaseManager = JobLinkerFirebaseManager.getInstance();
    if (isConnected) {
        // Network is available - set user online
        firebaseManager.updateUserOnlineStatus(userId, true);
        Log.d(TAG, "User set to online: " + userId);
        // Resume any pending operations
        // For example: sync messages, upload pending data, etc.
    } else {
        // Network is unavailable - set user offline
        firebaseManager.updateUserOnlineStatus(userId, false);
        Log.d(TAG, "User set to offline: " + userId);
        // Handle offline mode
        // For example: queue pending operations, show offline indicator, etc.
    }
}
}
```

BootReceiver

BroadcastReceiver for BOOT_COMPLETED. Re-registers any pending notifications or alarms after the device restarts.

BootReceiver.java (Java)

```
package com.example.joblinker.receivers;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.util.Log;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import com.example.joblinker.utils.SharedPreferencesManager;
/**
 * Receiver for device boot events
 * Used to initialize app services after device restart
 */
public class BootReceiver extends BroadcastReceiver {
    private static final String TAG = "BootReceiver";
    @Override
    public void onReceive(Context context, Intent intent) {
        if (intent == null || intent.getAction() == null) {
            return;
        }
        if (Intent.ACTION_BOOT_COMPLETED.equals(intent.getAction())) {
            Log.d(TAG, "Device boot completed");
            // Initialize app after boot
            initializeApp(context);
        }
        private void initializeApp(Context context) {
            // Note: do NOT set the user online here – the app is not open after a reboot,
            // only the OS fired a broadcast. Online status is set in MainActivity/onResume.
            Log.d(TAG, "Boot completed – skipping online-status update (app not yet open)");
        }
    }
}
```


Source Code

Utilities – Helper Classes

ValidationHelper

Provides static methods to validate email format, phone number, full name, and password strength.

ValidationHelper.java (Java)

```
package com.example.joblinker.utils;
import android.text.TextUtils;
import android.util.Patterns;
public class ValidationHelper {
/**
 * Validate email address
 */
public static boolean isValidEmail(String email) {
return !TextUtils.isEmpty(email) && Patterns.EMAIL_ADDRESS.matcher(email).matches();
}
/**
 * Validate phone number
 */
public static boolean isValidPhone(String phone) {
return !TextUtils.isEmpty(phone) && Patterns.PHONE.matcher(phone).matches() && phone [...];
}
/**
 * Validate password
 */
public static boolean isValidPassword(String password) {
return !TextUtils.isEmpty(password) && password.length() >= 6;
}
/**
 * Validate URL
 */
public static boolean isValidUrl(String url) {
return !TextUtils.isEmpty(url) && Patterns.WEB_URL.matcher(url).matches();
}
/**
 * Check if string is empty
 */
public static boolean isEmpty(String text) {
return TextUtils.isEmpty(text) || text.trim().isEmpty();
}
/**
 * Validate passwords match
 */
public static boolean doPasswordsMatch(String password, String confirmPassword) {
return !TextUtils.isEmpty(password) && password.equals(confirmPassword);
}
/**
 * Validate name
 */
public static boolean isValidName(String name) {
return !TextUtils.isEmpty(name) && name.trim().length() >= 2;
}
/**
 * Validate number
 */
public static boolean isValidNumber(String number) {
if (TextUtils.isEmpty(number)) return false;
try {
Double.parseDouble(number);
return true;
}
```

```
    } catch (NumberFormatException e) {  
        return false;  
    }  
}  
}
```

ImageUtils

Wraps Glide for loading circular and standard profile images from URLs. Handles null/empty URL gracefully.

ImageUtils.java (Java)

```
package com.example.joblinker.utils;
import android.content.Context;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.net.Uri;
import android.widget.ImageView;
import com.bumptech.glide.Glide;
import com.bumptech.glide.load.engine.DiskCacheStrategy;
import com.bumptech.glide.request.RequestOptions;
import com.example.joblinker.R;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
public class ImageUtils {
    /**
     * Load image with Glide
     */
    public static void loadImage(Context context, String imageUrl, ImageView imageView) {
        if (context == null || imageView == null) return;
        Glide.with(context)
            .load(imageUrl)
            .placeholder(R.drawable.ic_person_placeholder)
            .error(R.drawable.ic_person_placeholder)
            .diskCacheStrategy(DiskCacheStrategy.ALL)
            .into(imageView);
    }
    /**
     * Load circular image – always fetches fresh copy, no disk cache
     * so profile photo updates are visible immediately after save.
     */
    public static void loadCircularImage(Context context, String imageUrl, ImageView imageVi [...]
        if (context == null || imageView == null) return;
        Glide.with(context)
            .load(imageUrl)
            .apply(new RequestOptions()
                .circleCrop()
                .placeholder(R.drawable.ic_person_placeholder)
                .error(R.drawable.ic_person_placeholder)
                .diskCacheStrategy(DiskCacheStrategy.NONE) // never cache to disk
                .skipMemoryCache(true)) // never cache in mem [...]
            .into(imageView);
    }
    /**
     * Load circular image with disk cache (use for conversation/job lists
     * where the image rarely changes and speed matters more).
     */
    public static void loadCircularImageCached(Context context, String imageUrl, ImageView i [...]
        if (context == null || imageView == null) return;
        RequestOptions options = new RequestOptions()
            .circleCrop()
            .placeholder(R.drawable.ic_person_placeholder)
            .error(R.drawable.ic_person_placeholder)
```

```

        .diskCacheStrategy(DiskCacheStrategy.ALL);
        Glide.with(context)
            .load(imageUrl)
            .apply(options)
            .into(imageView);
    }
    /**
     * Load company logo
     */
    public static void loadCompanyLogo(Context context, String imageUrl, ImageView imageView [...])
    if (context == null || imageView == null) return;
    Glide.with(context)
        .load(imageUrl)
        .placeholder(R.drawable.ic_company_placeholder)
        .error(R.drawable.ic_company_placeholder)
        .diskCacheStrategy(DiskCacheStrategy.ALL)
        .into(imageView);
    }
    /**
     * Compress image
     */
    public static File compressImage(Context context, Uri imageUrl) throws IOException {
        // Decode inside try-with-resources so the stream is always closed,
        // even if BitmapFactory.decodeStream() throws
        Bitmap original;
        try (InputStream inputStream = context.getContentResolver().openInputStream(imageUrl [...])
            original = BitmapFactory.decodeStream(inputStream);
        }
        if (original == null) throw new IOException("Could not decode image");
        // Resize if too large
        int maxWidth = 1024, maxHeight = 1024;
        Bitmap bitmap;
        if (original.getWidth() > maxWidth || original.getHeight() > maxHeight) {
            float scale = Math.min(
                (float) maxWidth / original.getWidth(),
                (float) maxHeight / original.getHeight());
            bitmap = Bitmap.createScaledBitmap(original,
                Math.round(original.getWidth() * scale),
                Math.round(original.getHeight() * scale), true);
            original.recycle(); // ■ free original after scaling
        } else {
            bitmap = original;
        }
        // Compress to JPEG
        File tempFile = new File(context.getCacheDir(),
            "compressed_" + System.currentTimeMillis() + ".jpg");
        try (ByteArrayOutputStream bos = new ByteArrayOutputStream();
            FileOutputStream fos = new FileOutputStream(tempFile)) {
            bitmap.compress(Bitmap.CompressFormat.JPEG, 80, bos);
            fos.write(bos.toByteArray());
        } finally {
            if (bitmap != original) bitmap.recycle();
        }
        return tempFile;
    }
    /**
     * Get file from URI
     */
    public static File getFileFromUri(Context context, Uri uri) throws IOException {
        InputStream inputStream = context.getContentResolver().openInputStream(uri);
        File tempFile = new File(context.getCacheDir(), "temp_" + System.currentTimeMillis() [...])
        FileOutputStream outputStream = new FileOutputStream(tempFile);
        byte[] buffer = new byte[1024];
        int length;

```

```
if (inputStream != null) {  
    while ((length = inputStream.read(buffer)) > 0) {  
        outputStream.write(buffer, 0, length);  
    }  
    inputStream.close();  
}  
outputStream.close();  
return tempFile;  
}  
}
```

DateTimeHelper

Formats timestamps as relative time strings ('just now', '5 min ago', etc.) and absolute date strings.

DateTimeHelper.java (Java)

```
package com.example.joblinker.utils;
import android.text.format.DateUtils;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.Date;
import java.util.Locale;
import java.util.TimeZone;
import java.util.concurrent.TimeUnit;

public class DateTimeHelper {
    private static final String TAG = "DateTimeHelper";

    // Date Format Patterns
    private static final String PATTERN_TIME_12H = "h:mm a";
    private static final String PATTERN_TIME_24H = "HH:mm";
    private static final String PATTERN_DATE_SHORT = "MMM dd, yyyy";
    private static final String PATTERN_DATE_MEDIUM = "MMMM dd, yyyy";
    private static final String PATTERN_DATE_LONG = "EEEE, MMMM dd, yyyy";
    private static final String PATTERN_DATETIME = "MMM dd, yyyy h:mm a";
    private static final String PATTERN_DATETIME_FULL = "EEEE, MMMM dd, yyyy h:mm a";
    private static final String PATTERN_DAY_NAME = "EEEE";
    private static final String PATTERN_MONTH_YEAR = "MMMM yyyy";
    private static final String PATTERN_ISO_8601 = "yyyy-MM-dd'T'HH:mm:ss.SSS'Z'";

    /**
     * Format timestamp to time string (e.g., "2:30 PM")
     */
    public static String formatTime(long timestamp) {
        SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_TIME_12H, Locale.getDefault());
        return sdf.format(new Date(timestamp));
    }

    /**
     * Format timestamp to 24-hour time string (e.g., "14:30")
     */
    public static String formatTime24H(long timestamp) {
        SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_TIME_24H, Locale.getDefault());
        return sdf.format(new Date(timestamp));
    }

    /**
     * Format timestamp to date string (e.g., "Jan 15, 2025")
     */
    public static String formatDate(long timestamp) {
        SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_DATE_SHORT, Locale.getDefault()) [...]
        return sdf.format(new Date(timestamp));
    }

    /**
     * Format timestamp to medium date string (e.g., "January 15, 2025")
     */
    public static String formatDateMedium(long timestamp) {
        SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_DATE_MEDIUM, Locale.getDefault()) [...]
        return sdf.format(new Date(timestamp));
    }

    /**
     * Format timestamp to long date string (e.g., "Monday, January 15, 2025")
     */
    public static String formatDateLong(long timestamp) {
```

```

SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_DATE_LONG, Locale.getDefault());
return sdf.format(new Date(timestamp));
}
/**
 * Format timestamp to datetime string (e.g., "Jan 15, 2025 2:30 PM")
 */
public static String formatDateTime(long timestamp) {
SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_DATETIME, Locale.getDefault());
return sdf.format(new Date(timestamp));
}
/**
 * Format timestamp to full datetime string
 */
public static String formatDateTimeFull(long timestamp) {
SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_DATETIME_FULL, Locale.getDefault [...])
return sdf.format(new Date(timestamp));
}
/**
 * Get relative time string (e.g., "2 hours ago", "Just now", "Yesterday")
 */
public static String getRelativeTime(long timestamp) {
long now = System.currentTimeMillis();
long diff = now - timestamp;
// Just now (less than 1 minute)
if (diff < TimeUnit.MINUTES.toMillis(1)) {
return "Just now";
}
// Minutes ago
else if (diff < TimeUnit.HOURS.toMillis(1)) {
long minutes = TimeUnit.MILLISECONDS.toMinutes(diff);
return minutes + (minutes == 1 ? " minute ago" : " minutes ago");
}
// Hours ago
else if (diff < TimeUnit.DAYS.toMillis(1)) {
long hours = TimeUnit.MILLISECONDS.toHours(diff);
return hours + (hours == 1 ? " hour ago" : " hours ago");
}
// Days ago (up to 7 days)
else if (diff < TimeUnit.DAYS.toMillis(7)) {
long days = TimeUnit.MILLISECONDS.toDays(diff);
if (days == 1) {
return "Yesterday";
}
return days + " days ago";
}
// Weeks ago (up to 4 weeks)
else if (diff < TimeUnit.DAYS.toMillis(30)) {
long weeks = TimeUnit.MILLISECONDS.toDays(diff) / 7;
return weeks + (weeks == 1 ? " week ago" : " weeks ago");
}
// Months ago (up to 12 months)
else if (diff < TimeUnit.DAYS.toMillis(365)) {
long months = TimeUnit.MILLISECONDS.toDays(diff) / 30;
return months + (months == 1 ? " month ago" : " months ago");
}
// Years ago
else {
long years = TimeUnit.MILLISECONDS.toDays(diff) / 365;
return years + (years == 1 ? " year ago" : " years ago");
}
}
/**
 * Get relative time for future dates (e.g., "in 2 hours", "in 3 days")
 */

```

```

public static String getRelativeTimeFuture(long timestamp) {
    long now = System.currentTimeMillis();
    long diff = timestamp - now;
    if (diff <= 0) {
        return "Now";
    }
    // Minutes
    if (diff < TimeUnit.HOURS.toMillis(1)) {
        long minutes = TimeUnit.MILLISECONDS.toMinutes(diff);
        return "in " + minutes + (minutes == 1 ? " minute" : " minutes");
    }
    // Hours
    else if (diff < TimeUnit.DAYS.toMillis(1)) {
        long hours = TimeUnit.MILLISECONDS.toHours(diff);
        return "in " + hours + (hours == 1 ? " hour" : " hours");
    }
    // Days
    else if (diff < TimeUnit.DAYS.toMillis(7)) {
        long days = TimeUnit.MILLISECONDS.toDays(diff);
        if (days == 1) {
            return "Tomorrow";
        }
        return "in " + days + " days";
    }
    // Weeks
    else if (diff < TimeUnit.DAYS.toMillis(30)) {
        long weeks = TimeUnit.MILLISECONDS.toDays(diff) / 7;
        return "in " + weeks + (weeks == 1 ? " week" : " weeks");
    }
    // Months
    else if (diff < TimeUnit.DAYS.toMillis(365)) {
        long months = TimeUnit.MILLISECONDS.toDays(diff) / 30;
        return "in " + months + (months == 1 ? " month" : " months");
    }
    // Years
    else {
        long years = TimeUnit.MILLISECONDS.toDays(diff) / 365;
        return "in " + years + (years == 1 ? " year" : " years");
    }
}
/**
 * Format call duration from seconds (e.g., "02:45" or "01:30:15")
 */
public static String formatCallDuration(long seconds) {
    long hours = seconds / 3600;
    long minutes = (seconds % 3600) / 60;
    long secs = seconds % 60;
    if (hours > 0) {
        return String.format(Locale.getDefault(), "%02d:%02d:%02d", hours, minutes, secs [...])
    } else {
        return String.format(Locale.getDefault(), "%02d:%02d", minutes, secs);
    }
}
/**
 * Format duration from milliseconds (e.g., "2h 30m" or "45m 30s")
 */
public static String formatDuration(long milliseconds) {
    long seconds = milliseconds / 1000;
    long minutes = seconds / 60;
    long hours = minutes / 60;
    long days = hours / 24;
    if (days > 0) {
        hours = hours % 24;
        return days + "d " + hours + "h";
    }
}

```



```

    } else if (hours > 0) {
        minutes = minutes % 60;
        return hours + "h " + minutes + "m";
    } else if (minutes > 0) {
        seconds = seconds % 60;
        return minutes + "m " + seconds + "s";
    } else {
        return seconds + "s";
    }
}

/**
 * Get message timestamp display format
 * - If today: show time (e.g., "2:30 PM")
 * - If yesterday: show "Yesterday"
 * - If this week: show day name (e.g., "Monday")
 * - Otherwise: show date (e.g., "Jan 15")
 */
public static String getMessageTimestamp(long timestamp) {
    long now = System.currentTimeMillis();
    long diff = now - timestamp;
    // If today, show time
    if (isToday(timestamp)) {
        return formatTime(timestamp);
    }
    // If yesterday
    else if (isYesterday(timestamp)) {
        return "Yesterday";
    }
    // If within a week, show day name
    else if (diff < TimeUnit.DAYS.toMillis(7)) {
        SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_DAY_NAME, Locale.getDefault( [...])
        return sdf.format(new Date(timestamp));
    }
    // Otherwise show date
    else {
        return formatDate(timestamp);
    }
}

/**
 * Get conversation timestamp (similar to WhatsApp format)
 */
public static String getConversationTimestamp(long timestamp) {
    if (isToday(timestamp)) {
        return formatTime(timestamp);
    } else if (isYesterday(timestamp)) {
        return "Yesterday";
    } else if (isThisWeek(timestamp)) {
        SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_DAY_NAME, Locale.getDefault( [...])
        return sdf.format(new Date(timestamp));
    } else if (isThisYear(timestamp)) {
        SimpleDateFormat sdf = new SimpleDateFormat("MMM dd", Locale.getDefault());
        return sdf.format(new Date(timestamp));
    } else {
        SimpleDateFormat sdf = new SimpleDateFormat("MMM dd, yyyy", Locale.getDefault()) [...]
        return sdf.format(new Date(timestamp));
    }
}

/**
 * Check if timestamp is today
 */
public static boolean isToday(long timestamp) {
    Calendar calendar = Calendar.getInstance();
    calendar.setTimeInMillis(timestamp);
    Calendar today = Calendar.getInstance();

```

```

return calendar.get(Calendar.YEAR) == today.get(Calendar.YEAR) &&
calendar.get(Calendar.DAY_OF_YEAR) == today.get(Calendar.DAY_OF_YEAR);
}
/**
 * Check if timestamp is yesterday
 */
public static boolean isYesterday(long timestamp) {
    Calendar calendar = Calendar.getInstance();
    calendar.setTimeInMillis(timestamp);
    Calendar yesterday = Calendar.getInstance();
    yesterday.add(Calendar.DAY_OF_YEAR, -1);
    return calendar.get(Calendar.YEAR) == yesterday.get(Calendar.YEAR) &&
    calendar.get(Calendar.DAY_OF_YEAR) == yesterday.get(Calendar.DAY_OF_YEAR);
}
/**
 * Check if timestamp is in this week
 */
public static boolean isThisWeek(long timestamp) {
    Calendar calendar = Calendar.getInstance();
    calendar.setTimeInMillis(timestamp);
    Calendar now = Calendar.getInstance();
    return calendar.get(Calendar.YEAR) == now.get(Calendar.YEAR) &&
    calendar.get(Calendar.WEEK_OF_YEAR) == now.get(Calendar.WEEK_OF_YEAR);
}
/**
 * Check if timestamp is in this year
 */
public static boolean isThisYear(long timestamp) {
    Calendar calendar = Calendar.getInstance();
    calendar.setTimeInMillis(timestamp);
    Calendar now = Calendar.getInstance();
    return calendar.get(Calendar.YEAR) == now.get(Calendar.YEAR);
}
/**
 * Get day name from timestamp (e.g., "Monday")
 */
public static String getDayName(long timestamp) {
    SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_DAY_NAME, Locale.getDefault());
    return sdf.format(new Date(timestamp));
}
/**
 * Get month and year (e.g., "January 2025")
 */
public static String getMonthYear(long timestamp) {
    SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_MONTH_YEAR, Locale.getDefault()) [...];
    return sdf.format(new Date(timestamp));
}
/**
 * Get time difference in human readable format
 */
public static String getTimeDifference(long startTime, long endTime) {
    long diff = endTime - startTime;
    if (diff < 0) {
        return "0s";
    }
    return formatDuration(diff);
}
/**
 * Parse date string to timestamp
 */
public static long parseDate(String dateString, String pattern) {
    try {
        SimpleDateFormat sdf = new SimpleDateFormat(pattern, Locale.getDefault());
        Date date = sdf.parse(dateString);
    }
}

```

```

return date != null ? date.getTime() : 0;
} catch (ParseException e) {
e.printStackTrace();
return 0;
}
}
/**
 * Get start of day timestamp
 */
public static long getStartOfDay(long timestamp) {
Calendar calendar = Calendar.getInstance();
calendar.setTimeInMillis(timestamp);
calendar.set(Calendar.HOUR_OF_DAY, 0);
calendar.set(Calendar.MINUTE, 0);
calendar.set(Calendar.SECOND, 0);
calendar.set(Calendar.MILLISECOND, 0);
return calendar.getTimeInMillis();
}
/**
 * Get end of day timestamp
 */
public static long getEndOfDay(long timestamp) {
Calendar calendar = Calendar.getInstance();
calendar.setTimeInMillis(timestamp);
calendar.set(Calendar.HOUR_OF_DAY, 23);
calendar.set(Calendar.MINUTE, 59);
calendar.set(Calendar.SECOND, 59);
calendar.set(Calendar.MILLISECOND, 999);
return calendar.getTimeInMillis();
}
/**
 * Get age from date of birth
 */
public static int getAge(long dateOfBirth) {
Calendar dob = Calendar.getInstance();
dob.setTimeInMillis(dateOfBirth);
Calendar today = Calendar.getInstance();
int age = today.get(Calendar.YEAR) - dob.get(Calendar.YEAR);
if (today.get(Calendar.DAY_OF_YEAR) < dob.get(Calendar.DAY_OF_YEAR)) {
age--;
}
return age;
}
/**
 * Check if date is in the past
 */
public static boolean isPast(long timestamp) {
return timestamp < System.currentTimeMillis();
}
/**
 * Check if date is in the future
 */
public static boolean isFuture(long timestamp) {
return timestamp > System.currentTimeMillis();
}
/**
 * Add days to timestamp
 */
public static long addDays(long timestamp, int days) {
Calendar calendar = Calendar.getInstance();
calendar.setTimeInMillis(timestamp);
calendar.add(Calendar.DAY_OF_YEAR, days);
return calendar.getTimeInMillis();
}
}

```

```

/**
 * Add hours to timestamp
 */
public static long addHours(long timestamp, int hours) {
    return timestamp + TimeUnit.HOURS.toMillis(hours);
}
/**
 * Add minutes to timestamp
 */
public static long addMinutes(long timestamp, int minutes) {
    return timestamp + TimeUnit.MINUTES.toMillis(minutes);
}
/**
 * Get timestamp for specific date
 */
public static long getTimestamp(int year, int month, int day) {
    Calendar calendar = Calendar.getInstance();
    calendar.set(year, month - 1, day, 0, 0, 0);
    calendar.set(Calendar.MILLISECOND, 0);
    return calendar.getTimeInMillis();
}
/**
 * Get timestamp for specific datetime
 */
public static long getTimestamp(int year, int month, int day, int hour, int minute) {
    Calendar calendar = Calendar.getInstance();
    calendar.set(year, month - 1, day, hour, minute, 0);
    calendar.set(Calendar.MILLISECOND, 0);
    return calendar.getTimeInMillis();
}
/**
 * Format timestamp to ISO 8601 format
 */
public static String formatISO8601(long timestamp) {
    SimpleDateFormat sdf = new SimpleDateFormat(PATTERN_ISO_8601, Locale.getDefault());
    sdf.setTimeZone(TimeZone.getTimeZone("UTC"));
    return sdf.format(new Date(timestamp));
}
/**
 * Parse ISO 8601 date string to timestamp
 */
public static long parseISO8601(String dateString) {
    return parseDate(dateString, PATTERN_ISO_8601);
}
/**
 * Get greeting based on time of day
 */
public static String getGreeting() {
    Calendar calendar = Calendar.getInstance();
    int hour = calendar.get(Calendar.HOUR_OF_DAY);
    if (hour >= 5 && hour < 12) {
        return "Good morning";
    } else if (hour >= 12 && hour < 17) {
        return "Good afternoon";
    } else if (hour >= 17 && hour < 21) {
        return "Good evening";
    } else {
        return "Good night";
    }
}
/**
 * Check if two timestamps are on the same day
 */
public static boolean isSameDay(long timestamp1, long timestamp2) {

```

```
Calendar cal1 = Calendar.getInstance();
cal1.setTimeInMillis(timestamp1);
Calendar cal2 = Calendar.getInstance();
cal2.setTimeInMillis(timestamp2);
return cal1.get(Calendar.YEAR) == cal2.get(Calendar.YEAR) &&
cal1.get(Calendar.DAY_OF_YEAR) == cal2.get(Calendar.DAY_OF_YEAR);
}
/**
 * Get current timestamp
 */
public static long now() {
    return System.currentTimeMillis();
}
/**
 * Get timestamp for tomorrow at midnight
 */
public static long getTomorrow() {
    Calendar calendar = Calendar.getInstance();
    calendar.add(Calendar.DAY_OF_YEAR, 1);
    calendar.set(Calendar.HOUR_OF_DAY, 0);
    calendar.set(Calendar.MINUTE, 0);
    calendar.set(Calendar.SECOND, 0);
    calendar.set(Calendar.MILLISECOND, 0);
    return calendar.getTimeInMillis();
}
/**
 * Get timestamp for yesterday at midnight
 */
public static long getYesterday() {
    Calendar calendar = Calendar.getInstance();
    calendar.add(Calendar.DAY_OF_YEAR, -1);
    calendar.set(Calendar.HOUR_OF_DAY, 0);
    calendar.set(Calendar.MINUTE, 0);
    calendar.set(Calendar.SECOND, 0);
    calendar.set(Calendar.MILLISECOND, 0);
    return calendar.getTimeInMillis();
}
}
```

LocaleHelper

Manages app language switching. Applies locale to Activity context using attachBaseContext.

LocaleHelper.java (Java)

```
package com.example.joblinker.utils;
import android.content.Context;
import android.content.SharedPreferences;
import android.content.res.Configuration;
import android.content.res.Resources;
import android.os.Build;
import java.util.Locale;
public class LocaleHelper {
    private static final String SELECTED_LANGUAGE = "Locale.Helper.Selected.Language";
    private static final String PREFS_NAME = "JobLinkerPrefs";
    /**
     * Set the app's language
     */
    public static Context setLocale(Context context, String language) {
        persist(context, language);
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
            return updateResources(context, language);
        }
        return updateResourcesLegacy(context, language);
    }
    /**
     * Get saved language
     */
    public static String getLanguage(Context context) {
        SharedPreferences prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
        return prefs.getString(SELECTED_LANGUAGE, "en"); // Default to English
    }
    /**
     * Save language preference
     */
    private static void persist(Context context, String language) {
        SharedPreferences prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
        prefs.edit().putString(SELECTED_LANGUAGE, language).apply();
    }
    /**
     * Update resources for API 24+
     */
    private static Context updateResources(Context context, String language) {
        Locale locale = getLocaleFromLanguage(language);
        Locale.setDefault(locale);
        Configuration configuration = context.getResources().getConfiguration();
        configuration.setLocale(locale);
        configuration.setLayoutDirection(locale);
        return context.createConfigurationContext(configuration);
    }
    /**
     * Update resources for older APIs
     */
    @SuppressWarnings("deprecation")
    private static Context updateResourcesLegacy(Context context, String language) {
        Locale locale = getLocaleFromLanguage(language);
        Locale.setDefault(locale);
        Resources resources = context.getResources();
        Configuration configuration = resources.getConfiguration();
```

```
configuration.locale = locale;
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.JELLY_BEAN_MR1) {
    configuration.setLayoutDirection(locale);
}
resources.updateConfiguration(configuration, resources.getDisplayMetrics());
return context;
}
/**
 * Convert language name to locale
 */
private static Locale getLocaleFromLanguage(String language) {
    switch (language.toLowerCase()) {
        case "arabic":
        case "ar":
            return new Locale("ar");
        case "hebrew":
        case "he":
            return new Locale("he");
        case "spanish":
        case "es":
            return new Locale("es");
        case "french":
        case "fr":
            return new Locale("fr");
        case "german":
        case "de":
            return new Locale("de");
        case "italian":
        case "it":
            return new Locale("it");
        case "portuguese":
        case "pt":
            return new Locale("pt");
        case "russian":
        case "ru":
            return new Locale("ru");
        case "chinese":
        case "zh":
            return new Locale("zh");
        case "japanese":
        case "ja":
            return new Locale("ja");
        case "korean":
        case "ko":
            return new Locale("ko");
        case "hindi":
        case "hi":
            return new Locale("hi");
        case "dutch":
        case "nl":
            return new Locale("nl");
        case "swedish":
        case "sv":
            return new Locale("sv");
        case "norwegian":
        case "no":
            return new Locale("no");
        case "danish":
        case "da":
            return new Locale("da");
        case "finnish":
        case "fi":
            return new Locale("fi");
        case "polish":
```

```
case "pl":
    return new Locale("pl");
case "turkish":
case "tr":
    return new Locale("tr");
case "english":
case "en":
    default:
        return new Locale("en");
}
}
/**
 * Get language code from full name
 */
public static String getLanguageCode(String language) {
    switch (language.toLowerCase()) {
        case "arabic": return "ar";
        case "hebrew": return "he";
        case "spanish": return "es";
        case "french": return "fr";
        case "german": return "de";
        case "italian": return "it";
        case "portuguese": return "pt";
        case "russian": return "ru";
        case "chinese": return "zh";
        case "japanese": return "ja";
        case "korean": return "ko";
        case "hindi": return "hi";
        case "dutch": return "nl";
        case "swedish": return "sv";
        case "norwegian": return "no";
        case "danish": return "da";
        case "finnish": return "fi";
        case "polish": return "pl";
        case "turkish": return "tr";
        case "english":
        default: return "en";
    }
}
}
```


LogoutHelper

Shows a confirmation dialog before logging out. Clears SharedPreferences and navigates to LoginActivity.

LogoutHelper.java (Java)

```
package com.example.joblinker.utils;
import android.app.Activity;
import android.app.AlertDialog;
import android.content.Context;
import android.content.Intent;
import android.os.Handler;
import android.os.Looper;
import android.widget.Toast;
import com.example.joblinker.R;
import com.example.joblinker.activities.LoginActivity;
import com.example.joblinker.firebase.JobLinkerFirebaseManager;
import java.io.File;
public class LogoutHelper {
/**
 * Show logout confirmation dialog
 */
public static void showLogoutDialog(Activity activity) {
    new AlertDialog.Builder(activity)
        .setTitle(activity.getString(R.string.logout))
        .setMessage(activity.getString(R.string.logout_message))
        .setPositiveButton(activity.getString(R.string.yes), (dialog, which) ->
            performLogout(activity))
        .setNegativeButton(activity.getString(R.string.cancel), null)
        .setCancelable(true)
        .show();
}
/**
 * Perform logout with progress dialog
 */
public static void performLogout(Activity activity) {
    ProgressDialogHelper progressDialog = new ProgressDialogHelper(activity, "Logging ou [...]
    progressDialog.show();
    JobLinkerFirebaseManager firebaseManager = JobLinkerFirebaseManager.getInstance();
    // Read userId BEFORE clearing prefs
    String userId = SharedPreferencesManager.getInstance(activity).getUserId();
    new Handler(Looper.getMainLooper()).postDelayed(() -> {
        // Mark user offline before signing out
        if (userId != null) {
            firebaseManager.updateUserOnlineStatus(userId, false);
        }
        // Full logout – removes all Firestore listeners and signs out of Firebase Auth
        firebaseManager.logout();
        clearSharedPreferences(activity);
        clearCache(activity);
        progressDialog.dismiss();
        Toast.makeText(activity, "Logged out successfully", Toast.LENGTH_SHORT).show();
        navigateToLogin(activity);
    }, 1000);
}
/**
 * Quick logout without dialog
 */
public static void quickLogout(Activity activity) {
    JobLinkerFirebaseManager firebaseManager = JobLinkerFirebaseManager.getInstance();
```

```

String userId = SharedPreferencesManager.getInstance(activity).getUserId();
if (userId != null) firebaseManager.updateUserOnlineStatus(userId, false);
firebaseManager.logout();
clearSharedPreferences(activity);
clearCache(activity);
Toast.makeText(activity, "Logged out", Toast.LENGTH_SHORT).show();
navigateToLogin(activity);
}
/**
 * Clear SharedPreferences
 */
private static void clearSharedPreferences(Context context) {
    context.getSharedPreferences("JobLinkerPrefs", Context.MODE_PRIVATE)
        .edit()
        .clear()
        .apply();
}
/**
 * Clear app cache
 */
private static void clearCache(Context context) {
    try {
        File cache = context.getCacheDir();
        deleteDir(cache);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
/**
 * Recursively delete directory
 */
private static boolean deleteDir(File dir) {
    if (dir != null && dir.isDirectory()) {
        String[] children = dir.list();
        if (children != null) {
            for (String child : children) {
                boolean success = deleteDir(new File(dir, child));
                if (!success) {
                    return false;
                }
            }
        }
        return dir != null && dir.delete();
    }
}
/**
 * Navigate to login and clear activity stack
 */
private static void navigateToLogin(Activity activity) {
    Intent intent = new Intent(activity, LoginActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
    activity.startActivity(intent);
    activity.finish();
}
}

```

SharedPreferencesManager

Singleton manager for user session data: userId, userName, userRole, language, currency, login state.

SharedPreferencesManager.java (Java)

```
package com.example.joblinker.utils;
import android.content.Context;
import android.content.SharedPreferences;
public class SharedPreferencesManager {
    private static final String PREF_NAME = "JobLinkerPrefs";
    private static final String KEY_USER_ID = "userId";
    private static final String KEY_USER_NAME = "userName";
    private static final String KEY_USER_EMAIL = "userEmail";
    private static final String KEY_USER_ROLE = "userRole";
    private static final String KEY_USER_AVATAR = "userAvatar";
    private static final String KEY_LANGUAGE = "language";
    private static final String KEY_CURRENCY = "currency";
    private static final String KEY_IS_LOGGED_IN = "isLoggedIn";
    private static final String KEY_NOTIFICATIONS_ENABLED = "notificationsEnabled";
    private static final String KEY_ONLINE_STATUS_VISIBLE = "onlineStatusVisible";
    private static SharedPreferencesManager instance;
    private SharedPreferences sharedPreferences;
    private SharedPreferences.Editor editor;
    private SharedPreferencesManager(Context context) {
        sharedPreferences = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE);
        editor = sharedPreferences.edit();
    }
    public static synchronized SharedPreferencesManager getInstance(Context context) {
        if (instance == null) {
            instance = new SharedPreferencesManager(context.getApplicationContext());
        }
        return instance;
    }
    // User ID
    public void setUserId(String userId) {
        editor.putString(KEY_USER_ID, userId);
        editor.apply();
    }
    public String getUserId() {
        return sharedPreferences.getString(KEY_USER_ID, null);
    }
    // User Name
    public void setUserName(String userName) {
        editor.putString(KEY_USER_NAME, userName);
        editor.apply();
    }
    public String getUserName() {
        return sharedPreferences.getString(KEY_USER_NAME, "");
    }
    // User Email
    public void setUserEmail(String userEmail) {
        editor.putString(KEY_USER_EMAIL, userEmail);
        editor.apply();
    }
    public String getUserEmail() {
        return sharedPreferences.getString(KEY_USER_EMAIL, "");
    }
    // User Role
    public void setUserRole(String userRole) {
```



```
return sharedPreferences.getBoolean(key, defaultValue);
}
public void saveInt(String key, int value) {
    editor.putInt(key, value);
    editor.apply();
}
public int getInt(String key, int defaultValue) {
    return sharedPreferences.getInt(key, defaultValue);
}
public void saveString(String key, String value) {
    editor.putString(key, value);
    editor.apply();
}
public String getString(String key, String defaultValue) {
    return sharedPreferences.getString(key, defaultValue);
}
// Clear all data
public void clearAll() {
    editor.clear();
    editor.apply();
}
}
```

ProgressDialogHelper

Displays and dismisses a non-cancelable ProgressDialog overlay during long operations (photo upload, job submission, registration). Prevents double-taps.

ProgressDialogHelper.java (Java)

```
package com.example.joblinker.utils;
import android.app.AlertDialog;
import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.ProgressBar;
import android.widget.TextView;
/**
 * A non-deprecated replacement for android.app.ProgressDialog.
 * Uses AlertDialog + ProgressBar instead.
 */
public class ProgressDialogHelper {
    private final AlertDialog dialog;
    public ProgressDialogHelper(Context context, String message) {
        // Build layout programmatically – no extra XML needed
        android.widget.LinearLayout layout = new android.widget.LinearLayout(context);
        layout.setOrientation(android.widget.LinearLayout.HORIZONTAL);
        layout.setGravity(android.view.Gravity.CENTER_VERTICAL);
        int pad = dp(context, 24);
        layout.setPadding(pad, pad, pad, pad);
        ProgressBar progressBar = new ProgressBar(context);
        int size = dp(context, 40);
        android.widget.LinearLayout.LayoutParams pbParams =
            new android.widget.LinearLayout.LayoutParams(size, size);
        progressBar.setLayoutParams(pbParams);
        layout.addView(progressBar);
        TextView tvMessage = new TextView(context);
        android.widget.LinearLayout.LayoutParams tvParams =
            new android.widget.LinearLayout.LayoutParams(
                android.widget.LinearLayout.LayoutParams.WRAP_CONTENT,
                android.widget.LinearLayout.LayoutParams.WRAP_CONTENT);
        tvParams.setMarginStart(dp(context, 20));
        tvMessage.setLayoutParams(tvParams);
        tvMessage.setText(message);
        tvMessage.setTextSize(16f);
        layout.addView(tvMessage);
        dialog = new AlertDialog.Builder(context)
            .setView(layout)
            .setCancelable(false)
            .create();
    }
    public void show() {
        if (!dialog.isShowing()) {
            dialog.show();
        }
    }
    public void dismiss() {
        if (dialog.isShowing()) {
            dialog.dismiss();
        }
    }
    public boolean isShowing() {
        return dialog.isShowing();
    }
}
```

```
}  
private static int dp(Context context, int dp) {  
    return (int) (dp * context.getResources().getDisplayMetrics().density);  
}  
}
```


Conclusion

Self-Reflection & References

Self-Reflection & Conclusion

Personal Reflection

Working on JobLinker for nearly a year was both challenging and deeply rewarding. This project pushed me well beyond the boundaries of what I had studied in the classroom, requiring me to independently research and implement technologies like Firebase Firestore real-time listeners, Agora voice calls, and multi-step form navigation — all while maintaining a consistent, professional user experience.

Challenges Faced

Real-time Chat: Implementing reliable real-time messaging with read receipts and typing indicators required careful Firestore listener management to avoid memory leaks and excessive reads.

Audio Recording: Recording and playing back voice messages involved managing MediaRecorder lifecycle, audio focus, and file uploads to Firebase Storage — each step requiring research and testing.

Role-based UI: Dynamically showing/hiding features based on user role (Employer vs. Job Seeker) required consistent state management throughout the entire app.

Firebase Indexes: Compound Firestore queries required creating composite indexes. Debugging FAILED_PRECONDITION errors taught me to plan data structure and queries together from the start.

Firestore Ordering: Combining orderBy with inequality filters (where + orderBy) required the first orderBy to match the inequality field — a subtle Firestore constraint that took debugging to discover.

Android 12+ Permissions: Strict foreground service requirements on Android 12+ for CallService required adding FOREGROUND_SERVICE permission and proper notification channels.

What I Learned

- End-to-end Android app development from zero to a production-ready application.
- Firebase ecosystem: Authentication, Firestore, Storage, and Cloud Messaging.
- Real-time communication patterns using Firestore listeners and Agora SDK.
- RecyclerView adapter patterns with DiffUtil for performance.
- Fragment lifecycle management and ViewPager2 navigation.
- Material Design principles applied to create a clean, professional UI.
- Debugging complex multi-threaded issues on Android.
- Version control with Git and maintaining a public GitHub repository.

Future Improvements

- Integrate AI-powered job matching that suggests relevant jobs based on the user's skills and experience.
- Add a video resume feature where job seekers can record and attach short video introductions to their applications.

- Implement a scheduling system for interviews directly within the app.
- Add employer analytics dashboard showing application rates, view counts, and conversion metrics.
- Expand to iOS using Kotlin Multiplatform or Flutter.

References

- **Firestore Documentation** — <https://firebase.google.com/docs>
- **Agora RTC SDK for Android** — <https://docs.agora.io/en/voice-calling/get-started/get-started-sdk>
- **Android Developer Guides** — <https://developer.android.com/guide>
- **Material Design 3 Components** — <https://m3.material.io/components>
- **Glide Image Loading** — <https://bumptech.github.io/glide/>
- **RecyclerView with DiffUtil – Android Tutorial** — <https://developer.android.com/reference/androidx/recyclerview/widget/DiffUtil>
- **ViewPager2 Guide** — <https://developer.android.com/guide/navigation/navigation-swipe-view-2>
- **Firestore – Real-time Updates** — <https://firebase.google.com/docs/firestore/query-data/listen>